

MACHINE LEARNING

Question Bank on Introduction to Machine Learning and Machine Learning Models

1 marker

1. What is Machine learning?
 - a) The selective acquisition of knowledge through the use of computer programs
 - b) The selective acquisition of knowledge through the use of manual programs
 - c) The autonomous acquisition of knowledge through the use of computer programs
 - d) The autonomous acquisition of knowledge through the use of manual programs
2. Which of the following is not a supervised machine learning algorithm?
 - a) K-means
 - b) Naïve Bayes
 - c) SVM for classification problems
 - d) Decision tree
3. Which algorithm is best suited for a binary classification problem?
 - a) K-nearest Neighbors
 - b) Decision Trees
 - c) Random Forest
 - d) Linear Regression
4. Which type of data is used in supervised learning?
 - a) Unlabeled data
 - b) Labeled data
 - c) Both
 - d) Neither
5. Which type of machine learning algorithm falls under the category of “unsupervised learning”?
 - a) Linear Regression
 - b) K-means Clustering
 - c) Decision Trees
 - d) Random Forest

6. Which one of the following models is a generative model used in machine learning?
- a) Support vector machines
 - b) Naïve Bayes
 - c) Logistic Regression
 - d) Linear Regression
7. Decision tree uses the inductive learning machine learning approach.
- a) False
 - b) True
8. Which is an example of classification problem?
- a) Predicting house prices
 - b) Diagnosing a disease as positive/negative
 - c) Finding groups in data
 - d) Reducing dimensions
9. Which algorithm can be used for both classification and regression?
- A. KNN
 - B. Linear Regression
 - C. Naive Bayes
 - D. None
10. What is the output of a classification algorithm?
- A. Continuous value
 - B. Discrete label
 - C. Cluster
 - D. Probability only

2 Marker

11. Which technique is commonly used for market segmentation?

- A. PCA
- B. Clustering
- C. Regression
- D. Classification

12. What is the key difference between supervised and unsupervised learning?

- a) Supervised learning requires labeled data, while unsupervised learning does not.
- b) Supervised learning predicts labels, while unsupervised learning discovers patterns.
- c) Supervised learning is used for classification, while unsupervised learning is used for regression.
- d) Supervised learning is always more accurate than unsupervised learning.

13. In which scenario would you apply machine learning?

- A To create a simple rules-based system
- B To develop predictions from historical data
- C To manually program every output
- D To increase computer speed

14. Which learning paradigm aims to maximize a reward signal through exploration and exploitation?

- A Supervised Learning
- B Unsupervised Learning
- C Reinforcement Learning
- D Semi-supervised Learning

15. What distinguishes reinforcement learning from other learning paradigms?

- A The use of labeled data
- B Learning from unlabeled data
- C Learning through rewards and penalties
- D None of the above

16. Which of the following is an example of supervised learning?

- A. Grouping customers based on purchasing behavior
- B. Predicting house prices using labeled historical data
- C. Finding hidden patterns in unlabeled data
- D. Reducing the dimensionality of a dataset

17. Which machine learning technique is primarily used to predict a continuous numerical value?

- A. Classification
- B. Clustering
- C. Regression
- D. Association

18. Which machine learning approach is suitable for grouping similar data points without predefined labels?

- A. Regression
- B. Classification
- C. Clustering
- D. Reinforcement Learning

19. Which type of machine learning is commonly used for detecting whether an email is spam or not spam?

- A. Classification
- B. Clustering
- C. Regression
- D. Dimensionality Reduction

20. What is the main purpose of a machine learning model?

- A. To learn patterns from data and make predictions or decisions
- B. To eliminate the need for data
- C. To manually define every possible output
- D. To increase the physical storage capacity of a computer

21. An online shopping company wants to divide its customers into different groups based on purchasing behavior. There are no predefined labels indicating customer types. The company wants to discover hidden patterns for targeted marketing.

Questions:

1. Which type of machine learning is appropriate?
2. Which unsupervised learning technique is suitable?
3. Name one algorithm that can be used for this task and explain why.

22. A warehouse robot learns to move packages from one location to another. Every time it reaches the destination without collision, it receives a positive reward. If it collides with obstacles, it receives a penalty. Initially, the robot makes random moves, but over time it learns the best path.

Questions:

1. Identify the type of machine learning used.
2. Explain the role of the agent, environment, and reward in this scenario.
3. Why is reinforcement learning preferred over supervised learning for this application?

23. What is reinforcement learning? What is the role of rewards and penalties?

The objective is to maximize the cumulative reward by learning the best policy.

24. Define a well-posed Machine Learning problem. What are the three components of Tom Mitchell's PTE framework?

25. Why Decision tree uses the inductive learning approach.

4 marker

21. Differentiate between Regression and Classification.
22. Explain the key elements of Reinforcement Learning.
23. Compare Generative and Discriminative Models.
24. Observe the following figure.

Training Data

(Labelled)

|
V

Train Model

|
V

Predict Class

Answer the following:

- a) Which learning paradigm is shown? (1)
 - b) Name one regression algorithm. (1)
 - c) Name one classification algorithm. (1)
 - d) Mention one application of supervised learning. (1)
26. Explain Clustering and Association in Unsupervised Learning with one real-world application for each.
27. Explain Tom Mitchell's definition of Machine Learning using the P-T-E framework with an example.
28. A hospital has collected patient records containing age, blood pressure, cholesterol level, medical history, and disease diagnosis. The hospital wants to:
- Predict whether a new patient is diabetic.
 - Group patients with similar health conditions.
 - Develop an intelligent system that recommends personalized treatment plans based on patient recovery.

Identify the appropriate Machine Learning technique for each task. Justify your choice and suggest one suitable algorithm.

29. A hospital wants to:

- Predict diabetes.
- Group patients with similar diseases.
- Recommend personalized treatment.

30. A streaming platform wants to recommend movies to users based on their viewing history. Identify the P-T-E components and justify your answers.

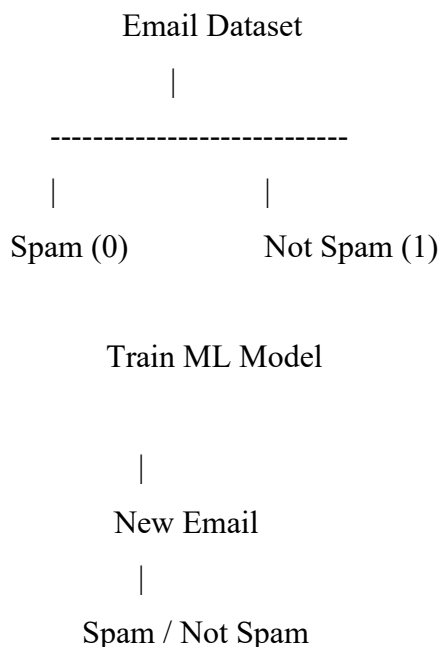
6/8 /10 Marker

31. A hospital wants to develop an AI system that predicts whether a patient has diabetes based on features such as age, BMI, blood glucose level, blood pressure, and family history. Historical patient records are available, and each record is labeled as Diabetic or Non-Diabetic.

Answer the following:

- a) Which type of Machine Learning should be used? Justify your answer. (2 Marks)
- b) Is this a Regression or Classification problem? Explain. (2 Marks)
- c) Identify the Task (T), Training Experience (E), and Performance Measure (P) according to Tom Mitchell's definition. (3 Marks)
- d) Suggest one suitable machine learning algorithm. (1 Mark)

32. Study the following figure.



Answer the following:

- a) Identify the Machine Learning paradigm. (2)
- b) Is this regression or classification? Justify. (2)
- c) Identify the Task (T), Experience (E), and Performance Measure (P). (3)
- d) Suggest one suitable algorithm. (1)

33. An AgriTech startup has installed IoT sensors across thousands of farms. The sensors collect data such as:

- Soil moisture
- Temperature
- Humidity
- Rainfall
- Soil nutrients
- Crop type

The company wants to:

1. Predict the crop yield before harvesting.
2. Group farms having similar soil and weather conditions.
3. Automatically determine the best irrigation schedule to maximize crop yield while minimizing water usage.

Answer the following:

- a) Identify the Machine Learning technique suitable for each problem.
- b) Justify your answer.
- c) Suggest one suitable algorithm for each task.
- d) Mention the expected output.

34. Compare Supervised Learning, Unsupervised Learning, and Reinforcement Learning based on the following parameters:

- Type of training data
- Learning objective
- Output
- Common algorithms
- Applications

35. An online retail company wants to improve customer experience by:

- Predicting future sales.
- Detecting fraudulent transactions.
- Recommending products frequently purchased together.

Identify the appropriate Machine Learning technique for each task. Justify your answer and suggest one suitable algorithm for each problem.

Question 1

A botanist wants an algorithm to learn the concept of when a specific rare plant "Thrives". The dataset contains three attributes: Light (High, Medium, Low), Water (Daily, Weekly), and Fertilizer (Yes, No).

Given the following new training data, explicitly trace the Find-S algorithm step-by-step to derive the final hypothesis. Finally, predict the outcome for a new instance: $x_{new} = \{\text{Medium, Daily, Yes}\}$.

- D1: {High, Daily, Yes} → Positive
- D2: {Low, Daily, No} → Negative
- D3: {High, Weekly, Yes} → Positive
- D4: {Medium, Weekly, Yes} → Negative
- D5: {High, Weekly, No} → Positive

Question 2

In the formal framework of concept learning, define the "Concept Space", the "Hypothesis Space", and the "Version Space". Explain the mathematical relationship and nesting between these three spaces based on the lecture diagrams.

Question 3

Consider a machine learning model predicting smartwatch purchase behavior based on 5 attributes: Brand (3 possible values), Strap Material (3 possible values), Cellular Capability (2 values), GPS (2 values), and Heart Rate Monitor (2 values).

Formulate and calculate the exact sizes of the Feature Space, the Concept Space, and the Hypothesis Space (assuming a conjunctive inductive bias).

Question 4

Despite being the fundamental concept learning algorithm, the Find-S algorithm has significant flaws. Discuss four primary limitations of this algorithm as taught in your coursework.

Question 5

A user's movie watching preferences are logged using four attributes: Genre (Action, Comedy, Drama), Length (Short, Long), Origin (Domestic, Foreign), and Snacks (Yes, No). Target Concept: "Watch".

- X1: {Action, Short, Domestic, Yes} → Positive
- X2: {Action, Long, Domestic, No} → Positive
- X3: {Comedy, Short, Foreign, Yes} → Negative
- X4: {Drama, Long, Domestic, No} → Positive

Trace the Find-S algorithm and state the final hypothesis. Based on this, will the user watch {Comedy, Long, Domestic, Yes}?

Question 6

Provide Tom Mitchell's formal definition of "Concept Learning". Furthermore, explain how a high-level concept is physically represented in a machine-learning algorithm using the "Feature Space" (use a concrete example to illustrate).

Question 7

Define "Inductive Bias" and explain why assuming a conjunctive concept is considered a "Restriction Bias". Additionally, what is the inherent danger regarding the "Target Concept" when applying such a bias?

Question 8

A real estate application uses 7 attributes to define a house. Two of these attributes have exactly 4 possible values each. The remaining five attributes are all binary (2 possible values each).

Calculate the total number of concepts that can be defined over this instance space, and the number of semantically distinct hypotheses that can be formed using a conjunctive restriction bias.

Question 9

Consider a simple dataset determining if a job offer is "Good". There are only two binary attributes: Salary (High, Low) and Remote (Yes, No).

The training dataset is:

- D1: {High, Yes} → Positive
- D2: {Low, Yes} → Negative

First, define all semantically distinct hypotheses in the hypothesis space. Then, evaluate each one to explicitly derive the final Version Space.

Question 10

1. Explain the inherent assumption Find-S makes about instances by default.
2. State the two specific conditions under which the output hypothesis of Find-S will be guaranteed to be consistent with negative examples.
3. Describe the theoretical inverse approach to Find-S suggested in the slides.
4. Name the algorithm that combines both the Find-S approach and this inverse approach.

1. Define Accuracy, Precision, Recall and Specificity, providing their mathematical formulas.

2. Describe the components of a 2×2 Confusion Matrix.

3. Define and provide the formulas for bias and variance in context to Machine Learning. What do high bias and high variance indicate?

5. What is the primary purpose of using Cross-Validation instead of a simple single Train/Test split? Briefly list two key advantages of using Cross-Validation.

MODERATE QUESTIONS:

6. Given the following 2×2 confusion matrix:

Actual Positive: TP = 70, FN = 30

Actual Negative: FP = 10, TN = 90

Calculate: (a) Accuracy, (b) Precision, (c) Recall, and (d) Specificity.

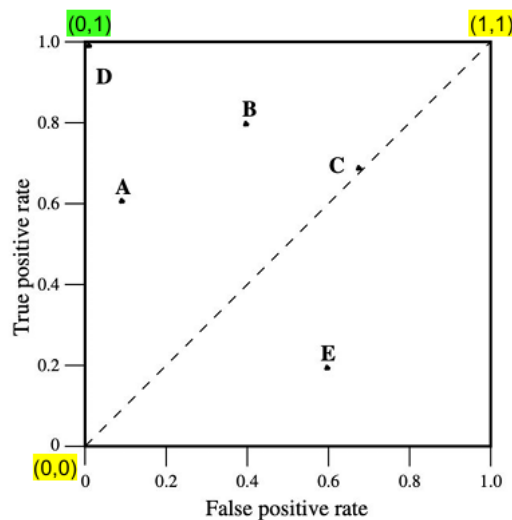
7. A model records a Precision of 0.6 and a Recall of 0.8. Calculate its F1-Score.

8. How does increasing model complexity affect Bias, Variance and total error? Sketch or describe the general curve trends.

HARD QUESTIONS:

9. Refer to the standard ROC space plot where the y -axis represents the True Positive Rate and the x -axis represents the False Positive Rate. Describe the classification behavior associated with each of the following:

- (a) A model located at $(0,0)$ versus a model located at $(1,1)$.
- (b) A model located at $(0,1)$.
- (c) Points lying along the diagonal line $y = x$.
- (d) Points lying strictly within the lower-right triangular region.



10. Prove that for a binary classification task, $FPR = 1 - \text{Specificity}$.

6/8 Marks

Q1. Explain the Bias-Variance Tradeoff in detail. Describe how total error behaves as model complexity increases from an overly simple model to an overly flexible model. Provide the mathematical formulation decomposing Total Expected Prediction Error at a point x into its three fundamental components (Bias², Variance, and Irreducible Error)

Q2. The true underlying relationship between the target variable y and the input variable x is given by: $y = 3x + e$ where the random noise ' e ' follows a normal distribution with mean 0 and variance = 0.5.

Three different training datasets are sampled from this same distribution. When trained on each dataset, a learning algorithm produces the following constant prediction models:

- Model 1: $f_1(x) = 5.7$
- Model 2: $f_2(x) = 6.2$
- Model 3: $f_3(x) = 5.8$

For an evaluation test point $x_0 = 2$, the true noise-free value is $f(2) = 3(2) = 6$.

Calculate each of the following step-by-step, showing all formulas and intermediate arithmetic:

Q3. In high-dimensional feature spaces, visual inspection of decision boundaries is impossible. Explain the systematic diagnostic procedure used to classify model performance into High Bias, High Variance, High Bias & High Variance, or Low Bias & Low Variance using Training Error and Test Error relative to the optimal Bayes (human baseline) error.

Construct a structured diagnostic reference table showing realistic hypothetical error percentages for each of the four scenarios assuming an optimal Bayes error of 1%.

Q4. Consider a rare disease screening dataset containing 9,950 healthy patient scans and 50 disease-positive scans. A naive majority-class classifier predicts every single scan as "healthy".

- Calculate the naive classifier's overall Accuracy.
- Explain in detail why this Accuracy is dangerously misleading.
- Discuss why Precision, Recall, and the F1-Score provide superior diagnostic fidelity for high-stakes imbalanced domains.

Given: 9950 Healthy (Negative) scans, 50 Disease-positive (Positive) scans. Naive model classifies all as Healthy.

$TP = 0, FN = 50, TN = 9950, FP = 0, Total = 10000$

$Accuracy = (TP + TN) / Total = (0 + 9950) / 10000 = 99.5\%$

Why it is problematic: A 99.5% accuracy gives a false sense of security. In clinical deployment, this model fails 100% of the diseased patients, leading to untreated illness and fatal outcomes.

Evaluation with Precision and Recall:

$Recall = TP / (TP + FN) = 0 / 50 = 0\%$. This immediately exposes complete system failure.

$F1-Score = 0\%$, providing an accurate summary of zero detection utility.

Q5. In a 3-class vehicle classification task (Sedan, SUV, Truck) with confusion matrix entries written as $P(\text{Actual}, \text{Predicted})$:

- Explain the "One-vs-All" approach used to isolate a single class against all others.
- Write down the explicit summation expressions for TP, TN, FP, and FN specifically for the SUV class.
- Write the algebraic formulas for Precision and Recall for the SUV class.

Q6. What is a Receiver Operating Characteristic (ROC) curve? In the ROC space (FPR on the x-axis, TPR on the y-axis), describe the exact operational meaning of the following points and regions:

- Point (0, 0)
- Point (1, 1)
- Point (0, 1)
- The diagonal line $y = x$
- The region lying strictly below the diagonal line $y = x$

Q7. Differentiate between "Conservative" and "Liberal" classification strategies in the context of an ROC trajectory. If you are deploying an Airport Security Automated Luggage Scanner to flag high-yield explosive contraband, where should your chosen operating threshold lie on the ROC curve (Conservative or Liberal region)? Justify your decision from a risk-cost perspective.

Q8. A binary medical classifier produces the following operating points across varying probability thresholds:

Decision Threshold	False Positive Rate (FPR)	True Positive Rate (TPR)
0.2	0.80	1.00
0.5	0.30	0.90
0.7	0.10	0.70

0.9	0.00	0.40
-----	------	------

(a) If clinical protocol mandates that the False Positive Rate must not exceed 10% ($FPR \leq 0.10$), determine the best threshold to select and state its corresponding TPR.

(b) Sketch the approximate piecewise ROC curve starting from (0,0) through the threshold points up to (1,1).

Q9. A binary classification evaluation on a dataset of $N = 250$ instances yields:

- Precision = 0.80
- Sensitivity = $8/9$ (approximately 0.8889)
- Specificity = $7/8$ (0.875)

Formulate the system of equations to determine the exact integer values for the confusion matrix (TP, TN, FP, FN). Then, calculate the overall Accuracy and the exact F1-Score. Show your complete step-by-step mathematical derivation.

Q10. Explain the concept of "Class Skew". Why are metrics such as Accuracy, Precision, and the F1-Score inherently sensitive to shifts in class prevalence/distribution, whereas the ROC curve and AUC remain invariant under changing class proportions? Discuss why combining ROC analysis with PR/F1 analysis provides a complete view of model robustness.

UE24CS352A | Machine Learning

DECISION TREES: ID3, ENTROPY, INFORMATION GAIN, OVERFITTING AND PRUNING

15 Descriptive Questions | 4 Marks Each | Maximum Marks: 60

SECTION A | DESCRIPTIVE QUESTIONS (Q1 - Q15) Answer all questions.

ENTROPY | MEDIUM

1. Define entropy as it is used in decision tree learning. Write the general formula for the entropy of a set S whose target attribute takes c distinct values, and state the maximum value the entropy can reach. A node holds 14 training instances, of which 9 are labelled 'Yes' and 5 are labelled 'No'. Compute the entropy of this node. [4 marks]

INFORMATION GAIN | EASY

2. Write the formula for the average information entropy $I(A)$ of an attribute A and the formula for $\text{Gain}(S, A)$, explaining what each term stands for. For the PlayTennis data set $\text{Entropy}(S) = 0.94$ over 14 instances. Splitting on Outlook produces S_{sunny} with 5 instances and entropy 0.971, S_{overcast} with 4 instances and entropy 0, and S_{rain} with 5 instances and entropy 0.971. Compute $\text{Gain}(S, \text{Outlook})$. [4 marks]

CONSTRUCTING A DECISION TREE | MEDIUM

3. The five training instances below describe the concept 'job acceptance', using the attributes salary and location. Format: (salary, location, job acceptance)
- Tier 1, MUM, YES
 - Tier 1, BLR, YES
 - Tier 2, BLR, NO
 - Tier 1, HYD, NO
 - Tier 2, MUM, YES

Compute the entropy of the whole set and the information gain of Location. Given that $\text{Gain}(S, \text{salary}) = 0.0202$, state which attribute becomes the root node and why. [4 marks]

INFORMATION GAIN | MEDIUM

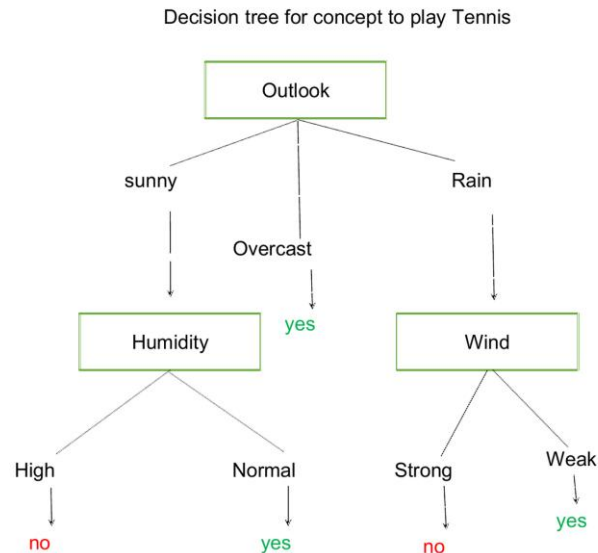
4. After Outlook is chosen as the root of the PlayTennis tree, the branch Outlook = Sunny carries the five instances below. Format: (Temp, Humidity, Windy, PlayTennis)
- High, High, Weak, No
 - High, High, Strong, No
 - Medium, High, Weak, No
 - Cool, Normal, Weak, Yes
 - Medium, Normal, Strong, Yes

Given $\text{Entropy}(S_{\text{sunny}}) = 0.971$, compute $\text{Gain}(S_{\text{sunny}}, \text{Humidity})$. Given also that $\text{Gain}(S_{\text{sunny}}, \text{Temp}) = 0.571$ and $\text{Gain}(S_{\text{sunny}}, \text{Windy}) = 0.02$, state which attribute is chosen at this level and explain why no further splitting is needed below it. [4 marks]

ENTROPY | MEDIUM

5. A data set of 8 instances has a target attribute Y that takes three values: 3 instances belong to class T , 2 to class E and 3 to class F . Compute $\text{Entropy}(S)$. State the maximum entropy this target attribute could have had, and explain the condition under which that maximum would be reached. [4 marks]

6. Explain how a decision tree represents a target function as a disjunction of conjunctions. Using the PlayTennis decision tree shown below, write the set of if-then rules for the paths that lead to the classification 'Yes'. [4 marks]



7. Describe the hypothesis space that ID3 searches and the nature of the search it performs. State two consequences of the fact that ID3 performs no backtracking. [4 marks]

8. State the inductive bias of ID3 and explain why it is described as a preference bias. Explain why this bias favours attributes with many values, and contrast the source of ID3's inductive bias with that of the FIND-S / CANDIDATE-ELIMINATION algorithm. [4 marks]

9. List the main issues that arise in decision tree learning. Explain how ID3 can deal with attributes that have missing values, and why attributes with differing costs need special treatment. [4 marks]

10. Give the formal definition of overfitting in terms of the error of a hypothesis over the training data and over the entire distribution of data. Explain the definition in words, and state the conditions under which overfitting turns out to be harmful. [4 marks]

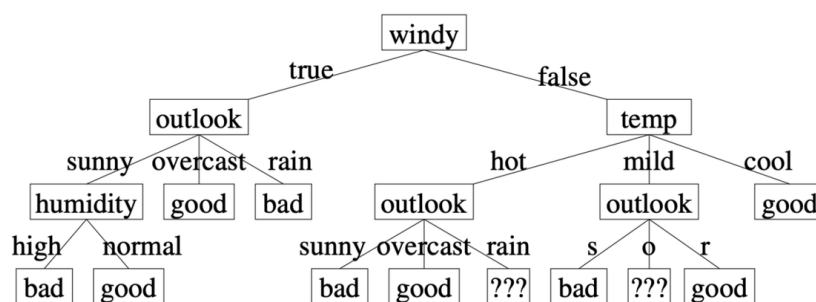
11. The graph below plots mean average error against tree depth, for both the training set and the validation set. Explain the behaviour of the two curves, identify the regions of underfitting and overfitting, and state how the depth at which to stop growing the tree is determined. [4 marks]



12. Pruning is used to avoid overfitting in decision trees. Explain what pruning means, distinguish between early termination and post-pruning, and state which of the two is found to be more effective and why. [4 marks]

13. Describe the reduced error pruning algorithm. State how the available data is divided for this purpose, and describe how the size of the tree and its accuracy over the validation set change as pruning proceeds. [4 marks]

14. Describe the rule post-pruning procedure. For the decision tree shown below, write the rule corresponding to the path (windy = false, temp = hot, outlook = sunny), show how that rule could be pruned, and explain why the pruned rules must be sorted before they are used. [4 marks]



15. ID3 as originally defined is restricted to attributes that take on a discrete set of values. Explain how a continuous-valued attribute is incorporated into the learned tree and how the threshold is selected. For the data below, identify the candidate thresholds and compute the information gain of $\text{Temp} > 54$.

Temp	:	40	48	60	72	80	90
Play Sport	:	no	no	yes	yes	yes	no

[4 marks]

MACHINE LEARNING

Question Bank on Introduction to Machine Learning and Machine Learning Models

Question 1.

Explain how **cross-validation** helps detect overfitting. Compare it with a simple train-test split and discuss its advantages for model selection.

Question 2.

Explain why **feature scaling** is important when training Logistic Regression using continuous variables. Compare **Normalization** and **Standardization**.

Question 3.

Explain why **Linear Regression is not suitable for binary classification**. Discuss how Logistic Regression overcomes these limitations using the sigmoid function.

Question 4

Explain why Logistic Regression is preferred over Linear Regression for binary classification problems.

Question 5

A dataset contains **500 continuous variables**, but only **40** are relevant. Explain:

How irrelevant variables affect model performance.

Why feature selection is necessary.

Compare Filter, Wrapper, and Embedded feature selection methods.

Which method would you recommend and why?

Question 6

A telecom company develops a Logistic Regression model using:

Monthly Charges

Tenure

Data Usage

Number of Complaints

Customer ID

Contract Type

The model achieves:

Training Accuracy = 98%

Testing Accuracy = 73%

Analyze the possible causes of poor performance and propose a complete strategy to improve the model.

Question 7

A Logistic Regression model predicts the following probabilities:

0.95, 0.83, 0.76, 0.62, 0.49, 0.41, 0.28

Determine the predicted class for:

a) Threshold = 0.5

b) Threshold = 0.8

Explain how changing the threshold affects precision and recall.

Question 8

Explain why the decision boundary of Logistic Regression is linear. Describe how polynomial feature expansion changes the decision boundary without changing the learning algorithm.

Question 9

A dataset contains:

350 continuous variables

20 categorical variables

5 identifier columns

Design a feature selection strategy before training Logistic Regression.

Question 10

Three Logistic Regression models are trained.

Rank the models and explain your reasoning.

Model	Feature s	Regularizatio n	Trainin g	Validatio n
A	25	None	90%	88%
B	250	None	100%	65%
C	250	L2	95%	92%

Question 11

A hospital collects the following patient data:

Age

BMI

Blood Glucose

Insulin Level

Blood Pressure

Physical Activity Score

The Logistic Regression model achieves:

Training Accuracy = 95%

Validation Accuracy = 94%

Test Accuracy = 70%

The validation and test datasets come from different hospitals.

Why is there a significant drop in test accuracy?

Is the model overfitted?

Explain the concept of domain shift.

How can the model be made more robust?

Question 12

A healthcare organization is developing a disease prediction system using **Logistic Regression** with **250 continuous variables**. The initial model shows:

Training Accuracy = **99%**

Validation Accuracy = **70%**

Design a complete strategy to improve the model. Your answer should include preprocessing, feature engineering, overfitting control, model validation, and evaluation.

Question 13

Two logistic regression models produce the following results:

- (a) Identify the overfitting model.
- (b) Explain what the cost values indicate.
- (c) Which model would you choose?

Model Training Cost Validation Cost

A	0.42	0.45
B	0.05	1.20

Question 14

A logistic regression model initially has two features:

$$z = -3 + 2x_1 + x_2$$

Later, 20 additional features are added. The training accuracy increases from 85% to 99%, but test accuracy decreases from 83% to 65%.

- (a) Explain what happened.
- (b) What happens to the decision boundary when additional features are introduced?
- (c) How can regularization help?

Question 15

A bank develops three logistic regression models for predicting loan default:

- (a) Which model is overfitting?
- (b) Which model should be selected?
- (c) Explain why Model C has such a low training cost.
- (d) Suggest two methods to improve Model C

Model No. of Features Training Cost Test Cost

A	5	0.48	0.52
B	15	0.25	0.31
C	100	0.02	1.15

PES UNIVERSITY

Department of Computer Science and Engineering

UE24CS352A – Machine Learning

Topic: K-Nearest Neighbour (KNN)

1. Distinguish between eager learning and lazy learning. Why is K-Nearest Neighbour considered a lazy, instance-based learning algorithm?
2. Write the Minkowski distance formula used in KNN. Explain the significance of Manhattan distance and Euclidean distance.
3. Using Euclidean distance, classify the query point (24, 80) from the given training dataset with $K = 3$.
4. Explain how KNN performs regression. Predict the output for a query instance using $K = 3$ and $K = 5$.
5. Classify the query sample using $K = 1$, $K = 3$, and $K = 5$. Which value of K is most appropriate and why?
6. Explain Distance Weighted KNN. How does inverse-distance weighting improve the prediction compared to vanilla KNN?
7. Classify the given fraud detection query before and after feature normalization. Explain why the classification changes.
8. How are categorical attributes converted for use in KNN? Explain Boolean encoding, ordinal encoding, and one-hot encoding with examples.
9. What is the Curse of Dimensionality? Explain its impact on KNN and suggest methods to overcome it.
10. Explain Attribute Weighted Distance in KNN. Classify the query using the weighted distance metric and justify the final prediction.

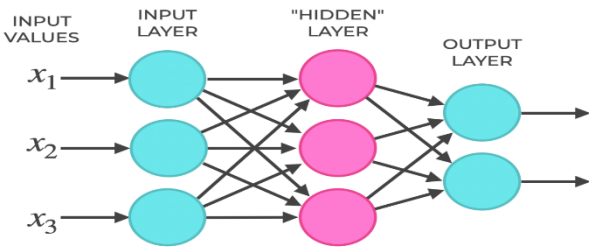
Unit 2 – 4 Marks Descriptive Question Bank

Question No	1
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Back-propagation
Question	What is backpropagation? Describe the steps involved in training using backpropagation.
Long Answer	Backpropagation is a fundamental training algorithm for multilayer neural networks. It minimizes prediction error by adjusting weights and biases based on how much they contributed to the error. Steps: 1. Forward Pass: Compute outputs from inputs using current weights. 2. Loss Calculation: Compare predicted output with actual output using a loss function (e.g., Mean Squared Error). 3. Backward Pass: Calculate gradients of the loss with respect to each weight by applying the chain rule of calculus. 4. Weight Update: Modify weights using a technique like Gradient Descent: $w_{\text{new}} = w_{\text{old}} - \eta \times \partial L / \partial w$ where, η is the learning rate. This iterative process continues until the model converges to minimum loss, effectively "learning" the optimal mapping from input to output.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	2
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Perceptrons
Question	Why can a single-layer perceptron not solve the XOR problem? Explain with reasoning and a solution.
Long Answer	A single-layer perceptron uses a linear decision boundary to classify inputs. The XOR problem involves input-output pairs that are not linearly separable, meaning no straight line can divide the classes correctly in feature space. Since the perceptron lacks hidden layers and non-linear activation mechanisms, it cannot learn this pattern. Solving XOR requires a non-linear model like a Multilayer Perceptron (MLP) with at least one hidden layer, which allows it to capture more complex relationships.

Marks	4
-ve applicable?	N
% Applicable	N

Question No	3
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Multi-layer networks
Question	Write a detailed note on multi-layer perceptrons (MLPs) with a diagram.

Long Answer	An MLP consists of an input layer, one or more hidden layers, and an output layer. Each neuron in a layer is fully connected to the neurons in the next layer. MLPs use non-linear activation functions such as ReLU or Sigmoid. - Features: - Capable of learning complex non-linear functions - Require backpropagation and optimization algorithms like SGD or Adam A SIMPLE EXAMPLE OF A MULTILAYER PERCEPTRON  Diagram: - Input Layer → Hidden Layer(s) → Output Layer - Each arrow represents weighted connections
-------------	---

Marks	4
-ve applicable?	N
% Applicable	N

Question No	4
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Introduction to Artificial Neural Networks
Question	Given the input vector $x = [1, 2]$, weights $w = [0.5, 0.2]$, bias $b = 0.1$, and target value $t = 1$, calculate the following:

	1. The net input z . 2. The neuron output y using the sigmoid function. 3. The prediction error using $E = 1/2 (t - y)^2$.
Long Answer	Solution 1. Net input $z = (1 \times 0.5) + (2 \times 0.2) + 0.1$ $z = 0.5 + 0.4 + 0.1 = 1.0$ 2. Sigmoid output $y = \sigma(z) = 1 / (1 + e^{-z})$ $y = 1 / (1 + e^{-1}) \approx 0.731$ 3. Prediction error $E = 1/2 (t - y)^2$ $E = 1/2 (1 - 0.731)^2$ $E = 1/2 (0.269)^2 \approx 0.0362$ Final Answer $z = 1.0, y \approx 0.731, E \approx 0.0362$ The net input is the weighted sum of the inputs plus the bias. The sigmoid function transforms this net input into an output between 0 and 1. The squared-error term measures the difference between the target value and the predicted output; a smaller value indicates a prediction closer to the target.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	5						
Question Type							
Difficulty level	Easy						
Chapter	Unit 2						
Topic	Back-propagation						
Question	A neural network contains four hidden layers with sigmoid activation. During backpropagation, assume that the sigmoid derivative at each hidden layer is 0.2, all connecting weights along a particular path are 1, and the gradient received from the output layer is 1. <table border="1"> <tr> <td>1. Calculate the gradient reaching the first hidden layer.</td><td>2 Marks</td></tr> <tr> <td>2. Identify and explain the problem illustrated by this result.</td><td>1 Mark</td></tr> <tr> <td>3. State one method to reduce this problem.</td><td>1 Mark</td></tr> </table>	1. Calculate the gradient reaching the first hidden layer.	2 Marks	2. Identify and explain the problem illustrated by this result.	1 Mark	3. State one method to reduce this problem.	1 Mark
1. Calculate the gradient reaching the first hidden layer.	2 Marks						
2. Identify and explain the problem illustrated by this result.	1 Mark						
3. State one method to reduce this problem.	1 Mark						
Long Answer	1. Gradient reaching the first hidden layer (2 Marks) Using the chain rule, the gradient is repeatedly multiplied by the weight and the derivative of the sigmoid function at each hidden layer. Gradient = $1 \times (1 \times 0.2)^4$ Gradient = $0.2^4 = 0.0016$ Therefore, the gradient reaching the first hidden layer is 0.0016. 2. Problem illustrated (1 Mark) The gradient becomes extremely small because it is repeatedly multiplied by sigmoid derivatives, whose maximum value is 0.25. Consequently, the earlier layers receive very small weight updates and learn very slowly or may stop learning. This is called the vanishing gradients problem. 3. One remedy (1 Mark) Use ReLU or Leaky ReLU activation functions in the hidden layers instead of						

	sigmoid. Other suitable remedies include careful weight initialization, batch normalization, and residual connections.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	6						
Question Type							
Difficulty level	Easy						
Chapter	Unit 2						
Topic	Multi-layer networks						
Question	Input layer 1 $x_1 = 1$ 2 $x_2 = 0$ Hidden layer 3 h_3 4 h_4 Output layer 5 y_5 $w_{13} = 2$ $w_{23} = -3$ $w_{14} = 1$ $w_{24} = 4$ $w_{35} = 2$ $w_{45} = -1$ Unipolar step function $f(v) = 1 \text{ if } v \geq 0$ $f(v) = 0 \text{ if } v < 0$ The network in Figure uses the unipolar step function $f(v) = 1$ for $v \geq 0$ and 0 otherwise. The weights are: <table><tr><td>$w_{13} = 2$</td><td>$w_{23} = -3$</td><td>$w_{14} = 1$</td></tr><tr><td>$w_{24} = 4$</td><td>$w_{35} = 2$</td><td>$w_{45} = -1$</td></tr></table> For the input values $x_1 = 1$ and $x_2 = 0$, what is the final network output? Notation: w_{ij} denotes the weight from neuron i to neuron j. No bias terms are specified.	$w_{13} = 2$	$w_{23} = -3$	$w_{14} = 1$	$w_{24} = 4$	$w_{35} = 2$	$w_{45} = -1$
$w_{13} = 2$	$w_{23} = -3$	$w_{14} = 1$					
$w_{24} = 4$	$w_{35} = 2$	$w_{45} = -1$					
Long Answer	Detailed Step-by-Step Solution Step 1: Compute the net input and output of hidden neuron 3 Neuron 3 receives the two network inputs through weights w_{13} and w_{23}. $v_3 = x_1w_{13} + x_2w_{23}$$v_3 = (1)(2) + (0)(-3) = 2$ $h_3 = f(v_3) = f(2) = 1$						

	Step 2: Compute the net input and output of hidden neuron 4 Neuron 4 receives the inputs through weights w_{14} and w_{24}. $v_4 = x_1w_{14} + x_2w_{24}$ $v_4 = (1)(1) + (0)(4) = 1$ $h_4 = f(v_4) = f(1) = 1$ Step 3: Compute the net input and final output of neuron 5 The outputs of neurons 3 and 4 become the inputs to output neuron 5. $v_5 = h_3w_{35} + h_4w_{45}$ $v_5 = (1)(2) + (1)(-1) = 1$ $y_5 = f(v_5) = f(1) = 1$ Calculation Summary <table><thead><tr><th>Neuron</th><th>Net-input calculation</th><th>Net input</th><th>Step output</th></tr></thead><tbody><tr><td>3</td><td>$1(2) + 0(-3)$</td><td>2</td><td>1</td></tr><tr><td>4</td><td>$1(1) + 0(4)$</td><td>1</td><td>1</td></tr><tr><td>5</td><td>$1(2) + 1(-1)$</td><td>1</td><td>1</td></tr></tbody></table>	Neuron	Net-input calculation	Net input	Step output	3	$1(2) + 0(-3)$	2	1	4	$1(1) + 0(4)$	1	1	5	$1(2) + 1(-1)$	1	1
Neuron	Net-input calculation	Net input	Step output														
3	$1(2) + 0(-3)$	2	1														
4	$1(1) + 0(4)$	1	1														
5	$1(2) + 1(-1)$	1	1														
Marks	4																
-ve applicabl e?	N																
% Applicabl e	N																

Question No	7
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Perceptrons
Question	Consider two perceptrons defined by the threshold expression $w_0 + w_1 x_1 + w_2 x_2 > 0$. Perceptron A: $w_0 = 1, w_1 = 2, w_2 = 1$ Perceptron B: $w_0 = 0, w_1 = 2, w_2 = 1$ True or False: Perceptron A is more general than Perceptron B.
Long Answer	Step 1: Write the positive-class condition for each perceptron Perceptron A: $1 + 2x_1 + x_2 > 0$ Perceptron B: $2x_1 + x_2 > 0$ Step 2: Show that every positive example of B is also positive for A Suppose perceptron B classifies an input as positive. Then $2x_1 + x_2 > 0$ Adding 1 to both sides gives $1 + 2x_1 + x_2 > 1 > 0$ Therefore, the same input is also classified as positive by perceptron A.

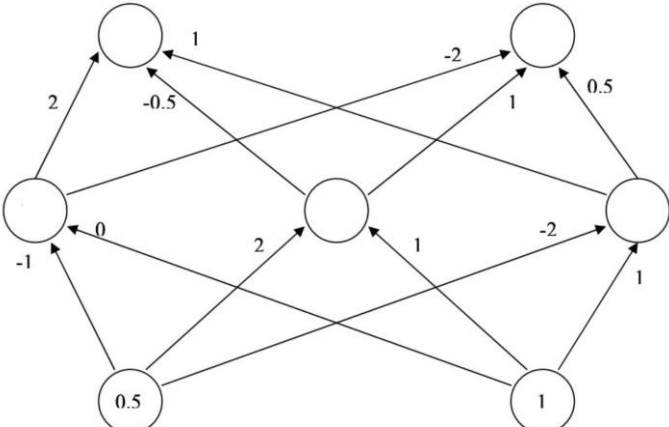
	Step 3: Show that A accepts at least one additional input Consider the input $x_1 = 0$ and $x_2 = 0$. <table><tr><th>Perceptron</th><th>Score</th><th>Threshold test</th><th>Output</th></tr><tr><td>A</td><td>$1 + 2(0) + 0 = 1$</td><td>$1 > 0$</td><td>Positive</td></tr><tr><td>B</td><td>$0 + 2(0) + 0 = 0$</td><td>$0 > 0$ is false</td><td>Negative</td></tr></table> Thus, perceptron A classifies some inputs as positive that perceptron B classifies as negative. Step 5: Apply the definition of more-general-than A hypothesis is more general than another when its set of positive instances contains the positive-instance set of the other hypothesis. Positive(B) $\subset$ Positive(A) Final Answer: TRUE Perceptron A is more general than Perceptron B because every input classified positively by B is also positive for A, while A additionally classifies some inputs, such as (0, 0), as positive. Because Perceptron A is guaranteed to fire any time Perceptron B fires (plus Perceptron A also accepts some inputs that B rejects)	Perceptron	Score	Threshold test	Output	A	$1 + 2(0) + 0 = 1$	$1 > 0$	Positive	B	$0 + 2(0) + 0 = 0$	$0 > 0$ is false	Negative
Perceptron	Score	Threshold test	Output										
A	$1 + 2(0) + 0 = 1$	$1 > 0$	Positive										
B	$0 + 2(0) + 0 = 0$	$0 > 0$ is false	Negative										
Marks	4												
-ve applicable?	N												
% Applicable	N												

Question No	8
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Multi-layer networks
Question	What is regularization in a neural network? Explain how L1 and L2 regularization help reduce overfitting. State one difference between them. (4 Marks)
Long Answer	Regularization improves the generalization of a neural network by adding a penalty to the original loss function. The penalty discourages excessively large weights. Large weights can make the network highly sensitive to small variations in the inputs. This allows the model to fit noise and minor irregularities in the training data, leading to overfitting. L1 Regularization (Lasso) $L_{\text{total}} = L_{\text{original}} + \lambda \sum_i w_i$ L1 regularization penalizes the absolute values of the weights. Drives less important weights completely down to zero. Acts as an automatic feature selector, removing noisy pathways from the network entirely L2 Regularization (Ridge) $L_{\text{total}} = L_{\text{original}} + \lambda \sum_i w_i^2$ L2 regularization penalizes the squared weights. Forces the network to keep weight values small, preventing any single input feature or neuron from

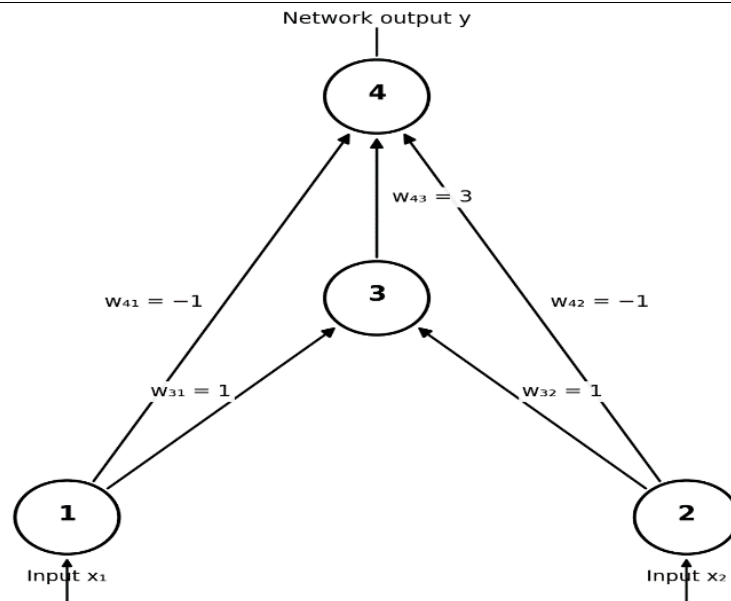
	dominating the final output. L_2 regularization encourages weights toward 0, but never pushes weights all the way to zero. Regularization parameter λ controls the strength of the penalty. A very small λ may not sufficiently reduce overfitting, while an excessively large λ can make the model too simple and cause underfitting.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	9
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Multi-layer networks
Question	What are four possible ways to reduce overfitting in a neural network? Explain each method briefly.
Long Answer	Overfitting occurs when a neural network learns the training data too closely, including noise and accidental patterns, and therefore performs poorly on unseen data. The following methods can reduce overfitting: 1. Use more training data or data augmentation A larger and more varied dataset helps the network learn general patterns instead of memorizing individual training samples. When collecting more data is difficult, data augmentation can create additional training examples through suitable transformations. 2. Apply regularization L_1 or L_2 regularization adds a penalty to the original loss function. This discourages excessively large weights and reduces unnecessary model complexity, thereby improving generalization. 3. Use dropout During training, dropout temporarily sets the outputs of randomly selected neurons to zero. This prevents the network from depending too heavily on specific neurons and encourages the learning of more robust features. 4. Reduce model complexity Using fewer hidden layers, fewer neurons, or fewer trainable parameters lowers the capacity of the network to memorize noise. A simpler model is often more suitable when the available training dataset is limited.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	10
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Multi-layer networks
Question	Question on Linear Neurons: The following is a network of linear neurons, that is, neurons whose output is identical to their net input, $o_i = \text{net}_i$. The numbers in the circles indicate the

	output of a neuron, and the numbers at connections indicate the value of the corresponding weight. a) Compute the output of the hidden-layer and the output-layer neurons for the given input (0.5, 1) and enter those values into the corresponding circles. b) What is the output of the network for the input (1, 2), i.e., the left input neuron having the value 1 and the right one having the value 2? Do you have to do all the network computations once again in order to answer this question? Explain why you do or do not have to do this. 
Long Answer	a) Result: (-2, 3). b) It's (-4, 6). We do not have to do all the computations, because every neuron computes a linear function on its inputs, which means that the entire network computes a linear function. For such a function, if we double the input, we simply double the output as well.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	11
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Multi-layer networks
Question	



The feed-forward multilayer perceptron shown in Figure has binary inputs and binary outputs. Every unit uses the following unipolar step rule:
output = 1 if net input $\geq$ threshold; otherwise output = 0.

The weights are $w_{31} = 1$, $w_{32} = 1$, $w_{41} = -1$, $w_{42} = -1$, and $w_{43} = 3$. The threshold of hidden unit 3 is $\theta_3 = 1.5$, and the threshold of output unit 4 is $\theta_4 = -0.5$. The input units reproduce their binary inputs. Which Boolean function is computed by the network?

Long Answer

Detailed Step-by-Step Solution

Step 1: Determine the output of hidden unit 3

The net input to hidden unit 3 is

$$\text{net}_3 = w_{31}x_1 + w_{32}x_2 = x_1 + x_2.$$

Since $\theta_3 = 1.5$, unit 3 produces 1 only when $x_1 + x_2 \geq 1.5$. For binary inputs, this occurs only for $x_1 = 1$ and $x_2 = 1$. Therefore, hidden unit 3 computes AND.

$$h_3 = x_1 \text{ AND } x_2.$$

Step 2: Determine the output of unit 4

The net input to output unit 4 is

$$\text{net}_4 = w_{41}x_1 + w_{42}x_2 + w_{43}h_3 = -x_1 - x_2 + 3h_3.$$

Unit 4 has threshold $\theta_4 = -0.5$. Hence, $y = 1$ when $\text{net}_4 \geq -0.5$; otherwise $y = 0$.

Step 3: Evaluate all possible input combinations

x_1	x_2	net_3	h_3	net_4	Comparison	y
0	0	0	0	0	$0 \geq -0.5$	1
0	1	1	0	-1	$-1 < -0.5$	0
1	0	1	0	-1	$-1 < -0.5$	0
1	1	2	1	1	$1 \geq -0.5$	1

Step 4: Identify the Boolean function

The output is 1 when the two inputs are equal, namely for (0,0) and (1,1), and it

	is 0 when the inputs are different, namely for (0,1) and (1,0). This is the truth table of the XNOR (logical equivalence) function.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	12
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Back-propagation

Question

The diagram illustrates a neural network with the following components and values:

- Input Layer:**
 - Node 1: $x_1 = 0.05$
 - Node 2: $x_2 = 0.10$
- Hidden Layer:**
 - Node h_0 : Bias, value $h_0 = 1$
 - Node h_1 : Net input $net = 0.3775$, Output $o_1 = 0.5932$
 - Node h_2 : Net input $net = 0.3925$, Output $o_2 = 0.5968$
- Output Layer:**
 - Node o_1 : Net input $net = 1.1058$, Output 0.7513
 - Node o_2 : Net input $net = 1.2248$, Output 0.7729
- Weights and Biases:**
 - From h_0 to h_1 : $b = 0.35$
 - From h_0 to h_2 : $b = 0.35$
 - From h_1 to o_1 : $W = 0.49$
 - From h_1 to o_2 : $W = 0.45$
 - From h_2 to o_1 : $W = 0.50$
 - From h_2 to o_2 : $W = 0.55$
 - From o_1 to o_2 : $W = 0.50$

Consider the given Neural Networks and Values.

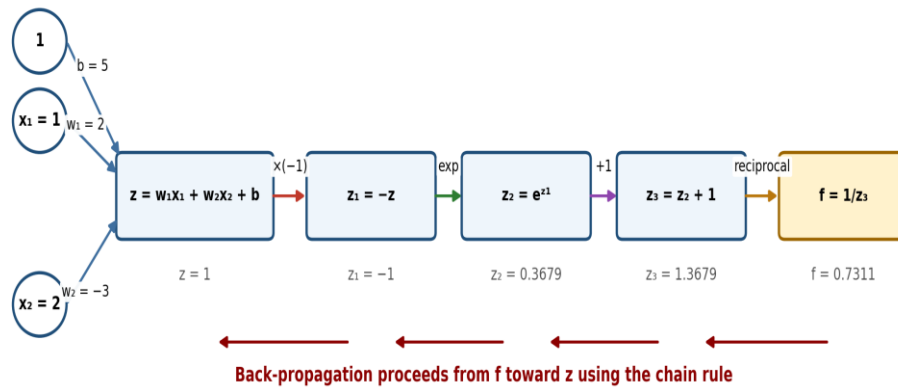
Neuron # in Output Layer (k)	Target Values at output Neuron (t)	Predicted Values (O)	Error in prediction ($t_k - o_k$)
1	0.01	0.7513	-0.7413
2	0.99	0.7729	0.2171

If eta (η) is 0.5, Update W_5 in the output layer.

Long Answer	Step 1: Select the relevant output neuron W_5 connects h_1 to output neuron o_1. Therefore, W_5 is updated using only the error signal of output neuron 1. The target and error of output neuron 2 do not enter the derivative of W_5. Step 2: Compute the prediction error at output neuron 1 $e_1 = t_1 - o_1$ $e_1 = 0.01 - 0.7513 = -0.7413$
-------------	--

	Step 3: Compute the sigmoid derivative at output neuron 1 $\sigma'(\text{net}_1) = o_1(1 - o_1)$ $\sigma'(\text{net}_1) = (0.7513)(1 - 0.7513) = 0.186848$ Step 4: Compute the output-layer delta Using the negative-gradient convention, $\delta_1 = (t_1 - o_1)o_1(1 - o_1)$ $\delta_1 = (-0.7413)(0.186848) = -0.138511$ Step 5: Compute the change in W_5 $\Delta W_5 = \eta \delta_1 h_1$ $\Delta W_5 = (0.5)(-0.138511)(0.5933) = -0.041089$ Step 6: Update W_5 $W_5(\text{new}) = W_5(\text{old}) + \Delta W_5$ $W_5(\text{new}) = 0.40 + (-0.041089)$ $W_5(\text{new}) = 0.358911 \approx 0.3589$
Marks	4
-ve applicable?	N
% Applicable	N

Question No	13
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Back-propagation
Question	Consider the computational graph shown below. The input values are $x_1 = 1$ and $x_2 = 2$. The parameters are $w_1 = 2$, $w_2 = -3$, and $b = 5$. The forward computations are defined as $z = w_1x_1 + w_2x_2 + b, \quad z_1 = -z, \quad z_2 = e^{z_1}, \quad z_3 = z_2 + 1, \quad f = 1/z_3$ Given that $\partial J / \partial f = 1$, use back-propagation and the chain rule to calculate: (i) $\partial J / \partial z$ (1 Mark) (ii) $\partial J / \partial z_3$ (1 Mark) (iii) $\partial J / \partial z_2$ (1 Mark) (iv) $\partial J / \partial z_1$ (1 Mark)



Long Answer

Forward-Pass Values

z	z ₁	z ₂	z ₃	f
1.0000	-1.0000	0.3679	1.3679	0.7311

Step-by-Step Solution

Back-propagation must start at the final output f and proceed in the reverse direction: $f \rightarrow z_3 \rightarrow z_2 \rightarrow z_1 \rightarrow z$.

Step 1: Calculate $\partial J / \partial z_3$

$$\begin{aligned} \partial J / \partial z_3 &= (\partial J / \partial f)(\partial f / \partial z_3) \\ f &= 1/z_3 \Rightarrow \partial f / \partial z_3 = -1/z_3^2 \end{aligned}$$

$$\partial J / \partial z_3 = (1)[-1/(1.3679)^2] = -0.5344$$

Step 2: Calculate $\partial J / \partial z_2$

$$z_3 = z_2 + 1 \Rightarrow \partial z_3 / \partial z_2 = 1$$

$$\partial J / \partial z_2 = (\partial J / \partial z_3)(\partial z_3 / \partial z_2) = (-0.5344)(1) = -0.5344$$

Step 3: Calculate $\partial J / \partial z_1$

$$z_2 = e^{z_1} \Rightarrow \partial z_2 / \partial z_1 = e^{z_1} = z_2$$

$$\partial J / \partial z_1 = (\partial J / \partial z_2)(\partial z_2 / \partial z_1) = (-0.5344)(0.3679) = -0.1966$$

Step 4: Calculate $\partial J / \partial z$

$$z_1 = -z \Rightarrow \partial z_1 / \partial z = -1$$

$$\partial J / \partial z = (\partial J / \partial z_1)(\partial z_1 / \partial z) = (-0.1966)(-1) = 0.1966$$

Final Answers

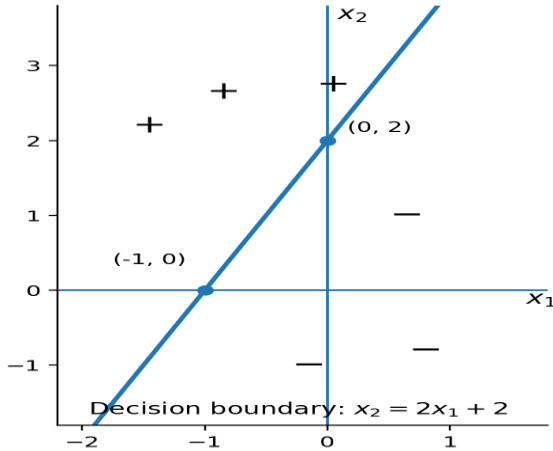
Part	Required Gradient	Answer
(i)	$\partial J / \partial z$	0.1966 $\approx$ 0.20
(ii)	$\partial J / \partial z_3$	-0.5344 $\approx$ -0.53
(iii)	$\partial J / \partial z_2$	-0.5344 $\approx$ -0.53
(iv)	$\partial J / \partial z_1$	-0.1966 $\approx$ -0.20

Check: The result $\partial J / \partial z = 0.1966$ is also equal to $f(1-f)$, which confirms the derivative of the sigmoid computation at $z = 1$.

Marks

4

-ve applicable?	N
% Applicable	N

Question No	14						
Question Type							
Difficulty level	Medium						
Chapter	Unit 2						
Topic	Perceptrons						
Question	Question on Constructing a Perceptron Decision Surface(Tom Mitchell) What are the values of weights w_0, w_1, and w_2 for the perceptron whose decision surface is illustrated in the figure? Assume that the surface crosses the x_1-axis at -1 and the x_2-axis at 2. The positive and negative regions are as marked in the figure.  Decision boundary: $x_2 = 2x_1 + 2$						
Long Answer	Solution Step 1: Write the perceptron decision boundary A two-input perceptron changes class on the line where its net input equals zero: $w_0 + w_1x_1 + w_2x_2 = 0$ Step 2: Use the two intercepts The decision line passes through the x_1-intercept (-1, 0) and the x_2-intercept (0, 2). <table border="1"> <thead> <tr> <th>Point on the boundary</th><th>Substitution in $w_0 + w_1x_1 + w_2x_2 = 0$</th></tr> </thead> <tbody> <tr> <td>(-1, 0)</td><td>$w_0 - w_1 = 0$, when $x_2=0$, $x_1=-1$</td></tr> <tr> <td>(0, 2)</td><td>$w_0 + 2w_2 = 0$, when $x_1=0$, $x_2=2$</td></tr> </tbody> </table> Step 3: Find the equation of the line The slope obtained from the two intercepts is $m = (2 - 0) / (0 - (-1)) = 2$ Since the x_2-intercept is 2, the line is $x_2 = 2x_1 + 2$ Rearranging into the perceptron boundary form gives $2 + 2x_1 - x_2 = 0$ Step 4: Read one possible set of coefficients Comparing $2 + 2x_1 - x_2 = 0$ with $w_0 + w_1x_1 + w_2x_2 = 0$ gives the candidate weights $w_0 = 2, \quad w_1 = 2, \quad w_2 = -1$	Point on the boundary	Substitution in $w_0 + w_1x_1 + w_2x_2 = 0$	(-1, 0)	$w_0 - w_1 = 0$, when $x_2=0$, $x_1=-1$	(0, 2)	$w_0 + 2w_2 = 0$, when $x_1=0$, $x_2=2$
Point on the boundary	Substitution in $w_0 + w_1x_1 + w_2x_2 = 0$						
(-1, 0)	$w_0 - w_1 = 0$, when $x_2=0$, $x_1=-1$						
(0, 2)	$w_0 + 2w_2 = 0$, when $x_1=0$, $x_2=2$						

	Step 5: Check the class orientation The line alone does not determine which side is positive. The figure shows that the origin (0, 0) lies in the negative region. Candidate score at (0, 0) = $2 + 2(0) - 0 = 2 > 0$ This candidate therefore labels the origin positive, which is opposite to the figure. Multiplying all coefficients by -1 preserves the same boundary but reverses the two class regions. $-2 - 2x_1 + x_2 = 0$ Step 6: Verify the corrected orientation <table><tr><th>Test point</th><th>Location in the figure</th><th>Score: $-2 - 2x_1 + x_2$</th><th>Classification</th></tr><tr><td>(0, 0)</td><td>Negative side</td><td>-2</td><td>Negative</td></tr><tr><td>(-1, 2)</td><td>Positive side</td><td>2</td><td>Positive</td></tr></table> Final Answer $w_0 = -2, w_1 = -2, w_2 = 1$	Test point	Location in the figure	Score: $-2 - 2x_1 + x_2$	Classification	(0, 0)	Negative side	-2	Negative	(-1, 2)	Positive side	2	Positive
Test point	Location in the figure	Score: $-2 - 2x_1 + x_2$	Classification										
(0, 0)	Negative side	-2	Negative										
(-1, 2)	Positive side	2	Positive										
Marks	4												
-ve applicable?	N												
% Applicable	N												

Question No	15																									
Question Type																										
Difficulty level	Difficult																									
Chapter	Unit 2																									
Topic	Perceptrons																									
Question	Apply the perceptron model to implement the two-input AND logic gate. Show the input values, selected weights, bias, net input, and output for all possible input combinations.																									
Long Answer	For an AND gate, the output is 1 only when both inputs are 1. Perceptron model: $z = w_1x_1 + w_2x_2 + b$ $y = 1$, if $z \geq 0$; otherwise $y = 0$ Choose the parameters: $w_1 = 0.25, w_2 = 0.25, b = -0.375$ Thus, the net input is: $z = 0.25x_1 + 0.25x_2 - 0.375$ <table><tr><th>x_1</th><th>x_2</th><th>Net input z</th><th>Condition</th><th>Output y</th></tr><tr><td>0</td><td>0</td><td>-0.375</td><td>$z < 0$</td><td>0</td></tr><tr><td>0</td><td>1</td><td>-0.125</td><td>$z < 0$</td><td>0</td></tr><tr><td>1</td><td>0</td><td>-0.125</td><td>$z < 0$</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0.125</td><td>$z \geq 0$</td><td>1</td></tr></table> Conclusion: The selected weights and bias produce output 1 only for the input (1, 1). Therefore, the perceptron correctly implements the AND logic gate.	x_1	x_2	Net input z	Condition	Output y	0	0	-0.375	$z < 0$	0	0	1	-0.125	$z < 0$	0	1	0	-0.125	$z < 0$	0	1	1	0.125	$z \geq 0$	1
x_1	x_2	Net input z	Condition	Output y																						
0	0	-0.375	$z < 0$	0																						
0	1	-0.125	$z < 0$	0																						
1	0	-0.125	$z < 0$	0																						
1	1	0.125	$z \geq 0$	1																						
Marks	4																									
-ve applicable?	N																									
% Applicable	N																									

Question No	16				
Question Type					
Difficulty level	Difficult				
Chapter	Unit 2				
Topic	Perceptrons				
Question	Apply the perceptron model to implement the two-input OR logic gate. Show the input values, selected weights, bias, net input, and output for all possible input combinations.				
Long Answer	Perceptron model: $z = w_1x_1 + w_2x_2 + b$ $y = 1$, if $z \geq 0$; otherwise $y = 0$ Selected parameters: $w_1 = 0.25$, $w_2 = 0.25$, $b = -0.125$ $z = (0.25)x_1 + (0.25)x_2 + (-0.125)$				
	x_1	x_2	Net input z	Condition	Output y
	0	0	-0.125	$z < 0$	0
	0	1	0.125	$z \geq 0$	1
	1	0	0.125	$z \geq 0$	1
	1	1	0.375	$z \geq 0$	1
	Conclusion: The selected parameters produce output 1 when at least one input is 1. Therefore, the perceptron correctly implements the OR gate.				
Marks	4				
-ve applicable?	N				
% Applicable	N				

Question No	17				
Question Type					
Difficulty level	Difficult				
Chapter	Unit 2				
Topic	Perceptrons				
Question	Apply the perceptron model to implement the two-inputNAND logic gate. Show the input values, selected weights, bias, net input, and output for all possible input combinations.				
Long Answer	Perceptron model: $z = w_1x_1 + w_2x_2 + b$ $y = 1$, if $z \geq 0$; otherwise $y = 0$ Selected parameters: $w_1 = -0.25$, $w_2 = -0.25$, $b = 0.375$ $z = (-0.25)x_1 + (-0.25)x_2 + (0.375)$				
	x_1	x_2	Net input z	Condition	Output y
	0	0	0.375	$z \geq 0$	1
	0	1	0.125	$z \geq 0$	1
	1	0	0.125	$z \geq 0$	1
	1	1	-0.125	$z < 0$	0
	Conclusion: The selected parameters produce output 0 only when both inputs are 1. Therefore, the perceptron correctly implements the NAND gate.				
Marks	4				
-ve applicable?	N				
% Applicable	N				

Question No	18
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Activation Functions
Question	Explain the Step (threshold) activation function and state why it is unsuitable for gradient-based training.
Long Answer	The Step function is defined as $f(x) = 1$ if $x \geq 0$, and $f(x) = 0$ otherwise. Its output range is the discrete set $\{0, 1\}$. It is unsuitable for gradient-based training because: • Its derivative is 0 everywhere it is defined, and undefined at $x = 0$. • A zero gradient gives no direction for the update, so the weights cannot be corrected. • The output is discontinuous, so a small change in the input produces no change until the threshold is crossed.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	19
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Optimizers
Question	List four important features of Adam that distinguish it from plain SGD.
Long Answer	1. Adaptive per-parameter learning rates using the second-moment estimate 2. Momentum-like smoothing through the first-moment estimate 3. Bias correction for the first- and second-moment estimates during early iterations 4. Effective handling of noisy or sparse gradients through adaptive scaling
Marks	4
-ve applicable?	N
% Applicable	N

Question No	20
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Support Vector Machines
Question	Compare support vectors and the hyperplane in detail.
Long Answer	Support vectors are the training samples that lie closest to the decision boundary. In canonical hard-margin form they satisfy $y_i(w \cdot x_i + b) = 1$, so they lie on the two supporting hyperplanes and define the margin. They are the active constraints of the hard-margin solution: moving or removing a support vector can change the solution, whereas samples strictly outside the margin normally do not alter the current maximum-margin boundary. The hyperplane is the decision surface $w \cdot x + b = 0$ that separates the samples of different classes. It is a line in two dimensions, a plane in three, and a hyperplane in higher dimensions. Its position and orientation are determined entirely by the support vectors, and the margin around it is $2/\ w\$.
Marks	4

-ve applicable?	N
% Applicable	N

Question No	21
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Activation Functions
Question	What are the advantages of the Tanh function?
Long Answer	• Zero-centered output in the range $(-1, +1)$, so the activations passed on are not all of the same sign • Stronger gradients than Sigmoid, since its maximum derivative is 1 against 0.25 for Sigmoid • Faster convergence than Sigmoid in many cases • Useful in hidden layers of shallow networks
Marks	4
-ve applicable?	N
% Applicable	N

Question No	22
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Activation Functions
Question	Justify why ReLU is preferred over Sigmoid.
Long Answer	• Faster computation, requiring only a comparison with zero rather than an exponential • No saturation for positive inputs, where the gradient remains exactly 1 • Reduces the vanishing gradient problem in deep networks • Produces sparse activations, since every negative input gives an output of 0. This can encourage sparse internal representations; actual computational savings occur only when the software or hardware implementation exploits that sparsity.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	23
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Support Vector Machines
Question	Define the margin of an SVM and explain what is meant by the maximum-margin hyperplane.
Long Answer	Margin: the width of the empty band around the decision boundary that contains no training samples. It is bounded by the two supporting hyperplanes, each passing through the nearest sample of one class. In canonical form the margin is $2/\ w\$, and each supporting hyperplane lies $1/\ w\$ from the boundary. Maximum-margin hyperplane: for a linearly separable dataset there are infinitely

	many separating hyperplanes. The maximum-margin hyperplane is the one for which this band is widest. Since the margin is $2/\ w\ $, the widest band corresponds to the smallest $\ w\ $. The samples lying on the supporting hyperplanes are the support vectors, and they alone decide which hyperplane is chosen.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	24
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Optimizers
Question	Explain Batch Gradient Descent and state two advantages and two disadvantages.
Long Answer	Batch Gradient Descent computes the gradient of the loss over the entire training set and then makes a single parameter update: $w_{\text{new}} = w_{\text{old}} - \eta \cdot (1/N) \sum_i \nabla L_i(w)$, for N training samples. One pass over the data therefore produces one update. Advantages: • The gradient is exact, so the descent direction is accurate and the path is stable. • The computation vectorizes well over the whole dataset. Disadvantages: • Every update needs a full pass over all N samples, which is slow on large datasets. • The full-batch gradient is deterministic and contains no sampling noise. In non-convex optimization this provides less stochastic exploration than SGD or mini-batch methods, although it does not mean that Batch Gradient Descent must get trapped in a poor local minimum.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	25		
Question Type			
Difficulty level	Easy		
Chapter	2		
Topic	Activation Functions		
Question	Compare the Sigmoid and Tanh activation functions.		
Long Answer	Property	Sigmoid	Tanh
	Expression	$1 / (1 + e^{-x})$	$(e^x - e^{-x}) / (e^x + e^{-x})$
	Output range	(0, 1)	(-1, +1)
	Zero-centered	No	Yes

	Derivative	$f(x)(1 - f(x))$	$1 - f(x)^2$
	Maximum derivative	0.25 at $x = 0$	1 at $x = 0$
Both saturate at their extremes, so both can contribute to vanishing gradients in deep networks. Tanh is often preferred in hidden layers because it is zero-centered, and its maximum derivative is 1 compared with 0.25 for Sigmoid. Thus the maximum derivative is four times larger, but the gradient is not necessarily four times larger at every input. Sigmoid remains useful in the output layer of a binary classifier, where an output in (0, 1) is required.			
Marks	4		
-ve applicable?	N		
% Applicable	N		

Question No	26
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Optimizers
Question	List and explain the main limitations of plain Gradient Descent that motivated the development of adaptive optimizers.
Long Answer	• A single fixed learning rate is applied to every parameter, although different parameters may need very different step sizes. • That rate must be tuned by hand: too small gives slow convergence, too large causes divergence. • It oscillates across steep directions while creeping along shallow ones. • It stalls on plateaus and near saddle points, where the gradient is close to zero. • It handles sparse features poorly, since rare features receive the same step size as frequent ones. Momentum addresses the oscillation and the plateaus, Adagrad addresses the per-parameter step size, and Adam combines both.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	27
Question Type	
Difficulty level	Medium
Chapter	2
Topic	Activation Functions
Question	A hospital develops an AI-based disease prediction model. The neural network receives two normalized patient parameters: blood test score (x_1) = 2 and risk factor score (x_2) = -1. The neuron uses the weights $w_1 = 1.5$ and $w_2 = 2$. The bias is 1, and the activation function used is ReLU. Find the neuron's output after applying the ReLU activation function.
Long Answer	Weighted sum: $z = (1.5)(2) + (2)(-1) + 1 = 3 - 2 + 1 = 2$ ReLU activation:

	$\text{ReLU}(2) = \max(0, 2) = 2$ Final Answer: 2
Marks	4
-ve applicable?	N
% Applicable	N

Question No	28
Question Type	
Difficulty level	Medium
Chapter	2
Topic	Optimizers
Question	A machine learning model has an initial weight $w = 5$ and a learning rate $\eta = 0.1$. The gradient of the loss function with respect to the weight is 2 at the first iteration and 1.5 at the second iteration. Calculate the updated weight after two iterations of Gradient Descent.
Long Answer	Gradient Descent update rule: $w_{\text{new}} = w - \eta(\partial J / \partial w)$ Iteration 1: $w_1 = 5 - (0.1 \times 2) = 5 - 0.2 = 4.8$ Iteration 2: $w_2 = 4.8 - (0.1 \times 1.5) = 4.8 - 0.15 = 4.65$ Final Answer: Updated Weight = 4.65
Marks	4
-ve applicable?	N
% Applicable	N

Question No	29
Question Type	
Difficulty level	Medium
Chapter	2
Topic	Optimizers
Question	Explain the Adam optimizer and discuss why it is widely used in deep learning.
Long Answer	Adam (Adaptive Moment Estimation) combines the advantages of Momentum and Adagrad by maintaining estimates of both the first and the second moments of the gradients. In the equations below $g_t = \nabla J(w_t)$ is the gradient at iteration t and η is the learning rate. Update equations: - First moment (mean): $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ - Second moment (uncentered variance): $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ - Bias correction: $\hat{m}_t = m_t / (1 - \beta_1^t)$ and $\hat{v}_t = v_t / (1 - \beta_2^t)$ - Weight update: $w = w - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ Advantages: - Fast convergence - Adaptive learning rate for every parameter - Handles noisy and sparse gradients effectively - Suitable for large datasets and deep neural networks, with little tuning required
Marks	4
-ve applicable?	N
% Applicable	N

Question No	30
Question Type	
Difficulty level	Medium
Chapter	2
Topic	Support Vector Machines
Question	Explain why a larger margin leads to better generalization, using the idea of tolerance to noise in the training data.
Long Answer	Consider a sample near a decision boundary. A small perturbation can move it across the boundary, whereas a sample farther from the boundary can tolerate a larger perturbation before its predicted class changes. Tolerance to noise: in canonical hard-margin SVM, the closest training samples are at distance $1/\ w\$ from the decision boundary. A perturbation smaller than a sample's own perpendicular distance to the boundary cannot move that sample across the boundary. Therefore, a wider margin increases the minimum geometric buffer around the training samples. Capacity control: requiring a wide margin rules out every hyperplane that squeezes between the classes, so fewer hypotheses remain admissible and the classifier is less able to fit accidental patterns in the training data. The maximum-margin boundary is therefore generally more robust to small perturbations and can improve generalization. This is a robustness argument, not a guarantee that every possible test sample will be classified correctly.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	31
Question Type	
Difficulty level	Medium
Chapter	2
Topic	Support Vector Machines
Question	Explain the difference between the functional margin and the geometric margin of a training sample, and state why the geometric margin is used to define the SVM objective.
Long Answer	Functional margin: for a sample (x_i, y_i), $\hat{\gamma}_i = y_i(w \cdot x_i + b)$. It is positive when the sample is on the correct side, and its size indicates confidence. Geometric margin: the perpendicular distance from the sample to the hyperplane, $\gamma_i = y_i(w \cdot x_i + b) / \ w\$. The two are related by $\gamma_i = \hat{\gamma}_i / \ w\$. Why the geometric margin is used: replacing (w, b) by $(2w, 2b)$ leaves the hyperplane $w \cdot x + b = 0$ unchanged but doubles every functional margin, so maximizing the functional margin is not a well-posed problem. The geometric margin is unchanged by this rescaling, because both the numerator and $\ w\$ scale by the same factor. Canonical form fixes the functional margin of the closest samples at 1, which removes the scaling freedom and makes their geometric margin $1/\ w\$, so the full margin is $2/\ w\$.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	32		
Question Type			
Difficulty level	Difficult		
Chapter	2		
Topic	Optimizers		
Question	Compare Momentum, Adagrad and Adam in terms of how each modifies the plain Gradient Descent update, and state a situation in which each is preferable.		
Long Answer	Plain Gradient Descent: $w \leftarrow w - \eta g$, where $g = \nabla J(w)$ is the gradient and η the learning rate. One fixed step size for every parameter, in the direction of the current gradient only.		
	Optimizer	How it modifies the update	What it fixes
	Momentum	$v_t = \beta v_{t-1} + \eta g$; $w \leftarrow w - v_t$. The direction is a running average of past gradients.	Oscillation and slow progress along shallow directions. The step size is still the same for every parameter.
	Adagrad	$G_t = G_{t-1} + g^2$; $w \leftarrow w - (\eta/(\sqrt{G_t} + \epsilon))g$. Each parameter gets its own step size.	The single global learning rate. It does not smooth the direction, and the step decays monotonically.
	Adam	Keeps m_t and v_t , applies bias correction, and updates $w \leftarrow w - \eta \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$.	Both problems at once. The decaying average replaces Adagrad's running sum, so the step does not collapse.
	When each is preferable: • Momentum: dense, well-scaled features on a surface with long narrow valleys. • Adagrad: sparse, high-dimensional features such as text. Unsuitable for long runs, since the accumulated sum eventually stalls learning. • Adam: the default when tuning time is short or parameters differ widely in scale.		
Marks	4		

-ve applicable?	N
% Applicable	N

Question No	33
Question Type	
Difficulty level	Difficult
Chapter	2
Topic	Activation Functions
Question	Discuss the mathematical properties of ReLU and how it impacts training speed.
Long Answer	The Rectified Linear Unit (ReLU) is one of the most commonly used activation functions in deep learning, due to its simplicity and efficiency. $\text{ReLU}(x) = \max(0, x)$ It returns the input directly if it is positive; otherwise it returns 0. Mathematical Properties: • Piecewise linear function: $\text{ReLU}(x) = 0$ if $x < 0$, and x if $x \geq 0$ • Non-linearity: although it is linear for $x > 0$, the kink at $x = 0$ introduces the non-linearity that a network requires • Derivative (used in backpropagation): 0 if $x < 0$, and 1 if $x > 0$ • Range: $[0, \infty)$ • Not differentiable at $x = 0$, handled in practice using sub-gradient methods Impact on training speed: • Efficient computation: ReLU is a single comparison with zero, which is faster than the exponentials required by Sigmoid or Tanh. • Sparse activation: for negative inputs the neuron outputs 0, which encourages sparse representations. In a dense implementation this does not automatically reduce the arithmetic cost of a training step; a computational speedup requires the implementation to exploit sparsity. • Reduces the risk of vanishing gradients: for positive inputs the local derivative of ReLU is 1, so ReLU itself does not shrink the backpropagated gradient at that unit. However, the overall network gradient can still vanish or explode because it is also multiplied by weights and by derivatives in other layers.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	34																				
Question Type																					
Difficulty level	Difficult																				
Chapter	2																				
Topic	Support Vector Machines																				
Question	An SVM trained on a two-dimensional dataset gives the hyperplane $w = (1, 1)$ and $b = -3$, in canonical form. The training samples are A(2, 2) with $y = +1$, B(1, 1) with $y = -1$, C(3, 2) with $y = +1$ and D(0, 1) with $y = -1$. Identify the support vectors and calculate the margin.																				
Long Answer	Decision function: $f(x) = x_1 + x_2 - 3$. A sample is a support vector when $y_i f(x_i) = 1$, that is, when it lies on a supporting hyperplane. <table><tr><th>Sample</th><th>$f(x)$</th><th>$y f(x)$</th><th>Support vector?</th></tr><tr><td>A(2, 2)</td><td>1</td><td>1</td><td>Yes</td></tr><tr><td>B(1, 1)</td><td>-1</td><td>1</td><td>Yes</td></tr><tr><td>C(3, 2)</td><td>5</td><td>5</td><td>No</td></tr><tr><td>D(0, 1)</td><td>-3</td><td>3</td><td>No</td></tr></table>	Sample	$f(x)$	$y f(x)$	Support vector?	A(2, 2)	1	1	Yes	B(1, 1)	-1	1	Yes	C(3, 2)	5	5	No	D(0, 1)	-3	3	No
Sample	$f(x)$	$y f(x)$	Support vector?																		
A(2, 2)	1	1	Yes																		
B(1, 1)	-1	1	Yes																		
C(3, 2)	5	5	No																		
D(0, 1)	-3	3	No																		

	A(2, 2)	$2 + 2 - 3 = 1$	$(+1)(1) = 1$	Yes
	B(1, 1)	$1 + 1 - 3 = -1$	$(-1)(-1) = 1$	Yes
	C(3, 2)	$3 + 2 - 3 = 2$	$(+1)(2) = 2$	No
	D(0, 1)	$0 + 1 - 3 = -2$	$(-1)(-2) = 2$	No
Support vectors: A(2, 2) and B(1, 1). C and D are correctly classified and lie outside the margin, so they do not influence the boundary. Margin: $\ w\ = \sqrt{1^2 + 1^2} = \sqrt{2} = 1.414$ Margin = $2/\ w\ = 2/1.414 = 1.414$ units The distance from the hyperplane to each support vector is $1/\ w\ = 0.707$, and the margin is twice this value.				
Marks	4			
-ve applicable?	N			
% Applicable	N			

Question No	35
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	What is a Support Vector Machine (SVM)? State its applications.
Long Answer	SVM is a supervised machine learning algorithm. It is a linear classifier that finds the optimal hyperplane to separate classes. It maximizes the margin between the two classes. The optimal hyperplane is obtained using Lagrange multipliers. Applications: Binary classification (main use) Regression (SVR) Pattern recognition
Marks	4
-ve applicable?	N
% Applicable	N

Question No	36
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain the concepts of Hyperplane and Support Vectors in Support Vector

	Machines (SVM) with suitable equations.
Long Answer	Hyperplane 1. A hyperplane is the decision boundary that separates different classes in an SVM. 2. SVM chooses the optimal hyperplane that maximizes the margin between the classes. 3. The general equation of a hyperplane is: $w^T x + b = 0$ where: 1. w = weight vector (normal to the hyperplane) 2. b = bias/intercept 4. The margin boundaries (gutters) are: $w^T x + b = 1$ $w^T x + b = -1$ Support Vectors 1. Support vectors are the training data points that lie on the margin boundaries. 2. They are the closest points to the hyperplane and determine its position. 3. Removing non-support vectors usually does not change the hyperplane, but removing support vectors can change it. 4. In the dual formulation, support vectors have non-zero Lagrange multipliers ($\alpha_i > 0$), while all other points have $\alpha_i = 0$. Classification Constraint For every training sample, $y_i(w^T x_i + b) \geq 1$ where: 1. y_i = class label (+1 or -1) 2. x_i = training sample 3. Diagram to draw in the exam: Draw three parallel lines labeled $w^T x + b = 1$, $w^T x + b = 0$, and $w^T x + b = -1$, with the points on the outer two lines marked as support vectors.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	37
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain Hard Margin SVM.
Long Answer	Hard Margin SVM is used when the training data is perfectly linearly separable , i.e., a straight line (or hyperplane) can separate the two classes without any misclassification.

	The objective of Hard Margin SVM is to find the optimal hyperplane that maximizes the margin, resulting in better generalization on unseen data. The decision boundary (optimal hyperplane) is: $w^T x + b = 0$ The two margin boundaries (gutters) are: $w^T x + b = 1$ $w^T x + b = -1$ The distance between these two boundaries is called the margin width, given by: $\text{Margin Width} = \frac{2}{\ w\ }$ To ensure that every training sample is correctly classified and lies outside the margin, the following constraint is imposed: $y_i(w^T x_i + b) \geq 1, i = 1, 2, \dots, m$ where: 1. y_i= class label (+1 or -1) 2. x_i= training sample 3. m= number of training samples The optimization problem is: $\min \frac{1}{2} \ w\ ^2$ subject to $y_i(w^T x_i + b) \geq 1$ Since Hard Margin SVM does not allow any misclassification, it is sensitive to outliers and performs well only when the data is perfectly linearly separable.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	38
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Differentiate between Hard Margin and Soft Margin SVM.

Long Answer	Hard Margin SVM	Soft Margin SVM
	Used when data is perfectly linearly separable.	Used when data is not perfectly linearly separable.
	Does not allow any misclassification.	Allows some misclassification using slack variables.
	Sensitive to outliers and noisy data.	More robust to outliers and noisy data.
	May overfit if the training data contains noise.	Better generalization on real-world datasets.
	Constraint: $y_i(w^T x_i + b) \geq 1$	Constraint: $y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$
	Objective: $\min \frac{1}{2} \ w\ ^2$	Objective: $\min \frac{1}{2} \ w\ ^2 + C \sum_i \xi_i$
Marks	4	
-ve applicable?	N	
% Applicable	N	

Question No	39
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	What are Kernel Functions? Explain any three.
Long Answer	Kernel functions compute the dot product in a higher-dimensional feature space without explicitly transforming the data. Types: Linear Kernel: Used for linearly separable data. Polynomial Kernel: Captures nonlinear relationships; high degree may cause overfitting. RBF (Gaussian) Kernel: Most commonly used and generally performs well.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	40
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	State the KKT conditions in SVM. Explain the purpose of each.

Long Answer	1. Primal Feasibility $y_i(w^T x_i + b) - 1 \geq 0$ 1. Ensures all training samples satisfy the SVM classification constraint. 2. Dual Feasibility $\alpha_i \geq 0$ 1. Ensures the Lagrange multipliers are valid (non-negative). 3. Complementary Slackness $\alpha_i [y_i(w^T x_i + b) - 1] = 0$ 1. Identifies the support vectors. Only support vectors have $\alpha_i > 0$. 4. Stationarity $\nabla_w L = 0, \nabla_b L = 0$ 1. Gives the optimal values of w and b, ensuring the optimal separating hyperplane is obtained. Summary: KKT conditions ensure the SVM solution is feasible, optimal, and correctly identifies the support vectors.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	41
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain the role of the regularization parameter C in Soft Margin SVM.
Long Answer	1. controls the trade-off between maximizing the margin and minimizing classification errors. 2. Small C: - Larger margin - More classification errors allowed 3. Large C: - Fewer classification errors - Smaller margin 4. Objective function: $\min \frac{1}{2} \ w\ ^2 + C \sum_i \xi_i$
Marks	4
-ve applicable?	N
% Applicable	N

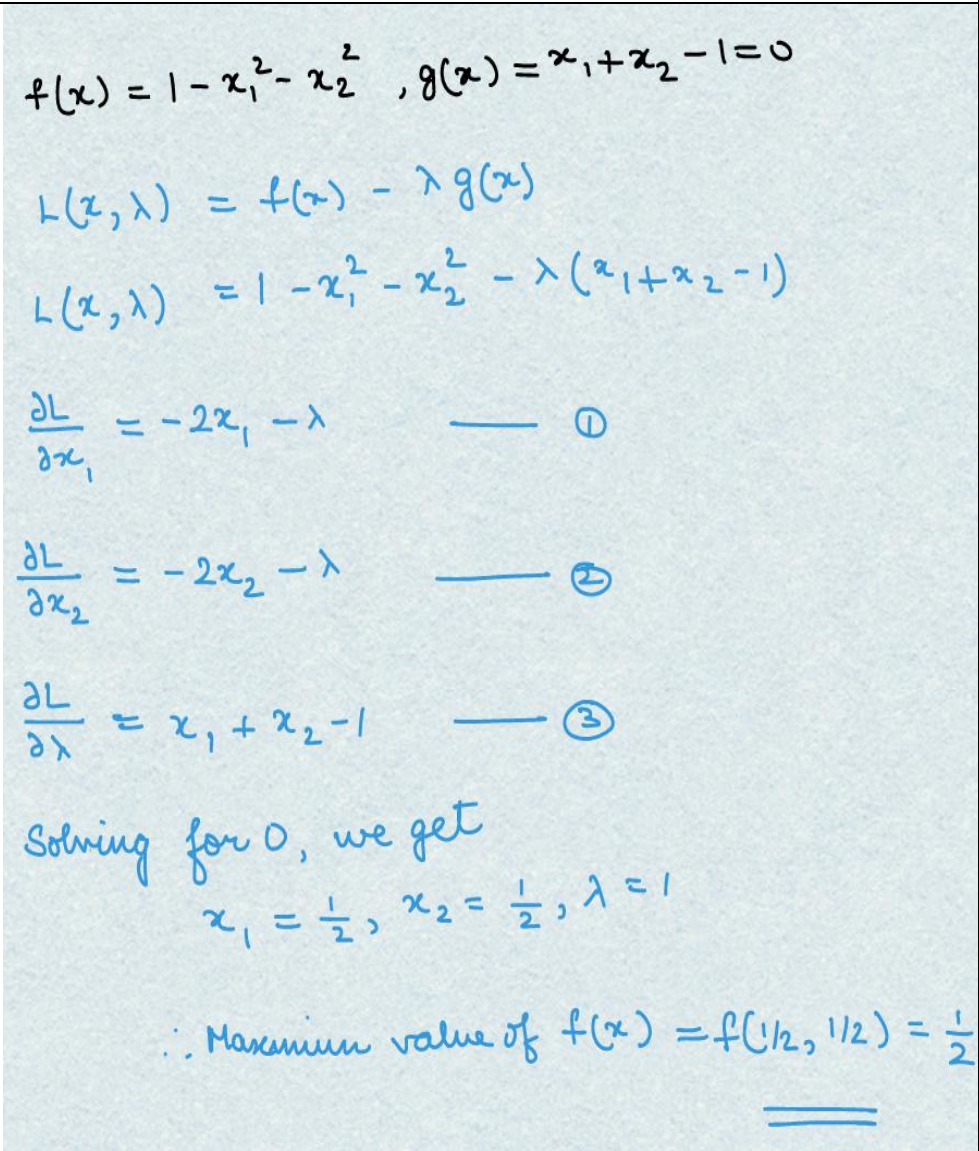
Question No	42
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	A dataset contains two perfectly separable classes. Which type of SVM would you choose? Justify your answer.
Long Answer	1. Use Hard Margin SVM. 2. Data is perfectly linearly separable. 3. It finds the hyperplane with the maximum margin. 4. Constraints: $y_i(w^T x_i + b) \geq 1$ 5. Margin width: $\frac{2}{\ w\ }$ 6. Gives good generalization when there are no outliers.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	43
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	A medical dataset contains noisy samples and a few incorrectly labelled records. Which SVM variant is more suitable? Explain.
Long Answer	1. Use Soft Margin SVM. 2. Hard Margin fails because data is not perfectly separable. 3. Soft Margin introduces slack variables (ξ_i). 4. It allows some misclassification while maximizing the margin. 5. Objective: 6. $\min \frac{1}{2} \ w\ ^2 + C \sum_i \xi_i$ 7. More robust to noisy datasets.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	44
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	A classification problem contains data that cannot be separated by a straight

	line. How does SVM solve this problem?
Long Answer	1. Use Kernel Functions. 2. Kernels map data into a higher-dimensional feature space. 3. Classification is performed in the transformed space. 4. Common kernels: 1. Linear 2. Polynomial 3. RBF (Gaussian) 5. RBF is generally recommended because it performs well for most nonlinear problems.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	45
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain the role of the regularization parameter C in Soft Margin SVM. Discuss the effect of increasing and decreasing its value.
Long Answer	The regularization parameter C controls the trade-off between maximizing the margin and minimizing classification errors in a Soft Margin SVM. The objective function is: $\min \frac{1}{2} \ w\ ^2 + C \sum_i \xi_i$ Effects of C: 1. Small C: 1. Gives more importance to maximizing the margin. 2. Allows more classification errors. 3. Produces a wider margin. 4. Better generalization and less prone to overfitting. 2. Large C: 1. Gives more importance to minimizing classification errors. 2. Allows fewer misclassifications. 3. Produces a narrower margin. 4. May lead to overfitting if the training data contains noise. Summary: 1. A small C prioritizes a larger margin and tolerates some errors. 2. A large C prioritizes correct classification of training samples, even at the cost of a smaller margin.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	46
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Using the method of Lagrange Multipliers, find the maximum value of $f(x) = 1 - x_1^2 - x_2^2$ subject to the constraint $x_1 + x_2 - 1 = 0.$
Long Answer	 $f(x) = 1 - x_1^2 - x_2^2$, $g(x) = x_1 + x_2 - 1 = 0$ $L(x, \lambda) = f(x) - \lambda g(x)$ $L(x, \lambda) = 1 - x_1^2 - x_2^2 - \lambda(x_1 + x_2 - 1)$ $\frac{\partial L}{\partial x_1} = -2x_1 - \lambda \quad \text{--- ①}$ $\frac{\partial L}{\partial x_2} = -2x_2 - \lambda \quad \text{--- ②}$ $\frac{\partial L}{\partial \lambda} = x_1 + x_2 - 1 \quad \text{--- ③}$ Solving for 0, we get $x_1 = \frac{1}{2}, x_2 = \frac{1}{2}, \lambda = 1$ $\therefore$ Maximum value of $f(x) = f(1/2, 1/2) = \frac{1}{2}$
Marks	4
-ve applicable ?	N
%	N

Applicable	
------------	--

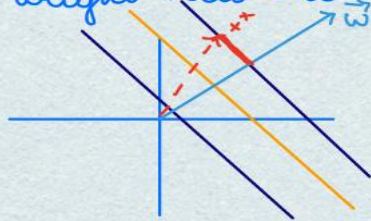
Question No	47
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain how Support Vector Machine (SVM) can be used for binary classification with a suitable example.
Long Answer	Support Vector Machine (SVM) is a supervised learning algorithm used primarily for binary classification. Consider the problem of classifying emails as Spam or Not Spam. - Each email is represented as a feature vector. - The SVM is trained to find the optimal hyperplane that maximizes the margin between the two classes. - The decision boundary is given by: $w^T x + b = 0$ - A new email is classified as: - Spam, if $w^T x + b > 0$ - Not Spam, if $w^T x + b < 0$ - If the data is non-linearly separable, a kernel function (such as the RBF kernel) is used to map the data into a higher-dimensional space where it can be separated effectively. Thus, SVM performs accurate binary classification by maximizing the margin between classes while achieving good generalization.
Marks	4
-ve applicable?	N
% Applicable	N

Question No	48
Question Type	
Difficulty level	Difficult
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Derive the expression for the width of the margin (canal) in a Hard Margin Support Vector Machine (SVM).

Long Answer

Calculating width of the canal

Project a positive point x^+ on the hyperplane $w^T x + b = 1$ on the weight vector $\vec{w}$



Find the length of the projection

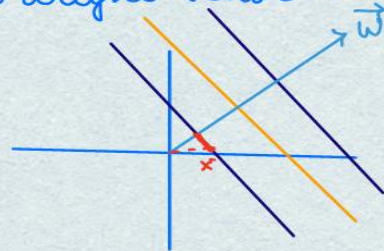
$$= \hat{w} \vec{x}^+$$

$$\text{where } \hat{w} = \frac{w}{\|w\|}$$

$$w^T x^+ + b = 1$$

$$w^T x^+ = 1 - b$$

Project a negative point x^- on the hyperplane $w^T x + b = -1$ on the weight vector $\vec{w}$



Find the length of the projection vector

$$= \hat{w} \vec{x}^-$$

$$\text{where } \hat{w} = \frac{w}{\|w\|}$$

$$w^T x^- + b = -1$$

$$w^T x^- = -1 - b$$

Divide both sides by $\|w\|$

$$\frac{w^T}{\|w\|} x^+ = \frac{1-b}{\|w\|} \quad \& \quad \frac{w^T x^-}{\|w\|} = \frac{-1-b}{\|w\|}$$

$$\hat{w} x^+ = \frac{1-b}{\|w\|} \quad \& \quad \hat{w} x^- = \frac{-1-b}{\|w\|}$$

$$\text{Now Width} = \hat{w} x^+ - \hat{w} x^-$$

$$\Rightarrow \text{Width} = \frac{1-b}{\|w\|} - \left(\frac{-1-b}{\|w\|} \right)$$

$$\boxed{\text{Width} = \frac{2}{\|w\|}}$$

Marks

4

-ve applicable?

N

% Applicable

N

Question No	49																		
Question Type																			
Difficulty level	Difficult																		
Chapter	Unit 2																		
Topic	Support Vector Machines (SVM)																		
Question	Consider the following linearly separable dataset: 1. Is the data linearly separable? ✓ 2. Discover a simple SVM that accurately discriminates the two classes. <table border="1" style="margin-left: auto; margin-right: auto;"> <thead> <tr> <th>point</th><th>class</th></tr> </thead> <tbody> <tr><td>(3,1)</td><td>+</td></tr> <tr><td>(3,-1)</td><td>+</td></tr> <tr><td>(6,1)</td><td>+</td></tr> <tr><td>(6,-1)</td><td>+</td></tr> <tr><td>(1,0)</td><td>-</td></tr> <tr><td>(0,1)</td><td>-</td></tr> <tr><td>(0,-1)</td><td>-</td></tr> <tr><td>(-1,0)</td><td>-</td></tr> </tbody> </table>	point	class	(3,1)	+	(3,-1)	+	(6,1)	+	(6,-1)	+	(1,0)	-	(0,1)	-	(0,-1)	-	(-1,0)	-
point	class																		
(3,1)	+																		
(3,-1)	+																		
(6,1)	+																		
(6,-1)	+																		
(1,0)	-																		
(0,1)	-																		
(0,-1)	-																		
(-1,0)	-																		
Long Answer	<i>Binding constraints :</i> $w^T x^+ + b = 1$ $w^T x^- + b = -1$ <u>Method - 1</u> For this q, support vectors are $S_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, S_2 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, S_3 = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$ The equations are, $\begin{aligned} w^T S_1 + b &= -1 \\ w^T S_2 + b &= +1 \\ w^T S_3 + b &= +1 \end{aligned} \quad \left. \begin{aligned} & \\ & \\ & \end{aligned} \right\} \begin{aligned} w_1 + b &= -1 \\ 3w_1 + w_2 + b &= +1 \\ 3w_1 - w_2 + b &= 1 \end{aligned}$ Solving the 3 we get, $w = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, b = -2$ <u>=====</u>																		

Method - 2

constraints : $\alpha_i \geq 0$

$$C1 \quad \sum_{i=1}^m \alpha_i y_i \geq 0$$

$$C2 \quad \sum_{i=1}^m \alpha_i y_i x_i \cdot x + b = 1$$

$$C3 \quad \sum_{i=1}^m \alpha_i y_i x_i \cdot x + b = -1$$

$$S_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \cdot 1$$

$$S_2 = \begin{bmatrix} 3 \\ 1 \end{bmatrix} \cdot +1$$

$$S_3 = \begin{bmatrix} 3 \\ -1 \end{bmatrix} \cdot +1$$

Applying C_1 on the support vectors,

$$\alpha_1(-1) + \alpha_2(1) + \alpha_3(1) = 0$$

$$\Rightarrow -\alpha_1 + \alpha_2 + \alpha_3 = 0 \quad \text{————— ①}$$

Applying C_2 & C_3

$$-\alpha_1 S_1 S_1 + \alpha_2 S_1 S_2 + \alpha_3 S_1 S_3 + b = -1$$

$$-\alpha_1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \alpha_2 \begin{bmatrix} 1 \\ 0 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 1 \end{bmatrix} + \alpha_3 \begin{bmatrix} 1 \\ 0 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ -1 \end{bmatrix} + b = -1$$

$$-\alpha_1 + 3\alpha_2 + 3\alpha_3 + b = -1 \quad \text{————— ②}$$

$$-\alpha_1 S_2 S_1 + \alpha_2 S_2 S_2 + \alpha_3 S_2 S_3 = +1$$

$$-\alpha_1 \begin{bmatrix} 3 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \alpha_2 \begin{bmatrix} 3 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 1 \end{bmatrix} + \alpha_3 \begin{bmatrix} 3 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ -1 \end{bmatrix} + b = 1$$

$$-3\alpha_1 + 10\alpha_2 + 8\alpha_3 + b = 1 \quad \text{————— ③}$$

Similarly, 4th one

$$-3\alpha_1 + 8\alpha_2 + 10\alpha_3 + b = 1$$

Solving 1, 2, 3, 4,

$$\alpha_1 = \frac{1}{2}, \alpha_2 = \frac{1}{4}, \alpha_3 = \frac{1}{4}, b = -2$$

$$w = \sum_{i=1}^m \alpha_i y_i x_i$$

$$= \frac{1}{2}(-1) \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \frac{1}{4}(1) \begin{bmatrix} 3 \\ 1 \end{bmatrix} + \frac{1}{4}(1) \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

$$= \begin{bmatrix} -1/2 \\ 0 \end{bmatrix} + \begin{bmatrix} 3/4 \\ 1/4 \end{bmatrix} + \begin{bmatrix} 3/4 \\ -1/4 \end{bmatrix}$$

$$w = \underline{\underline{\begin{bmatrix} 1 \\ 0 \end{bmatrix}}}$$

Marks	4
-ve applicable?	N
% Applicable	N

Question No	50
Question Type	
Difficulty level	Difficult
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain the primal optimization problem of Soft Margin SVM. Write its objective function and constraints.

Long Answer

The primal problem is:

$$\min_{w, b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i$$

s. t. c.

$$y_i (w^T x_i + b) \geq 1 - \xi_i \quad \forall (x_i, y_i)$$

$$\text{and } \xi_i \geq 0$$

Writing the corresponding Lagrangian function by introducing two Lagrangian multipliers α_i and β_i one per constraint, we get

$$\min_{w, b, \xi} L = \frac{1}{2} w \cdot w + C \sum_{i=1}^m \xi_i - \sum_{i=1}^m \alpha_i [y_i (w^T x_i + b) - 1 + \xi_i] - \sum_{i=1}^m \beta_i \xi_i$$

Minimizing of L w.r.to w, b, ξ is same as Maximizing w.r.to. α and β .

$$(a) \min_{w, b, \xi} \max_{\alpha \geq 0, \beta \geq 0} L(w, b, \xi, \alpha, \beta)$$

subject to the KKT conditions for the solutions are given as:

1. $\nabla_w L = 0, \nabla_b L = 0, \nabla_{\xi_i} L = 0$ (Stationary Condition)
2. $y_i (w^T x_i + b) - 1 + \xi_i \geq 0$ (Primal Feasibility Condition)
3. $\alpha_i (y_i (w^T x_i + b) - 1 + \xi_i) = 0$ (Complementary Slackness Condition)
4. $\alpha_i \geq 0, \beta_i \geq 0$ (Dual Feasibility Condition)
5. $y_i (w^T x_i + b) - 1 + \xi_i = 0$
6. $\beta_i \xi_i = 0$

Marks	4
-ve applicable?	N
% Applicable	N

Question No	51
Question Type	
Difficulty level	Difficult
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Write the objective function and constraints of the primal SVM optimization problem. Explain why it is a convex optimization problem.
Long Answer	1. Margin width: $\text{Margin} = \frac{2}{\ w\ }$ 1. SVM aims to maximize the margin for better class separation. 2. Objective function: $\max \frac{2}{\ w\ }$ 1. A larger margin reduces the generalization error of the classifier. 3. Equivalent minimization: $\max \frac{2}{\ w\ } \equiv \min \frac{1}{2} \ w\ ^2$ 1. Minimizing $\frac{1}{2} \ w\ ^2$ is mathematically equivalent and easier to optimize. 4. Constraints: $y_i(w^T x_i + b) \geq 1, i = 1, \dots, m$ 1. Ensures every training sample is correctly classified and lies on or outside the margin. 5. Final Primal Optimization Problem: $\min_{w,b} \frac{1}{2} \ w\ ^2 \text{ subject to } y_i(w^T x_i + b) \geq 1$ 6. Note: 1. The above is a convex optimization problem and is solved using Lagrange duality.
Marks	4
-ve applicable?	N
% Applicable	N

Unit 2 – 6 Marks Descriptive Question Bank

Question No	52
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Back-propagation
Question	The diagram shows a fully connected neural network created by FinTrust Bank. Nodes 1–5 form the input layer, nodes 6–9 form the hidden layer, and node 10 is the output neuron. All connection weights are initially 0.5. The input layer uses the identity activation function. For the given training record, update the weight $W_{8,10}$, which connects hidden neuron 8 to output neuron 10. Use a learning rate $\eta = 0.1$ and target $T = 0$. Training record: Age = 32, Annual Income = 15, Credit Utilization = 62, Payment History = 0, Credit Score = 784. Cost function: $E = (T - O_{10})^2$. All input→hidden weights = 0.5 All hidden→output weights = 0.5 No biases are specified; therefore, biases are taken as zero.
Long Answer	Step 1: Input-layer outputs Because the input layer uses the identity activation, the outputs of nodes 1–5 are equal to their input values. $O_1 = 32, O_2 = 15, O_3 = 62, O_4 = 0, O_5 = 784$ Step 2: Net input to each hidden neuron Every input-to-hidden weight is 0.5, and each hidden neuron receives all five input values. Therefore, neurons 6, 7, 8, and 9 have the same net input. $net_j = 0.5(32 + 15 + 62 + 0 + 784), j \in \{6, 7, 8, 9\}$ $net_j = 0.5(893) = 446.5$ Step 3: Hidden-layer outputs $O_j = \sigma(net_j) = 1 / (1 + e^{(-net_j)})$ $O_6 = O_7 = O_8 = O_9 = \sigma(446.5) \approx 1.0000$ The net input is very large and positive, so the sigmoid neurons are saturated and their outputs are approximately 1. Step 4: Net input and output at node 10

	$\text{net}_{10} = 0.5O_6 + 0.5O_7 + 0.5O_8 + 0.5O_9$ $\text{net}_{10} = 0.5(1 + 1 + 1 + 1) = 2.0$ $O_{10} = \sigma(\text{net}_{10}) = \sigma(2.0) = 0.880797$ Step 5: Compute the squared error $E = (T - O_{10})^2$ $E = (0 - 0.880797)^2 = 0.775803$ Step 6: Compute the gradient for $W_{8,10}$ Using the chain rule: $\frac{\partial E}{\partial W_{8,10}} = (\frac{\partial E}{\partial O_{10}})(\frac{\partial O_{10}}{\partial \text{net}_{10}})(\frac{\partial \text{net}_{10}}{\partial W_{8,10}})$ $\frac{\partial E}{\partial O_{10}} = 2(O_{10} - T)$ $\frac{\partial O_{10}}{\partial \text{net}_{10}} = O_{10}(1 - O_{10})$ $\frac{\partial \text{net}_{10}}{\partial W_{8,10}} = O_8$ $\frac{\partial E}{\partial W_{8,10}} = 2(0.880797 - 0)(0.880797)(1 - 0.880797)(1.0000)$ $\frac{\partial E}{\partial W_{8,10}} = 0.184956$ Step 7: Update $W_{8,10}$ $W_{8,10}(\text{new}) = W_{8,10}(\text{old}) - \eta(\frac{\partial E}{\partial W_{8,10}})$ $W_{8,10}(\text{new}) = 0.5 - 0.1(0.184956)$ $W_{8,10}(\text{new}) = 0.481504 \approx 0.4815$
Marks	6
-ve applicable?	N
% Applicable	N

Question No	53																
Question Type																	
Difficulty level	Easy																
Chapter	Unit 2																
Topic	Perceptrons																
Question	Design a two-input perceptron that implements the Boolean function A AND (NOT B).																
Long Answer	Perceptron Convention For a perceptron with inputs A and B, the net input is $v = w_1A + w_2B + b$ The binary step rule is Output = 1 when $v \geq 0$; otherwise Output = 0 Design a Perceptron for A AND (NOT B) Step 1: Write the desired truth table <table><tr><th>A</th><th>B</th><th>NOT B</th><th>A AND (NOT B)</th></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td></tr></table>	A	B	NOT B	A AND (NOT B)	0	0	1	0	0	1	0	0	1	0	1	1
A	B	NOT B	A AND (NOT B)														
0	0	1	0														
0	1	0	0														
1	0	1	1														

1	1	0	0
---	---	---	---

Step 2: Convert the truth-table rows into inequalities

The weighted sum must be nonnegative for the only true row, $A = 1$ and $B = 0$, and negative for the remaining three rows.

- For $A = 1, B = 0$ and output 1:
 $w_1(1) + w_2(0) + b \geq 0 \Rightarrow w_1 + b \geq 0$
- For $A = 0, B = 0$ and output 0:
 $w_1(0) + w_2(0) + b < 0 \Rightarrow b < 0$
- For $A = 1, B = 1$ and output 0:
 $w_1(1) + w_2(1) + b < 0 \Rightarrow w_1 + w_2 + b < 0$
- For $A = 0, B = 1$ and output 0:
 $w_1(0) + w_2(1) + b < 0 \Rightarrow w_2 + b < 0$

Step 3: Select values that satisfy all inequalities

- Since $b < 0$, choose a convenient negative bias: $b = -0.5$.
- From $w_1 + b \geq 0$, we get $w_1 - 0.5 \geq 0$, so $w_1 \geq 0.5$. Choose $w_1 = 1$.
- From $w_1 + w_2 + b < 0$, we get $1 + w_2 - 0.5 < 0$, so $w_2 < -0.5$. Choose $w_2 = -1$.
- Check the final condition: $w_2 + b = -1 - 0.5 = -1.5 < 0$.

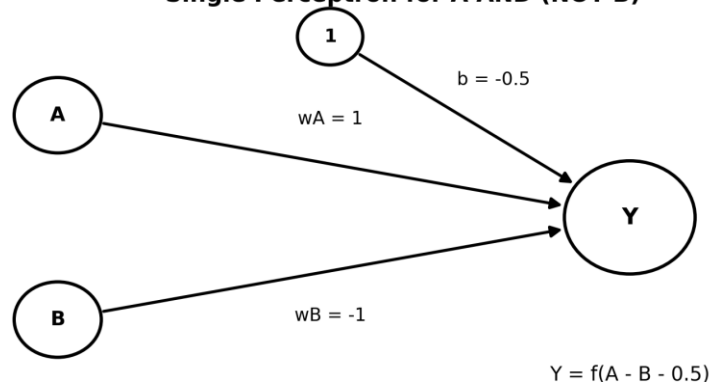
$$w_1 = 1, w_2 = -1, b = -0.5$$

$$Y = f(A - B - 0.5)$$

Step 4: Verify the selected perceptron

A	B	$v = A - B - 0.5$	Output Y	Required output
0	0	-0.5	0	0
0	1	-1.5	0	0
1	0	0.5	1	1
1	1	-0.5	0	0

Single Perceptron for A AND (NOT B)



The computed output matches the desired truth table for all four input combinations. Therefore, the chosen perceptron correctly implements A AND

	(NOT B).
Marks	6
-ve applicable?	N
% Applicable	N

Question No	54															
Question Type																
Difficulty level	Easy															
Chapter	Unit 2															
Topic	Perceptrons															
Question	Design a two-layer network of perceptrons that implements A XOR B. Determine suitable weights and biases, and verify each design using its truth table.															
Long Answer	Step 1: Write the XOR truth table <table><tr><th>A</th><th>B</th><th>A XOR B</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table> Step 2: Show why a single perceptron cannot implement XOR Assume that one perceptron with net input $w_1A + w_2B + b$ could implement XOR. The four truth-table rows would require: • For A = 0, B = 0 and output 0: $b < 0$ • For A = 1, B = 0 and output 1: $w_1 + b \geq 0$ • For A = 0, B = 1 and output 1: $w_2 + b \geq 0$ • For A = 1, B = 1 and output 0: $w_1 + w_2 + b < 0$ From $w_1 + b \geq 0$ and $w_2 + b \geq 0$, we obtain $w_1 \geq -b$ and $w_2 \geq -b$. Therefore, $w_1 + w_2 + b \geq -b - b + b = -b$ Since $b < 0$, we have $-b > 0$. Thus $w_1 + w_2 + b > 0$, which contradicts the condition $w_1 + w_2 + b < 0$ required for A = 1, B = 1. Therefore, XOR is not linearly separable and cannot be implemented by one perceptron. Step 3: Decompose XOR into linearly separable functions A XOR B = [A AND (NOT B)] OR [(NOT A) AND B] Use two perceptrons in the hidden layer. The first recognizes A = 1, B = 0, and the second recognizes A = 0, B = 1.	A	B	A XOR B	0	0	0	0	1	1	1	0	1	1	1	0
A	B	A XOR B														
0	0	0														
0	1	1														
1	0	1														
1	1	0														

Step 4: Design hidden neuron H_1

H_1 implements A AND (NOT B), so the previous result can be reused.

$$H_1 = f(A - B - 0.5)$$

Weights for H_1 : $w_{11} = 1$, $w_{12} = -1$, $b_1 = -0.5$

Step 5: Design hidden neuron H_2

H_2 must produce 1 only for A = 0 and B = 1.

- For A = 0, B = 1 and output 1:

$$w_{22} + b_2 \geq 0$$

- For A = 0, B = 0 and output 0:

$$b_2 < 0$$

- For A = 1, B = 0 and output 0:

$$w_{21} + b_2 < 0$$

- For A = 1, B = 1 and output 0:

$$w_{21} + w_{22} + b_2 < 0$$

Choose $b_2 = -0.5$. Then $w_{22} - 0.5 \geq 0$ requires $w_{22} \geq 0.5$, so choose $w_{22} = 1$. The condition $w_{21} + 1 - 0.5 < 0$ requires $w_{21} < -0.5$, so choose $w_{21} = -1$.

$$H_2 = f(-A + B - 0.5)$$

Weights for H_2 : $w_{21} = -1$, $w_{22} = 1$, $b_2 = -0.5$

Step 6: Design the output perceptron

The output neuron must perform OR on H_1 and H_2 . Let its net input be $v_1H_1 + v_2H_2 + b_o$.

- For $H_1 = 0$, $H_2 = 0$ and output 0:

$$b_o < 0$$

- For $H_1 = 1$, $H_2 = 0$ and output 1:

$$v_1 + b_o \geq 0$$

- For $H_1 = 0$, $H_2 = 1$ and output 1:

$$v_2 + b_o \geq 0$$

- For $H_1 = 1$, $H_2 = 1$ and output 1:

$$v_1 + v_2 + b_o \geq 0$$

Choose $b_o = -0.5$, $v_1 = 1$, and $v_2 = 1$. These values satisfy all four inequalities.

$$Y = f(H_1 + H_2 - 0.5)$$

Step 7: Verify the complete XOR network

A	B	$H_1 = A \text{ AND (NOT B)}$	$H_2 = (\text{NOT A}) \text{ AND B}$	$Y = H_1 \text{ OR } H_2$	Required XOR
0	0	0	0	0	0
0	1	0	1	1	1
1	0	1	0	1	1
1	1	0	0	0	0

	Two-layer perceptron network implementing A XOR B Final Answer A AND (NOT B): $Y = f(A - B - 0.5)$ A XOR B: $H_1 = f(A - B - 0.5)$, $H_2 = f(-A + B - 0.5)$, $Y = f(H_1 + H_2 - 0.5)$ A AND (NOT B) is linearly separable and can be implemented by one perceptron. XOR is not linearly separable, so a hidden layer is required. A perceptron learning algorithm can learn a valid set of separating weights for the first problem, but a single perceptron cannot learn XOR because no separating line exists.
Marks	6
-ve applicable?	N
% Applicable	N

Question No	55
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Multi-layer networks
Question	A neural network with nonlinear activation functions in the hidden layers and a linear activation function in the output layer is suitable for regression. Comment on this statement and explain whether such a network is equivalent to linear regression.
Long Answer	A neural network with nonlinear activation functions in the hidden layers and a linear activation function in the output layer is commonly used for regression problems. Let the hidden-layer representation be: $h = f(W_1x + b_1)$ where $f(\cdot)$ is a nonlinear activation function, W_1 is the hidden-layer weight matrix, b_1 is the bias vector, and x is the input vector. The output layer computes: $\hat{y} = W_2h + b_2$ Because the output activation is linear, the prediction is not restricted to a fixed interval and can take any real value. Hence, the network is suitable for predicting continuous quantities such as house price, temperature, sales, demand, or income. The network is not generally equivalent to ordinary linear regression. Linear

	regression predicts the output directly from the original inputs: $\hat{y} = w^T x + b$ In contrast, the hidden layers of the neural network first transform the inputs using nonlinear activation functions. Therefore, h is a nonlinear function of x. Although the final layer is linear with respect to h, the overall mapping from x to $\hat{y}$ can still be nonlinear. Conclusion: A linear output layer makes the network appropriate for regression, but the overall model usually performs nonlinear regression. It becomes equivalent to ordinary linear regression only when there are no nonlinear hidden layers, or when all layers use linear activation functions.
Marks	6
-ve applicable?	N
% Applicable	N

Question No	56
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Multi-layer networks
Question	Compare and contrast the perceptron, the multilayer perceptron, and the feed-forward neural network
Long Answer	A single-layer perceptron is the simplest feed-forward neural network. It has no hidden layer and generally uses a binary step or sign activation function. It is trained using the perceptron learning rule and can solve only linearly separable problems, such as AND and OR. A multilayer perceptron (MLP) contains one or more hidden layers. Its hidden neurons use differentiable nonlinear activation functions such as sigmoid, tanh, or ReLU, enabling training through backpropagation. These nonlinear hidden layers allow the MLP to learn complex relationships and solve nonlinearly separable problems such as XOR. A feed-forward neural network is a general class of networks in which information flows only from the input layer to the output layer, with no feedback connections. Both the single-layer perceptron and the multilayer perceptron are feed-forward networks. Thus, the single-layer perceptron is limited to linear classification, whereas an MLP is a feed-forward network capable of learning nonlinear patterns.
Marks	6
-ve applicable?	N
% Applicable	N

Question No	57
Question Type	
Difficulty level	Difficult
Chapter	Unit 2
Topic	Back-propagation
Question	A custom activation function is defined as $f(x) = \ln(1 + e^x)$.

	Consider an output neuron j in an artificial neural network trained using back-propagation and Stochastic Gradient Descent (SGD). Let w_{ji} be the weight connecting neuron i in the preceding layer to output neuron j, x_i be the input received from neuron i, net_j be the net input to output neuron j, $o_j = f(net_j)$ be its actual output, and t_j be its target output. Assuming the squared-error loss $E = \frac{1}{2}(t_j - o_j)^2,$ show that the SGD weight-update rule is $\Delta w_{ji} = \eta(t_j - o_j) \sigma(net_j) x_i,$ where $\sigma(z) = \frac{1}{1 + e^{-z}}$ is the sigmoid function. Also derive the corresponding bias-update expression. Show all differentiation steps clearly.
Long Answer	Solution 1. Define the output neuron's net input and output The net input to the output neuron j is $net_j = \sum_i w_{ji} x_i + b_j.$ The neuron uses the custom activation function $o_j = f(net_j) = \ln(1 + e^{net_j}).$ This activation function is commonly called the Softplus function. 2. Differentiate the custom activation function Start with $f(x) = \ln(1 + e^x).$ Using the chain rule, $\frac{df(x)}{dx} = \frac{1}{1 + e^x} \frac{d}{dx}(1 + e^x).$ Since $\frac{d}{dx}(1 + e^x) = e^x,$ we obtain

$$f'(x) = \frac{e^x}{1 + e^x}.$$

Divide the numerator and denominator by e^x :

$$f'(x) = \frac{1}{1 + e^{-x}}.$$

Therefore,

$$\boxed{f'(x) = \sigma(x)}.$$

At output neuron j ,

$$\boxed{\frac{\partial o_j}{\partial net_j} = \sigma(net_j)}.$$

3. Define the squared-error loss

For one training sample, the squared-error loss is

$$E = \frac{1}{2}(t_j - o_j)^2.$$

Because SGD is used, the parameters are updated immediately after processing this individual training sample.

4. Write the SGD weight-update rule

The change in the weight w_{ji} is

$$\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}},$$

and

$$w_{ji}^{new} = w_{ji}^{old} + \Delta w_{ji}.$$

Therefore, we must derive $\partial E / \partial w_{ji}$.

5. Apply the chain rule

The weight w_{ji} affects the error through net_j and o_j . Hence,

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}}.$$

Each derivative is evaluated separately.

5.1 Differentiate the error with respect to the output

$$E = \frac{1}{2}(t_j - o_j)^2.$$

Applying the power rule and chain rule,

$$\frac{\partial E}{\partial o_j} = \frac{1}{2} \cdot 2(t_j - o_j) \cdot \frac{\partial (t_j - o_j)}{\partial o_j}.$$

Since t_j is constant,

$$\frac{\partial(t_j - o_j)}{\partial o_j} = -1.$$

Thus,

$$\boxed{\frac{\partial E}{\partial o_j} = o_j - t_j}.$$

5.2 Differentiate the neuron output with respect to the net input

From the Softplus derivative obtained earlier,

$$\boxed{\frac{\partial o_j}{\partial net_j} = \sigma(net_j)}.$$

5.3 Differentiate the net input with respect to the weight

Since

$$net_j = \sum_i w_{ji} x_i + b_j,$$

for the particular weight w_{ji} ,

$$\boxed{\frac{\partial net_j}{\partial w_{ji}} = x_i}.$$

6. Compute the error gradient

Substituting the three derivatives into the chain-rule expression gives

$$\frac{\partial E}{\partial w_{ji}} = (o_j - t_j) \sigma(net_j) x_i.$$

Therefore,

$$\boxed{\frac{\partial E}{\partial w_{ji}} = (o_j - t_j) \sigma(net_j) x_i}.$$

7. Derive the weight-update expression

Using SGD,

$$\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}}.$$

Substitute the gradient:

$$\Delta w_{ji} = -\eta (o_j - t_j) \sigma(net_j) x_i.$$

Since $-(o_j - t_j) = t_j - o_j$,

$$\boxed{\Delta w_{ji} = \eta (t_j - o_j) \sigma(net_j) x_i}.$$

Hence, the updated weight is

$$w_{ji}^{new} = w_{ji}^{old} + \eta(t_j - o_j)\sigma(net_j)x_i.$$

8. Derive the bias-update expression

The bias update is

$$\Delta b_j = -\eta \frac{\partial E}{\partial b_j}.$$

Using the chain rule,

$$\frac{\partial E}{\partial b_j} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial net_j} \frac{\partial net_j}{\partial b_j}.$$

Because

$$\frac{\partial net_j}{\partial b_j} = 1,$$

we get

$$\frac{\partial E}{\partial b_j} = (o_j - t_j)\sigma(net_j).$$

Therefore,

$$\Delta b_j = \eta(t_j - o_j)\sigma(net_j).$$

The updated bias is

$$b_j^{new} = b_j^{old} + \eta(t_j - o_j)\sigma(net_j).$$

9. Delta-form representation

Define the output-layer error signal using the negative-gradient convention:

$$\delta_j = -\frac{\partial E}{\partial net_j} = (t_j - o_j)\sigma(net_j).$$

Then the parameter updates become

$$\Delta w_{ji} = \eta \delta_j x_i,$$

$$\Delta b_j = \eta \delta_j.$$

Final result

For the Softplus output activation $f(x) = \ln(1 + e^x)$, its derivative is the sigmoid function:

$$f'(x) = \sigma(x).$$

	Consequently, for one SGD training sample, $\Delta w_{ji} = \eta(t_j - o_j)\sigma(net_j)x_i$ and $\Delta b_j = \eta(t_j - o_j)\sigma(net_j).$
Marks	6
-ve applicable?	N
% Applicable	N

Question No	58
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Activation Functions
Question	Explain the vanishing gradient problem and describe how the choice of activation function affects it.
Long Answer	During backpropagation, the gradient reaching an early layer is a product of the weight matrices and the activation-function derivatives of all the layers above it. If those derivatives are consistently smaller than 1, the product shrinks exponentially with depth, and the weights of the early layers receive almost no update. This is the vanishing gradient problem: the deeper layers continue to train while the earlier layers remain effectively frozen, so the network fails to learn low-level features. Effect of the activation function: • Sigmoid: output range (0, 1) and maximum derivative 0.25. Across n layers the activation-derivative part of the product is bounded above by 0.25^n, so it can only shrink the gradient, and the problem is severe. Sigmoid also saturates for large positive or negative inputs, where its derivative approaches 0. • Tanh: output range (-1, +1) and maximum derivative 1. Being zero-centered it behaves better than Sigmoid, but it still saturates at both extremes. • ReLU: the derivative is 1 for a positive pre-activation, so the ReLU operation itself does not attenuate the local backpropagated gradient at an active neuron. This helps reduce the vanishing-gradient problem compared with saturating activations, but the complete network gradient can still shrink or grow because of weights and other layers. Its own weakness is the dying ReLU problem, where a neuron that remains in the negative region has derivative 0 and may stop learning. • Step: the derivative is 0 everywhere it is defined, so no gradient reaches any layer and gradient-based training is impossible.
Marks	6
-ve applicable?	N
% Applicable	N

Question No	59
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Optimizers
Question	How does momentum help accelerate optimization?
Long Answer	Momentum is an optimization technique that improves the performance of Gradient Descent by considering the previous weight updates while calculating the current update. Instead of moving only in the direction of the current gradient, momentum accumulates past gradients to accelerate convergence towards the minimum of the loss function. Working principle: • Momentum maintains a velocity that combines the previous update with the current gradient. • This accumulated velocity helps the optimizer move faster in the consistent direction, while components that reverse sign at each step cancel out, reducing unnecessary oscillation. Update equations: $v_t = \beta v_{t-1} + \eta \nabla J(w)$ $w_{t+1} = w_t - v_t$ Momentum accelerates optimization as follows: 1. It speeds up convergence by accumulating gradients from previous iterations. 2. It reduces oscillation in steep or narrow regions of the loss surface. 3. It can carry motion through shallow or flat regions and reduce stalling on plateaus; however, Momentum does not guarantee escape from a true local minimum. 4. It provides smoother parameter updates, resulting in faster and more stable training.
Marks	6
-ve applicable?	N
% Applicable	N

Question No	60																
Question Type																	
Difficulty level	Easy																
Chapter	2																
Topic	Optimizers																
Question	Explain Mini-batch Gradient Descent and compare it with Batch Gradient Descent and Stochastic Gradient Descent.																
Long Answer	Mini-batch Gradient Descent splits the training set into small groups of m samples, called mini-batches, and performs one parameter update per mini-batch: $w \leftarrow w - \eta \cdot (1/m) \sum_{i=1}^m \nabla L_i(w)$ For N training samples there are $\lceil N/m \rceil$ updates per epoch (exactly N/m when N is divisible by m). Typical values of m are 32, 64, 128 or 256. <table><tr><th>Property</th><th>Batch GD</th><th>Mini-batch GD</th><th>SGD</th></tr><tr><td>Samples per update</td><td>All N</td><td>m (e.g. 32–256)</td><td>1</td></tr><tr><td>Updates per epoch</td><td>1</td><td>$\lceil N/m \rceil$</td><td>N</td></tr><tr><td>Gradient quality</td><td>Exact</td><td>Good estimate</td><td>Very noisy</td></tr></table>	Property	Batch GD	Mini-batch GD	SGD	Samples per update	All N	m (e.g. 32–256)	1	Updates per epoch	1	$\lceil N/m \rceil$	N	Gradient quality	Exact	Good estimate	Very noisy
Property	Batch GD	Mini-batch GD	SGD														
Samples per update	All N	m (e.g. 32–256)	1														
Updates per epoch	1	$\lceil N/m \rceil$	N														
Gradient quality	Exact	Good estimate	Very noisy														

	Path to the minimum	Smooth	Slightly noisy	Highly noisy
	Memory required	Potentially high	One mini-batch	Low
	Hardware use	Good vectorization	Very good; efficient on accelerators	Limited vectorization
	Why mini-batch is the practical choice: - The averaging over m samples removes most of the noise of SGD while keeping the updates frequent. - A mini-batch fits in memory and maps efficiently onto GPU/TPU parallelism, providing a useful balance between vectorization and frequent parameter updates. - The remaining stochasticity can help exploration in non-convex loss surfaces, for example by helping the optimizer move through saddle or flat regions; it does not guarantee escape from local minima. For these reasons, mini-batch training is the standard practical choice in deep learning. In practice, the term SGD is also often used for mini-batch stochastic gradient training.			
Marks	6			
-ve applicable?	N			
% Applicable	N			

Question No	61																							
Question Type																								
Difficulty level	Medium																							
Chapter	2																							
Topic	Support Vector Machines																							
Question	Consider the following linearly separable training dataset: <table><thead><tr><th>Point</th><th>x_1</th><th>x_2</th><th>Class (y)</th></tr></thead><tbody><tr><td>P1</td><td>1</td><td>1</td><td>-1</td></tr><tr><td>P2</td><td>2</td><td>2</td><td>-1</td></tr><tr><td>P3</td><td>4</td><td>4</td><td>+1</td></tr><tr><td>P4</td><td>5</td><td>5</td><td>+1</td></tr></tbody></table> Determine the maximum-margin hyperplane, identify the support vector(s) and justify your answer, and compute the margin. (2 + 2 + 2 Marks)				Point	x_1	x_2	Class (y)	P1	1	1	-1	P2	2	2	-1	P3	4	4	+1	P4	5	5	+1
Point	x_1	x_2	Class (y)																					
P1	1	1	-1																					
P2	2	2	-1																					
P3	4	4	+1																					
P4	5	5	+1																					
Long Answer	Step 1: Identify the closest opposite-class pair. All four points lie on the line $x_1 = x_2$. The closest pair drawn from opposite classes is P2(2, 2) and P3(4, 4). The maximum-margin hyperplane is the perpendicular bisector of the segment joining them, so w is parallel to $P3 - P2 = (2, 2)$, that is $w = (a, a)$. Step 2: Apply the canonical conditions $y f(x) = 1$ at the support vectors. For P3 ($y = +1$): $4a + 4a + b = +1 \rightarrow 8a + b = 1$ For P2 ($y = -1$): $2a + 2a + b = -1 \rightarrow 4a + b = -1$ Subtracting: $4a = 2$, so $a = 0.5$ and $b = -1 - 4(0.5) = -3$																							

	Hyperplane: $0.5x_1 + 0.5x_2 - 3 = 0$, that is $x_1 + x_2 = 6$ Step 3: Verify the remaining points. P1(1, 1): $f = 0.5 + 0.5 - 3 = -2$, $y f = 2 > 1 \rightarrow$ not a support vector P4(5, 5): $f = 2.5 + 2.5 - 3 = 2$, $y f = 2 > 1 \rightarrow$ not a support vector Support vectors: P2(2, 2) and P3(4, 4), since only these satisfy $y f(x) = 1$. P1 and P4 lie strictly outside the margin. Step 4: Compute the margin. $\ w\ = \sqrt{(0.5)^2 + (0.5)^2} = \sqrt{0.5} = 0.707$ Margin = $2/\ w\ = 2/0.707 = 2.828$ units This agrees with the direct distance between the two support vectors, $\sqrt{(2^2 + 2^2)} = 2.828$.
Marks	6
-ve applicable?	N
% Applicable	N

Question No	62
Question Type	
Difficulty level	Medium
Chapter	2
Topic	Optimizers
Question	Explain the role of the learning rate in optimization. Compare how Gradient Descent (GD) and Stochastic Gradient Descent (SGD) are affected by the learning rate during the training of a machine learning model. (6 Marks)
Long Answer	The learning rate (η) is a hyperparameter that controls the size of the step taken during each iteration while updating the model parameters to minimize the loss function. Weight update rule (generic form): $w_{\text{new}} = w_{\text{old}} - \eta \cdot (\partial L / \partial w)$ Where η is the learning rate and $\partial L / \partial w$ is the gradient of the loss with respect to the weight. Effect of the learning rate: • Too small η: parameter changes become very small, convergence is slow, and the required training time can become impractically large. • Too large η: overshoots the minimum, may lead to divergence, causes oscillation in the loss function. Hence choosing the right learning rate is essential for effective and efficient training. Handling of the learning rate by the two optimizers: a) Gradient Descent (GD): • Gradient Descent can use either a constant learning rate or a learning-rate schedule. At a given iteration, the same η scales the full-batch gradient unless a parameter-wise adaptive method is used. • Because the full-batch gradient is deterministic for fixed parameters and data, its updates are stable; however, an excessively large η can still overshoot the minimum or cause divergence. b) Stochastic Gradient Descent (SGD): • SGD can also use either a constant learning rate or a learning-rate schedule. It updates the parameters after each sampled training example, which introduces stochasticity into the optimization path. • Because the stochastic gradient is noisy, a large η can make the path unstable. It is common to reduce η during training, but SGD does not inherently require a smaller learning rate than full-batch GD; the suitable value is problem-dependent.
Marks	6

-ve applicable?	N
% Applicable	N

Question No	63
Question Type	
Difficulty level	Difficult
Chapter	2
Topic	Support Vector Machines
Question	Explain the geometric intuition behind Support Vector Machines (SVM). How does SVM determine the optimal decision boundary, and what is the role of the margin and support vectors in classification?
Long Answer	The geometric intuition behind a Support Vector Machine is to find the optimal separating hyperplane that divides data points belonging to different classes while maximizing the distance (margin) between the classes. - Optimal Decision Boundary (2 Marks) - SVM identifies a hyperplane $w \cdot x + b = 0$ that separates the data into different classes. - Among all possible separating hyperplanes, SVM chooses the one with the maximum margin. - A larger margin generally improves robustness to small perturbations and can improve generalization to unseen data; it does not guarantee fewer classification errors on every dataset. - Margin (2 Marks) - The margin is the distance between the two supporting hyperplanes, each of which passes through the nearest data points of one class. - Maximizing the margin makes the classifier more robust to noise and less likely to overfit. - The margin is given by $\text{Margin} = 2/\ w\$, where w is the weight vector of the separating hyperplane, and the distance from the boundary to the nearest point of either class is $1/\ w\$. - Support Vectors (2 Marks) - The support vectors are the training samples that lie closest to the decision boundary, satisfying $y_i(w \cdot x_i + b) = 1$ in canonical form. - These points determine the position and orientation of the optimal hyperplane. - If the support vectors change, the decision boundary also changes. - In the standard hard-margin setting, data points that lie strictly outside the margin have inactive constraints and do not directly determine the maximum-margin boundary.
Marks	6
-ve applicable?	N
% Applicable	N

Question No	64
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain the concept of margin in Support Vector Machines. Describe Hard Margin and Soft Margin SVM in this context.
Long Answer	1. Margin - Definition: The distance between the decision boundary and the nearest data points from each class. - Objective: Support Vector Machines (SVM) aim to find the hyperplane that maximizes this margin, as a wider margin generally leads to better generalization on unseen data. - Boundaries: - The margin boundaries (gutters) are defined by $w^T x + b = +1$ and $w^T x + b = -1$. - The decision boundary is located at $w^T x + b = 0$. - Margin Width: $\text{Margin Width} = \frac{2}{\ w\ }$ 2. Hard Margin SVM - Applicability: Used when the data is perfectly linearly separable. - Behavior: Does not allow any training point to fall inside the margin or be misclassified. - Constraint: $y_i(w^T x_i + b) \geq 1 \quad \forall i$ 3. Soft Margin SVM - Applicability: Used when the data is not perfectly linearly separable (contains noise or overlapping classes). - Behavior: Introduces non-negative slack variables ($\xi_i \geq 0$) to allow controlled margin violations and misclassifications. - Constraint:

	$y_i(w^T x_i + b) \geq 1 - \xi_i \quad \text{where } \xi_i \geq 0$ 4. Optimization Objective: Minimizes the trade-off between maximizing the margin and penalizing errors: $\min_{w,b,\xi} \left(\frac{1}{2} \ w\ ^2 + C \sum_i \xi_i \right)$
Marks	6
-ve applicable?	N
% Applicable	N

Question No	65
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	What is the Lagrangian dual problem in SVM? Explain why it is used and state its general form.
Long Answer	Primal Problem Formulation A. Support Vector Machine (SVM) optimization begins as a constrained minimization problem: B. $\min_{w,b} \frac{1}{2} \ w\ ^2$ subject to $y_i(w^T x_i + b) \geq 1 \quad \forall i$ C. To solve this constrained problem, the Lagrange Multiplier method is applied, introducing a non-negative multiplier $\alpha_i \geq 0$ for each training instance's constraint. D. This constructs the Lagrangian function: E. $\mathcal{L}(w, b, \alpha) = \frac{1}{2} \ w\ ^2 - \sum_i \alpha_i [y_i(w^T x_i + b) - 1]$ F. Solving for the stationarity conditions ($\frac{\partial \mathcal{L}}{\partial w} = 0$ and $\frac{\partial \mathcal{L}}{\partial b} = 0$) and substituting the resulting relationships back into $\mathcal{L}$ converts the problem into its dual form, expressed purely in terms of the Lagrange multipliers α_i. General Form of the Dual Problem G. The dual optimization problem is formulated as: H. $\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i^T x_j)$ I. subject to $\alpha_i \geq 0 \quad \forall i$, and $\sum_i \alpha_i y_i = 0$ Why the Dual Problem is Preferred J. Kernel Trick Compatibility: The formulation depends on the training data strictly through inner products $(x_i^T x_j)$. Replacing this inner product with a kernel function $K(x_i, x_j)$ enables non-linear classification in high-dimensional spaces without explicitly computing feature transformations. K. Computational Efficiency: It is often significantly more efficient to solve when the number of features d is much larger than the number of data samples N, as the optimization parameter space depends on N rather than d.

	L. Sparsity & Support Vectors: The solution directly identifies support vectors via the Karush-Kuhn-Tucker (KKT) conditions. Only data points lying on or violating the margin have $\alpha_i > 0$, meaning only these critical points define the final decision boundary.
Marks	6
-ve applicable?	N
% Applicable	N

Question No	66
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	What are Kernel Functions in SVM? Explain the Linear and Polynomial kernels with their equations.
Long Answer	A. A kernel function computes the dot product between two data points as if they had been mapped into a higher-dimensional feature space, without explicitly performing that transformation. B. This is known as the kernel trick, and it allows SVM to handle data that is not linearly separable in its original space. C. General definition: $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ Linear Kernel: D. Corresponds to no transformation of the input space, i.e., $\phi(x) = x$. E. Equation: $K(x_i, x_j) = x_i^T x_j$ F. Used when the data is already linearly (or approximately linearly) separable. Polynomial Kernel: - Maps data into a higher-dimensional space using polynomial combinations of the original features. - Equation: $K(x_i, x_j) = (x_i^T x_j + c)^d$, where d is the degree of the polynomial and c is a constant. - Captures non-linear relationships between classes; however, a very high degree d can lead to overfitting.
Marks	6
-ve applicable?	N
% Applicable	N

Question No	67
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	What is a Linear Kernel? Derive its expression and explain when it is preferred over other kernels.

Long Answer	A kernel function $K(x_i, x_j)$ computes the inner product of two data points as if they had been mapped into a (possibly higher-dimensional) feature space $\phi(\cdot)$, without explicitly performing that mapping: $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ Derivation for the Linear Kernel: The linear kernel corresponds to the trivial mapping $\phi(x) = x$, i.e., no transformation of the input space. Substituting into the general kernel definition: $K(x_i, x_j) = \phi(x_i)^T \phi(x_j) = x_i^T x_j$ So the linear kernel is simply the ordinary dot product between the two input vectors: $K(x_i, x_j) = x_i^T x_j$ Use in the dual objective: Substituting this into the dual formulation replaces every occurrence of $x_i^T x_j$ with $K(x_i, x_j)$ When it is preferred: 1. When the data is already (approximately) linearly separable in its original feature space. 2. When the number of features is very large relative to the number of samples (e.g., text classification), where a non-linear mapping adds little benefit but more computational cost. 3. It has no extra hyperparameters (unlike the polynomial kernel's degree or the RBF kernel's γ), making it simpler and less prone to overfitting.
Marks	6
-ve applicable?	N
% Applicable	N

Question No	68
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain the Soft Margin SVM formulation. Discuss the effect of increasing and decreasing the regularization parameter C , with a brief example.
Long Answer	Soft Margin SVM Formulation Soft Margin SVM handles data that is not perfectly linearly separable by introducing non-negative slack variables $\xi_i \geq 0$, which relax the strict margin constraints: 1. Primal Optimization Objective: $\min_{w, b, \xi} \left(\frac{1}{2} \ w\ ^2 + C \sum_i \xi_i \right)$ 2. Subject to Constraints: $y_i(w^T x_i + b) \geq 1 - \xi_i \quad \text{and} \quad \xi_i \geq 0 \quad \forall i$

	Role and Effect of the Regularization Parameter (C) The hyperparameter $C > 0$ controls the trade-off between maximizing the margin width and minimizing classification errors/slack violations. 1. Large C Value: 1. Behavior: Imposes a heavy penalty on slack variables, tolerating fewer margin violations. 2. Result: Produces a narrower margin and behaves similarly to a Hard Margin SVM. 3. Trade-off: High risk of overfitting on noisy data or outliers. 2. Small C Value: 1. Behavior: Imposes a lighter penalty on slack variables, tolerating more margin violations. 2. Result: Produces a wider margin with better generalization on unseen data. 3. Trade-off: High risk of underfitting if set too small. Practical Example Consider a medical diagnostic dataset containing a few mislabeled patient records: 1. High C Scenario: Forces the decision boundary to warp around those mislabeled points to avoid penalizing them. This creates a narrow, noise-sensitive margin that performs poorly on new test data. 2. Low C Scenario: Treats the mislabeled records as allowable margin violations, resulting in a cleaner, wider margin that generalizes far better to future patients.
Marks	6
-ve applicable?	N
% Applicable	N

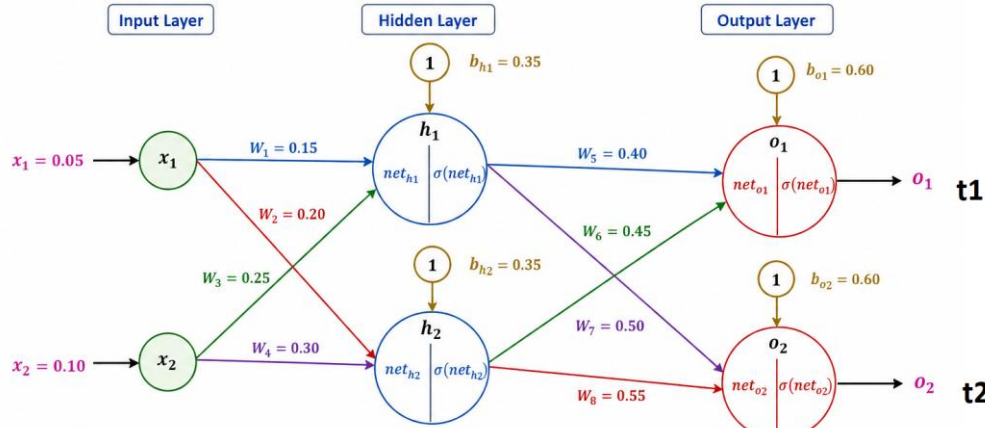
Question No	69
Question Type	
Difficulty level	Difficult
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Derive the Lagrangian dual of the Hard-Margin SVM starting from its primal problem, and explain how the KKT conditions identify the support vectors.
Long Answer	Step 1 — Form the Lagrangian. Introduce a multiplier $\alpha_i \geq 0$ for each inequality constraint: $L(w, b, \alpha) = \frac{1}{2} \ w\ ^2 - \sum_i \alpha_i [y_i(w^T x_i + b) - 1]$ Step 2 — Stationarity conditions. Set the partial derivatives of L with respect to the primal variables to zero: 1. $\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_i \alpha_i y_i x_i$ 2. $\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_i \alpha_i y_i = 0$ Step 3 — Substitute back into L. Replacing w and using $\sum \alpha_i y_i = 0$ eliminates w and b entirely, leaving an expression purely in terms of α: $L(\alpha) = \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i^T x_j)$

	Dual problem: $\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i^T x_j)$ subject to $\alpha_i \geq 0, \sum_i \alpha_i y_i = 0$ Identifying support vectors — Complementary Slackness (KKT): $\alpha_i [y_i (w^T x_i + b) - 1] = 0$ 1. If a point lies strictly outside the margin, $y_i (w^T x_i + b) - 1 > 0$, forcing $\alpha_i = 0$ — it does not influence w. 2. If $\alpha_i > 0$, then $y_i (w^T x_i + b) = 1$, i.e. the point lies exactly on a margin boundary — this is a support vector. 3. Since $w = \sum \alpha_i y_i x_i$, only support vectors ($\alpha_i > 0$) contribute to the final hyperplane; all other training points can be discarded without changing the solution.
Marks	6
-ve applicable?	N
% Applicable	N

Question No	70
Question Type	
Difficulty level	Difficult
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Derive the Lagrangian dual of the Soft-Margin SVM and explain why the resulting Lagrange multipliers satisfy the box constraint $0 \leq \alpha_i \leq C$.
Long Answer	Primal problem (Soft Margin): $\min \frac{1}{2} \ w\ ^2 + C \sum_i \xi_i$ subject to $y_i (w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$ Lagrangian: introduce $\alpha_i \geq 0$ for the margin constraints and $\mu_i \geq 0$ for $\xi_i \geq 0$: $L = \frac{1}{2} \ w\ ^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i (w^T x_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i$ Stationarity conditions: 1. $\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_i \alpha_i y_i x_i$ 2. $\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_i \alpha_i y_i = 0$ 3. $\frac{\partial L}{\partial \xi_i} = 0 \Rightarrow C - \alpha_i - \mu_i = 0 \Rightarrow \mu_i = C - \alpha_i$ Because $\mu_i \geq 0$, the last relation forces $\alpha_i \leq C$. Combined with $\alpha_i \geq 0$, this gives the box constraint: $0 \leq \alpha_i \leq C$ Resulting dual (identical objective to Hard Margin, different feasible region): $\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i^T x_j)$ subject to $0 \leq \alpha_i \leq C, \sum_i \alpha_i y_i = 0$ Interpretation of the box constraint: 1. $\alpha_i = 0 \rightarrow$ point correctly classified, outside the margin ($\xi_i = 0$), not a support vector.

	2. $0 < \alpha_i < C \rightarrow$ point lies exactly on the margin boundary ($\xi_i = 0$) — an "unbound" support vector.
	3. $\alpha_i = C \rightarrow$ point violates the margin or is misclassified ($\xi_i > 0$) — a "bound" support vector.
Marks	6
-ve applicable?	N
% Applicable	N

Unit 2 – 8 Marks Descriptive Question Bank

Question No	71
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Back-propagation
Question	 Consider the 2–2–2 multilayer perceptron shown in the figure. The input values are $x_1 = 0.05$ and $x_2 = 0.10$. The hidden-layer weights are $W_1 = 0.15$, $W_2 = 0.20$, $W_3 = 0.25$, and $W_4 = 0.30$. The output-layer weights are $W_5 = 0.40$, $W_6 = 0.45$, $W_7 = 0.50$, and $W_8 = 0.55$. The hidden-neuron biases are $b_{h1} = b_{h2} = 0.35$, and the output-neuron biases are $b_{o1} = b_{o2} = 0.60$. All hidden and output neurons use the sigmoid activation function. The target outputs are $t_1 = 0.01$ and $t_2 = 0.99$. Use the squared-error loss $E = \frac{1}{2}[(t_1 - o_1)^2 + (t_2 - o_2)^2]$ Assume a learning rate $\eta = 0.50$. Perform the following: - Compute the missing net inputs and activations of h_1, h_2, o_1, and o_2. - Compute the individual output errors E_1 and E_2 and the total error E. - Update W_5, W_6, and W_7 once using backpropagation. Do not update the biases.
Long Answer	Detailed Step-by-Step Solution Step 1: Sigmoid activation $\sigma(z) = 1 / (1 + e^{-z})$

Step 2: Forward propagation to hidden neuron h_1

$$\text{net}_{h1} = x_1W_1 + x_2W_2 + b_{h1}$$

$$\text{net}_{h1} = (0.05)(0.15) + (0.10)(0.20) + 0.35$$

$$\text{net}_{h1} = 0.3775$$

$$h_1 = \sigma(\text{net}_{h1}) = \sigma(0.3775) = 0.593269992$$

Step 3: Forward propagation to hidden neuron h_2

$$\text{net}_{h2} = x_1W_3 + x_2W_4 + b_{h2}$$

$$\text{net}_{h2} = (0.05)(0.25) + (0.10)(0.30) + 0.35$$

$$\text{net}_{h2} = 0.3925$$

$$h_2 = \sigma(\text{net}_{h2}) = \sigma(0.3925) = 0.596884378$$

Step 4: Forward propagation to output neuron o_1

$$\text{net}_{o1} = h_1W_5 + h_2W_6 + b_{o1}$$

$$\text{net}_{o1} = (0.593269992)(0.40) + (0.596884378)(0.45) + 0.60$$

$$\text{net}_{o1} = 1.105905967$$

$$o_1 = \sigma(\text{net}_{o1}) = 0.751365070$$

Step 5: Forward propagation to output neuron o_2

$$\text{net}_{o2} = h_1W_7 + h_2W_8 + b_{o2}$$

$$\text{net}_{o2} = (0.593269992)(0.50) + (0.596884378)(0.55) + 0.60$$

$$\text{net}_{o2} = 1.224921404$$

$$o_2 = \sigma(\text{net}_{o2}) = 0.772928465$$

Step 6: Compute the error at the output layer

$$E_1 = \frac{1}{2}(t_1 - o_1)^2$$

$$E_1 = \frac{1}{2}(0.01 - 0.751365070)^2 = 0.274811083$$

$$E_2 = \frac{1}{2}(t_2 - o_2)^2$$

$$E_2 = \frac{1}{2}(0.99 - 0.772928465)^2 = 0.023560026$$

$$E = E_1 + E_2 = 0.274811083 + 0.023560026 = 0.298371109$$

Step 7: Compute the output-layer error signals

Using the negative-gradient convention,

$$\delta_{ok} = -\partial E / \partial \text{net}_{ok} = (t_k - o_k)o_k(1 - o_k)$$

$$\delta_{o1} = (0.01 - 0.751365070)(0.751365070)(1 - 0.751365070)$$

$$\delta_{o1} = -0.138498562$$

$$\delta_{o2} = (0.99 - 0.772928465)(0.772928465)(1 - 0.772928465)$$

$$\delta_{o2} = 0.038098237$$

Step 8: Update W_5

$$W_5(\text{new}) = W_5(\text{old}) + \eta \delta_{o1} h_1$$

	$W_5(\text{new}) = 0.40 + (0.50)(-0.138498562)(0.593269992)$		
	$W_5(\text{new}) = 0.358916480$		
	Step 9: Update W_6		
	$W_6(\text{new}) = W_6(\text{old}) + \eta \delta_{o1} h_2$		
	$W_6(\text{new}) = 0.45 + (0.50)(-0.138498562)(0.596884378)$		
	$W_6(\text{new}) = 0.408666186$		
	Step 10: Update W_7		
	$W_7(\text{new}) = W_7(\text{old}) + \eta \delta_{o2} h_1$		
	$W_7(\text{new}) = 0.50 + (0.50)(0.038098237)(0.593269992)$		
	$W_7(\text{new}) = 0.511301270$		
	Final Answers		
	Quantity	Expression	Value
	h_1	$\sigma(\text{net}_{h1})$	0.593269992
	h_2	$\sigma(\text{net}_{h2})$	0.596884378
	o_1	$\sigma(\text{net}_{o1})$	0.751365070
	o_2	$\sigma(\text{net}_{o2})$	0.772928465
	E_1	$\frac{1}{2}(t_1 - o_1)^2$	0.274811083
	E_2	$\frac{1}{2}(t_2 - o_2)^2$	0.023560026
	Total E	$E_1 + E_2$	0.298371109
	δ_{o1}	$(t_1 - o_1)o_1(1 - o_1)$	-0.138498562
	δ_{o2}	$(t_2 - o_2)o_2(1 - o_2)$	0.038098237
	$W_5(\text{new})$	$W_5 + \eta \delta_{o1} h_1$	0.358916480
	$W_6(\text{new})$	$W_6 + \eta \delta_{o1} h_2$	0.408666186
	$W_7(\text{new})$	$W_7 + \eta \delta_{o2} h_1$	0.511301270
Marks	8		
-ve applicable?	N		
% Applicable	N		

Question No	72
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Back-propagation
Question	Input Layer Hidden Layer Output Layer For the network in Figure 1, $x_1 = 0.888$, $x_2 = -0.490$, $w_1 = 1.76$, $w_2 = 0.40$, $w_3 = 0.97$, $w_4 = 2.24$, $w_5 = 1.86$, $w_6 = -0.97$, and all biases are zero. Hidden and output neurons use sigmoid activation. For target $t = 1$ and learning rate $\eta = 0.1$, use one SGD/back-propagation step with $E = \frac{1}{2}(t - o)^2$ to compute the updated w_1 and w_5. Round to four decimal places.
Long Answer	Detailed Solution Step 1: Sigmoid function $\sigma(a) = 1/(1 + e^{-a}), \quad \sigma'(a) = \sigma(a)[1 - \sigma(a)]$ Step 2: Hidden neuron h_1 $a_{h1} = (1.76)(0.888) + (0.40)(-0.490) = 1.366880$ $h_1 = \sigma(1.366880) = 0.796876$ Step 3: Hidden neuron h_2 $a_{h2} = (0.97)(0.888) + (2.24)(-0.490) = -0.236240$ $h_2 = \sigma(-0.236240) = 0.441213$ Step 4: Output neuron $a_o = (1.86)(0.796876) + (-0.97)(0.441213) = 1.054212$

	$o = \sigma(1.054212) = 0.741583$ Step 5: Error and output delta $E = \frac{1}{2}(1 - 0.741583)^2 = 0.033390$ $\delta_o = (t - o)o(1 - o)$ $\delta_o = (1 - 0.741583)(0.741583)(1 - 0.741583) = 0.049522$ Step 6: Update w_5 $\Delta w_5 = \eta \delta_o h_1 = (0.1)(0.049522)(0.796876) = 0.003946$ $w_5(\text{new}) = 1.86 + 0.003946 = 1.8639$ Step 7: Hidden delta for h_1 Use the old output weight $w_5 = 1.86$ when computing the hidden delta. $\delta_{h1} = h_1(1 - h_1)w_5\delta_o$ $\delta_{h1} = (0.796876)(1 - 0.796876)(1.86)(0.049522) = 0.014910$ Step 8: Update w_1 $\Delta w_1 = \eta \delta_{h1} x_1 = (0.1)(0.014910)(0.888) = 0.001324$ $w_1(\text{new}) = 1.76 + 0.001324 = 1.7613$
Marks	8
-ve applicable ?	N
% Applicable	N

Question No	73
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Back-propagation
Question	Input Layer Hidden Layer Output Layer <pre> graph LR x1((x1 0.35)) -- w13=0.10 --> H3((H3)) x1 -- w14=0.40 --> H4((H4)) x2((x2 0.90)) -- w23=0.80 --> H3 x2 -- w24=0.60 --> H4 H3 -- w35=0.30 --> O5((O5)) H4 -- w45=0.90 --> O5 </pre> Assume that all neurons use the sigmoid activation function. Perform a forward pass and a backward pass for the network shown below. The target output is $y = 0.5$ and the learning rate is $\eta = 1$. After updating all weights once, perform another forward pass.

Long Answer

Solution: First Forward Pass

$$\sigma(a) = 1 / (1 + e^{-a})$$

Hidden neuron H₃

$$a_3 = w_{13}x_1 + w_{23}x_2$$

$$a_3 = (0.10)(0.35) + (0.80)(0.90) = 0.7550$$

$$y_3 = \sigma(a_3) = \sigma(0.7550) = 0.6803$$

Hidden neuron H₄

$$a_4 = w_{14}x_1 + w_{24}x_2$$

$$a_4 = (0.40)(0.35) + (0.60)(0.90) = 0.6800$$

$$y_4 = \sigma(a_4) = \sigma(0.6800) = 0.6637$$

Output neuron O₅

$$a_5 = w_{35}y_3 + w_{45}y_4$$

$$a_5 = (0.30)(0.6803) + (0.90)(0.6637) = 0.8014$$

$$y_5 = \sigma(a_5) = \sigma(0.8014) = 0.6903$$

$$\text{Prediction error} = \text{target} - \text{output} = 0.50 - 0.6903 = -0.1903$$

Solution: Backward Pass

Using the negative-gradient delta convention:

$$\delta_j = y_j(1 - y_j)(t_j - y_j), \text{ for an output unit}$$

$$\delta_j = y_j(1 - y_j) \sum_k w_{jk} \delta_k, \text{ for a hidden unit}$$

Output delta

$$\delta_5 = y_5(1 - y_5)(t - y_5)$$

$$\delta_5 = (0.6903)(1 - 0.6903)(0.50 - 0.6903) = -0.040681$$

Hidden deltas

$$\delta_3 = y_3(1 - y_3)w_{35}\delta_5$$

$$\delta_3 = (0.6803)(1 - 0.6803)(0.30)(-0.040681) = -0.002654$$

$$\delta_4 = y_4(1 - y_4)w_{45}\delta_5$$

$$\delta_4 = (0.6637)(1 - 0.6637)(0.90)(-0.040681) = -0.008172$$

Weight Updates

$$\Delta w_{ij} = \eta \delta_j x_i, \quad w_{ij}(\text{new}) = w_{ij}(\text{old}) + \Delta w_{ij}$$

i	j	Old w _{ij}	δ _j	Input x _i	η	Δw _{ij}	Updated w _{ij}
1	3	0.1000	-0.002654	0.3500	1.0	-0.000929	0.0991
2	3	0.8000	-0.002654	0.9000	1.0	-0.002389	0.7976
1	4	0.4000	-0.008172	0.3500	1.0	-0.002860	0.3971
2	4	0.6000	-0.008172	0.9000	1.0	-0.007354	0.5926
3	5	0.3000	-0.040681	0.6803	1.0	-0.027674	0.2723
4	5	0.9000	-0.040681	0.6637	1.0	-0.027002	0.8730

Second Forward Pass

$$a_3(\text{new}) = (0.0991)(0.35) + (0.7976)(0.90) = 0.7525$$

$$y_3(\text{new}) = \sigma(0.7525) = 0.6797$$

$$a_4(\text{new}) = (0.3971)(0.35) + (0.5926)(0.90) = 0.6724$$

$$y_4(\text{new}) = \sigma(0.6724) = 0.6620$$

$$a_5(\text{new}) = (0.2723)(0.6797) + (0.8730)(0.6620) = 0.7631$$

$$y_5(\text{new}) = \sigma(0.7631) = 0.6820$$

	New error = $0.50 - 0.6820 = -0.1820$ Conclusion: The network output changes from 0.6903 to 0.6820, moving closer to the target 0.50.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	74
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Back-propagation
Question	For an artificial neural network trained using back-propagation, derive the weight-update and bias-update expressions for an output neuron that uses the sigmoid activation function. Assume that the network is optimized using Stochastic Gradient Descent (SGD) and that the squared-error loss function is used. Show all steps of the derivation clearly.
Long Answer	Consider output neuron k. Let: • h_j: output of neuron j in the preceding layer • w_{kj}: weight connecting neuron j to output neuron k • b_k: bias of output neuron k • a_k: net input to output neuron k • y_k: predicted output • t_k: target output • η: learning rate 1. Net input to the output neuron $a_k = \sum_j w_{kj} h_j + b_k$ 2. Sigmoid output $y_k = \sigma(a_k) = 1 / (1 + e^{(-a_k)})$ 3. Squared-error loss $E_k = \frac{1}{2}(t_k - y_k)^2$ Using gradient descent, the weight-update rule is: $w_{kj}(\text{new}) = w_{kj}(\text{old}) - \eta(\partial E_k / \partial w_{kj})$ 4. Apply the chain rule $\partial E_k / \partial w_{kj} = (\partial E_k / \partial y_k)(\partial y_k / \partial a_k)(\partial a_k / \partial w_{kj})$ Error derivative: $\partial E_k / \partial y_k = y_k - t_k$ Sigmoid derivative: $\partial y_k / \partial a_k = \sigma(a_k)[1 - \sigma(a_k)] = y_k(1 - y_k)$ Net-input derivative: $\partial a_k / \partial w_{kj} = h_j$ 5. Error gradient

	$\partial E_k / \partial w_{kj} = (y_k - t_k) y_k (1 - y_k) h_j$ 6. Final weight-update expression $w_{kj}(\text{new}) = w_{kj}(\text{old}) - \eta (y_k - t_k) y_k (1 - y_k) h_j$ Equivalently, $w_{kj}(\text{new}) = w_{kj}(\text{old}) + \eta (t_k - y_k) y_k (1 - y_k) h_j$ Output-layer error term: $\delta_k = (t_k - y_k) y_k (1 - y_k)$ $\Delta w_{kj} = \eta \delta_k h_j$ $w_{kj}(\text{new}) = w_{kj}(\text{old}) + \Delta w_{kj}$ Bias update: Since $\partial a_k / \partial b_k = 1$, $b_k(\text{new}) = b_k(\text{old}) + \eta (t_k - y_k) y_k (1 - y_k)$ Final result: The factor $(t_k - y_k)$ represents the prediction error, $y_k(1 - y_k)$ is the derivative of sigmoid, and h_j is the input associated with the weight being updated.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	75
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Back-propagation
Question	Assume that all neurons use the sigmoid activation function. Perform a forward pass and a backward pass for the network shown below. The target output is $y = 1$, and the learning rate is $\eta = 0.9$. Update all weights and biases once, and then perform another forward pass. Input Layer $x_1 = 1$ $x_2 = 0$ $x_3 = 1$ Hidden Layer H_4 (bias $b_4 = -0.40$) H_5 (bias $b_5 = 0.20$) Output Layer O_6 (bias $b_6 = 0.10$) Weights: $w_{14} = 0.20$ $w_{24} = 0.40$ $w_{34} = -0.50$ $w_{15} = -0.30$ $w_{25} = 0.10$ $w_{35} = 0.20$ $w_{46} = -0.30$ $w_{56} = -0.20$ All hidden and output neurons use sigmoid activation.

Long
Answer

Solution: First Forward Pass

Hidden neuron H_4

$$a_4 = w_{14}x_1 + w_{24}x_2 + w_{34}x_3 + b_4$$

$$a_4 = (0.20)(1) + (0.40)(0) + (-0.50)(1) - 0.40 = -0.7000$$

$$y_4 = \sigma(a_4) = 0.3318$$

Hidden neuron H_5

$$a_5 = w_{15}x_1 + w_{25}x_2 + w_{35}x_3 + b_5$$

$$a_5 = (-0.30)(1) + (0.10)(0) + (0.20)(1) + 0.20 = 0.1000$$

$$y_5 = \sigma(a_5) = 0.5250$$

Output neuron O_6

$$a_6 = w_{46}y_4 + w_{56}y_5 + b_6$$

$$a_6 = (-0.30)(0.3318) + (-0.20)(0.5250) + 0.10 = -0.1045$$

$$y_6 = \sigma(a_6) = 0.4739$$

Prediction error = $1 - 0.4739 = 0.5261$

Solution: Backward Pass

Output delta

$$\delta_6 = y_6(1 - y_6)(t - y_6)$$

$$\delta_6 = (0.4739)(1 - 0.4739)(1 - 0.4739) = 0.131169$$

Hidden deltas

$$\delta_4 = y_4(1 - y_4)w_{46}\delta_6$$

$$\delta_4 = (0.3318)(1 - 0.3318)(-0.30)(0.131169) = -0.008725$$

$$\delta_5 = \gamma_5(1 - \gamma_5)w_{56}\delta_6$$

$$\delta_5 = (0.5250)(1 - 0.5250)(-0.20)(0.131169) = -0.006542$$

Updated Connection Weights

i	j	Old w_{ij}	δ_j	Input x_i	η	Δw_{ij}	Updated w_{ij}
4	6	-0.3000	0.131169	0.3318	0.9	0.039171	-0.2608
5	6	-0.2000	0.131169	0.5250	0.9	0.061975	-0.1380
1	4	0.2000	-0.008725	1.0000	0.9	-0.007852	0.1921
1	5	-0.3000	-0.006542	1.0000	0.9	-0.005888	-0.3059
2	4	0.4000	-0.008725	0.0000	0.9	-0.000000	0.4000
2	5	0.1000	-0.006542	0.0000	0.9	-0.000000	0.1000
3	4	-0.5000	-0.008725	1.0000	0.9	-0.007852	-0.5079
3	5	0.2000	-0.006542	1.0000	0.9	-0.005888	0.1941

Updated Biases

Bias	Old value	δ	η	Change	Updated value
b4	-0.4000	-0.008725	0.9	-0.007852	-0.4079
b5	0.2000	-0.006542	0.9	-0.005888	0.1941
b6	0.1000	0.131169	0.9	0.118052	0.2181

Second Forward Pass

$$a_4(\text{new}) = (0.1921)(1) + (0.4000)(0) + (-0.5079)(1) + (-0.4079) = -0.7236$$

	$y_4(\text{new}) = \sigma(-0.7236) = 0.3266$ $a_5(\text{new}) = (-0.3059)(1) + (0.1000)(0) + (0.1941)(1) + (0.1941) = 0.0823$ $y_5(\text{new}) = \sigma(0.0823) = 0.5206$ $a_6(\text{new}) = (-0.2608)(0.3266) + (-0.1380)(0.5206) + (0.2181) = 0.0610$ $y_6(\text{new}) = \sigma(0.0610) = 0.5152$ $\text{New error} = 1 - 0.5152 = 0.4848$ Conclusion: The network output changes from 0.4739 to 0.5152, moving toward the target value 1.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	76
Question Type	
Difficulty level	Difficult
Chapter	Unit 2
Topic	Back-propagation
Question	For an artificial neural network trained using back-propagation, derive the weight-update and bias-update expressions for an output neuron that uses the tanh activation function. Assume that the network is optimized using Stochastic Gradient Descent (SGD) and that the squared-error loss function is used. Show all steps of the derivation clearly.
Long Answer	Consider output neuron k. Let: • h_j: output of neuron j in the preceding layer • w_{kj}: weight connecting neuron j to output neuron k • b_k: bias of output neuron k • a_k: net input to output neuron k • y_k: predicted output • t_k: target output • η: learning rate 1. Net input to the output neuron $a_k = \sum_j w_{kj}h_j + b_k$ 2. Output of the neuron $y_k = \tanh(a_k)$ 3. Squared-error loss $E_k = \frac{1}{2}(t_k - y_k)^2$ Using gradient descent, the weight-update rule is: $w_{kj}(\text{new}) = w_{kj}(\text{old}) - \eta(\partial E_k / \partial w_{kj})$ 4. Apply the chain rule $\partial E_k / \partial w_{kj} = (\partial E_k / \partial y_k)(\partial y_k / \partial a_k)(\partial a_k / \partial w_{kj})$ Error derivative:

	$\partial E_k / \partial y_k = y_k - t_k$ Derivative of tanh: $\partial y_k / \partial a_k = 1 - \tanh^2(a_k) = 1 - y_k^2$ Net-input derivative: $\partial a_k / \partial w_{kj} = h_j$ 5. Error gradient $\partial E_k / \partial w_{kj} = (y_k - t_k)(1 - y_k^2)h_j$ 6. Final weight-update expression $w_{kj}(\text{new}) = w_{kj}(\text{old}) - \eta(y_k - t_k)(1 - y_k^2)h_j$ Equivalently, $w_{kj}(\text{new}) = w_{kj}(\text{old}) + \eta(t_k - y_k)(1 - y_k^2)h_j$ Output-layer error term: $\delta_k = (t_k - y_k)(1 - y_k^2)$ $\Delta w_{kj} = \eta \delta_k h_j$ $w_{kj}(\text{new}) = w_{kj}(\text{old}) + \Delta w_{kj}$ Bias update: Since $\partial a_k / \partial b_k = 1$, $b_k(\text{new}) = b_k(\text{old}) + \eta(t_k - y_k)(1 - y_k^2)$ Final result: The factor $(t_k - y_k)$ represents the prediction error, $(1 - y_k^2)$ is the derivative of tanh, and h_j is the input associated with the weight being updated.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	77
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Optimizers
Question	Design a pseudocode algorithm for the Adam Optimizer and analyze its computational complexity.
Long Answer	Initialize: $\theta \leftarrow \text{initial parameters}$ $m \leftarrow 0$ $v \leftarrow 0$ $t \leftarrow 0$ Repeat until convergence: $t \leftarrow t + 1$ Compute gradient g $m \leftarrow \beta_1 \times m + (1 - \beta_1) \times g$ $v \leftarrow \beta_2 \times v + (1 - \beta_2) \times g^2$ $\hat{m} \leftarrow m / (1 - \beta_1^t)$ $\hat{v} \leftarrow v / (1 - \beta_2^t)$ $\theta \leftarrow \theta - \eta \times \hat{m} / (\sqrt{\hat{v}} + \epsilon)$ Return θ Computational Complexity Optimizer-update time complexity: $O(n)$, where n is the number of parameters, because the Adam moment and parameter updates are element-

	wise. This $O(n)$ statement excludes the cost of computing the gradient itself, which depends on the model and the batch size. Extra optimizer-state space complexity: $O(n)$, because Adam stores first-moment (m) and second-moment (v) tensors for the parameters. This is roughly two additional parameter-sized state tensors. Plain SGD has no additional moment state, although training still requires parameters and gradients.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	78
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Optimizers
Question	Compute one iteration of Gradient Descent and one iteration of Gradient Descent with Momentum when $\beta = 0.9$, previous velocity = 2, parameter $\theta = 6$, learning rate $\eta = 0.1$ and gradient = 8.
Long Answer	Gradient Descent Given $\theta = 6$, gradient = 8, learning rate $\eta = 0.1$ Update: $\theta = 6 - 0.1(8) = 6 - 0.8 = 5.2$ Gradient Descent with Momentum Update rule: $v_t = \beta v_{t-1} + \eta \nabla J(\theta)$, then $\theta = \theta - v_t$ Velocity: $v = 0.9(2) + 0.1(8) = 1.8 + 0.8 = 2.6$ Update: $\theta = 6 - 2.6 = 3.4$ Comparison: plain Gradient Descent gives $\theta = 5.2$, whereas Momentum gives $\theta = 3.4$. The larger step is produced by the velocity carried forward from the previous iteration, which is what accelerates convergence along a consistent gradient direction.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	79
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Activation Functions
Question	Explain the role of activation functions in a neural network. Why is a non-linear activation function necessary, and what happens if every layer uses a linear activation?
Long Answer	Role of the activation function: each neuron computes an affine pre-activation $z = w \cdot x + b$ and applies an activation function f to give $f(z)$. Without a non-linear activation, compositions of layers remain affine. For gradient-based training, an activation should provide a usable derivative or subgradient almost everywhere; it need not be differentiable at every point, as illustrated by ReLU at $x = 0$. What happens if every layer is linear: A two-layer network computes $h = W_1 x + b_1$ and $y = W_2 h + b_2$. Substituting, $y = W_2(W_1 x + b_1) + b_2 = (W_2 W_1) x + (W_2 b_1 + b_2)$ Writing $W' = W_2 W_1$ and $b' = W_2 b_1 + b_2$ gives $y = W' x + b'$, which is a single linear

	layer. The same argument applies at any depth. Consequences: • Adding depth adds parameters but no representational power. • The network can represent only a linear function, so it cannot solve a problem that is not linearly separable, XOR being the standard example. • The purpose of the hidden layers is lost. A non-linear activation prevents this collapse, so each layer transforms the representation produced by the one below it and successive layers build increasingly complex features.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	80																												
Question Type																													
Difficulty level	Medium																												
Chapter	2																												
Topic	Support Vector Machines																												
Question	Consider the following training dataset: <table><tr><th>Sample</th><th>x_1</th><th>x_2</th><th>Class (y)</th></tr><tr><td>A</td><td>2</td><td>2</td><td>-1</td></tr><tr><td>B</td><td>4</td><td>4</td><td>-1</td></tr><tr><td>C</td><td>4</td><td>0</td><td>+1</td></tr><tr><td>D</td><td>6</td><td>2</td><td>+1</td></tr></table> Assume that the SVM classifier has found the optimal decision boundary in canonical form, with $w = [0.5, -0.5]$ and $b = -1$, that is $0.5x_1 - 0.5x_2 - 1 = 0$. 1. Determine the class (+1 or -1) of each training sample using the decision function. (2 Marks) 2. Identify the support vectors from the dataset. (2 Marks) 3. Calculate the margin of the classifier. (2 Marks) 4. Explain why maximizing the margin improves the performance of an SVM classifier. (2 Marks)				Sample	x_1	x_2	Class (y)	A	2	2	-1	B	4	4	-1	C	4	0	+1	D	6	2	+1					
Sample	x_1	x_2	Class (y)																										
A	2	2	-1																										
B	4	4	-1																										
C	4	0	+1																										
D	6	2	+1																										
Long Answer	Step 1: Classify the samples (2 Marks) The decision function is $f(x) = 0.5x_1 - 0.5x_2 - 1$. If $f(x) > 0$ the class is +1; if $f(x) < 0$ the class is -1. <table><tr><th>Sample</th><th>Calculation</th><th>$f(x)$</th><th>Predicted class</th><th>Given label</th></tr><tr><td>A(2, 2)</td><td>$0.5(2) - 0.5(2) - 1$</td><td>-1</td><td>-1</td><td>-1</td></tr><tr><td>B(4, 4)</td><td>$0.5(4) - 0.5(4) - 1$</td><td>-1</td><td>-1</td><td>-1</td></tr><tr><td>C(4, 0)</td><td>$0.5(4) - 0.5(0) - 1$</td><td>+1</td><td>+1</td><td>+1</td></tr><tr><td>D(6, 2)</td><td>$0.5(6) - 0.5(2) - 1$</td><td>+1</td><td>+1</td><td>+1</td></tr></table> All four predictions agree with the given labels, so the boundary separates the data				Sample	Calculation	$f(x)$	Predicted class	Given label	A(2, 2)	$0.5(2) - 0.5(2) - 1$	-1	-1	-1	B(4, 4)	$0.5(4) - 0.5(4) - 1$	-1	-1	-1	C(4, 0)	$0.5(4) - 0.5(0) - 1$	+1	+1	+1	D(6, 2)	$0.5(6) - 0.5(2) - 1$	+1	+1	+1
Sample	Calculation	$f(x)$	Predicted class	Given label																									
A(2, 2)	$0.5(2) - 0.5(2) - 1$	-1	-1	-1																									
B(4, 4)	$0.5(4) - 0.5(4) - 1$	-1	-1	-1																									
C(4, 0)	$0.5(4) - 0.5(0) - 1$	+1	+1	+1																									
D(6, 2)	$0.5(6) - 0.5(2) - 1$	+1	+1	+1																									

	with zero training error. Step 2: Identify the support vectors (2 Marks) The support vectors are the points closest to the decision boundary. The distance of a point from the hyperplane is $f(x) / w$. $ w = \sqrt{(0.5^2 + (-0.5)^2)} = \sqrt{0.5} = 0.707$ Distance of A = $-1 /0.707 = 1.414$ Distance of B = $-1 /0.707 = 1.414$ Distance of C = $+1 /0.707 = 1.414$ Distance of D = $+1 /0.707 = 1.414$ Every sample satisfies $y f(x) = 1$ and all four are equally distant from the hyperplane. Therefore A, B, C and D all act as support vectors. Step 3: Calculate the margin (2 Marks) Margin = $2/ w = 2/0.707 = 2.828$ units This is twice the per-point distance of 1.414 obtained in Step 2, as it must be, since the margin spans both sides of the boundary. Step 4: Importance of maximizing the margin (2 Marks) • A larger margin increases the separation between the two classes. • It reduces the chance of misclassification for new data. • It improves the generalization ability of the classifier. • It makes the model less sensitive to noise and to overfitting.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	81
Question Type	
Difficulty level	Medium
Chapter	2
Topic	Support Vector Machines
Question	An SVM has been trained on a linearly separable dataset and the maximum-margin hyperplane has been found. A new training sample is now added and the SVM is retrained. Analyze the effect on the hyperplane and on the margin in each of the following three cases, with justification. (a) The new sample lies well outside the margin, on the correct side of the boundary. (2 Marks) (b) The new sample lies exactly on one of the supporting hyperplanes. (3 Marks) (c) The new sample lies inside the margin, on the correct side of the boundary. (3 Marks)
Long Answer	General principle: the maximum-margin hyperplane is fixed only by the samples lying on the supporting hyperplanes. A sample constrains the solution only if $y f(x) = 1$; if $y f(x) > 1$ it is inactive. (a) Sample well outside the margin, correctly classified (2 Marks) • Here $y f(x) > 1$, so the sample satisfies the separation requirement with room to spare and adds no active constraint. • The hyperplane, the margin and the support vectors are unchanged. (b) Sample exactly on a supporting hyperplane (3 Marks) • Here $y f(x) = 1$, so the constraint is active and the sample becomes a new support vector. • The existing hyperplane still satisfies every constraint, and no wider margin is available, so the optimal solution does not move. • The hyperplane and the margin are unchanged, but the number of support vectors increases by one. (c) Sample inside the margin, correctly classified (3 Marks)

	• Here $0 < y f(x) < 1$, so the sample is on the correct side but violates the current margin. • The band around the boundary is no longer empty, so the existing hyperplane is no longer a valid solution and the optimizer must find a new one. • The optimizer must find a new maximum-margin separator. The boundary may shift, and because an additional constraint has been added, the new margin can only shrink or stay the same. • The new sample may become a support vector, but this is not guaranteed: after re-optimization it could lie outside the new margin. The set of previous support vectors may also change. Summary: adding data can only reduce the margin of a separable SVM, never increase it, and only samples at or inside the margin have any effect.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	82
Question Type	
Difficulty level	Difficult
Chapter	2
Topic	Optimizers
Question	The loss surface of a model has the shape of a long narrow valley: the gradient is very steep across the valley and very shallow along it, and the minimum lies at the far end. Analyze how Batch Gradient Descent, SGD, Momentum, Adagrad and Adam each behave on this surface, and identify which methods are expected to handle this geometry most effectively. Do not assume that one optimizer is universally the fastest.
Long Answer	Why this surface is hard: the curvature differs greatly between the two directions. Any η small enough to stay stable across the steep direction is far too small to make progress along the shallow one. • Batch Gradient Descent: the gradient points mostly across the valley, so the path zig-zags from wall to wall while creeping forward. The step size is limited by the steep direction, and raising η makes the crossing component overshoot. • SGD: the same zig-zag with sampling noise added, so the path is more erratic. Steps are cheap, so it covers more updates in the same time, but it cannot settle without decaying η. • Momentum: exactly the case it is designed for. The across-valley gradient reverses sign each step and cancels in the running average, while the along-valley component accumulates. The effective step along the floor is amplified by about $1/(1 - \beta)$, roughly ten times at $\beta = 0.9$. • Adagrad: attacks the problem from the other side, giving each direction its own step size. The steep direction builds a large G_t and is damped; the shallow one keeps a larger step. Its weakness is that G_t never decreases, so on a long valley it can stall before reaching the end. • Adam: combines momentum-like first-moment smoothing with adaptive scaling from the second-moment estimate. This can reduce oscillation and adjust effective step sizes across parameters. The second moment does not literally compute or equalize the curvature, but it often helps when gradient magnitudes differ substantially. Conclusion: no optimizer is guaranteed to converge fastest from this qualitative description alone. Momentum is particularly well suited to long narrow valleys because it damps alternating cross-valley motion and accumulates progress along

	consistent directions. Adam can also be very effective because it combines momentum with adaptive scaling. Which is fastest depends on the learning rate, momentum coefficients, parameter scaling and stochasticity. Adagrad may make strong early progress but its accumulated squared-gradient term can make later steps very small, while plain Batch Gradient Descent and SGD can exhibit substantial zig-zagging.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	83
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain the concepts of hyperplane, margin, and support vectors in an SVM classifier, with the relevant equations.
Long Answer	Hyperplane: - The hyperplane is the decision boundary that separates the two classes. $w^T x + b = 0$ w is the weight vector, normal (perpendicular) to the hyperplane and determining its orientation; b is the bias, which shifts the hyperplane without changing its orientation. Margin: - The margin is the perpendicular distance between the decision boundary and the closest data points of each class. - The two margin boundaries ("gutters") are parallel to the hyperplane: $w^T x + b = 1 \quad \text{and} \quad w^T x + b = -1$ - The width of the margin is: $\text{Margin Width} = \frac{2}{\ w\ }$ - SVM chooses w and b to maximise this margin, which improves generalisation on unseen data. Support Vectors: - Support vectors are the training points that lie exactly on the margin boundaries — the points closest to the hyperplane. - They alone determine the position and orientation of the optimal hyperplane; removing a non-support-vector point does not change the solution, but removing a support vector can. - In the dual formulation, support vectors are exactly the points with non-zero Lagrange multiplier ($\alpha_i > 0$). Classification constraint for every training sample: $y_i(w^T x_i + b) \geq 1$
Marks	8
-ve applicable?	N
% Applicable	N

Question No	84
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain Hard-Margin SVM classification. State its primal optimization problem and discuss its limitations.
Long Answer	Hard-Margin SVM: - Used only when the training data is perfectly linearly separable — a single hyperplane can separate both classes with zero misclassification. - The objective is to find the hyperplane that maximises the margin between the classes, since a wider margin generalises better. Decision boundary and margin boundaries: $w^T x + b = 0 \quad (\text{decision boundary})$ $w^T x + b = 1 \quad \text{and} \quad w^T x + b = -1 \quad (\text{margin boundaries})$ $\text{Margin Width} = \frac{2}{\ w\ }$ Classification constraint (every point must lie on the correct side, outside the margin): $y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, m$ Primal optimization problem: $\min_{w,b} \frac{1}{2} \ w\ ^2 \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1$ - Minimising $\frac{1}{2} \ w\ ^2$ is mathematically equivalent to maximising the margin $2/\ w\$, and gives a convex quadratic program with a unique global optimum. Limitations: - Requires the data to be perfectly linearly separable; fails entirely if even one point cannot satisfy the constraint. - Extremely sensitive to outliers and noise — a single mislabeled or borderline point can make the problem infeasible or force a very narrow margin. - Cannot handle overlapping classes, which is common in real-world data — this motivates Soft-Margin SVM.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	85
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	What are Kernel functions in SVM? Explain the Linear kernel and the Polynomial kernel with their equations and typical use cases.
Long Answer	Kernel functions: A kernel function computes the dot product of two points as if they had first been mapped into a higher-dimensional feature space $\phi(\cdot)$, without ever performing that mapping explicitly.

	$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ 2. This "kernel trick" lets SVM build non-linear decision boundaries in the original space while only ever computing dot products, which keeps the dual optimisation efficient. Linear Kernel: 1. Corresponds to no transformation of the input space, i.e. $\phi(x) = x$. $K(x_i, x_j) = x_i^T x_j$ 2. Used when the data is already linearly, or approximately linearly, separable, or when the number of features is very large relative to the number of samples (e.g. text classification). It has no extra hyperparameters, so it is fast and less prone to overfitting. Polynomial Kernel: $K(x_i, x_j) = (x_i^T x_j + c)^d$ 1. Here d is the degree of the polynomial and c is a constant that trades off the influence of higher-order versus lower-order terms. 2. It implicitly maps the data into a higher-dimensional space of polynomial feature combinations, allowing curved decision boundaries. 3. Used when class boundaries are visibly non-linear; however, a very high degree d increases model complexity and risks overfitting.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	86
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain the KKT conditions for Hard-Margin SVM classification, and illustrate with a numerical example how they identify the support vectors.
Long Answer	KKT conditions for the Hard-Margin SVM Lagrangian: $L(w, b, \alpha) = \frac{1}{2} \ w\ ^2 - \sum_i \alpha_i [y_i(w^T x_i + b) - 1]$ 1. Stationarity: $\nabla_w L = 0 \Rightarrow w = \sum \alpha_i y_i x_i$; $\nabla_b L = 0 \Rightarrow \sum \alpha_i y_i = 0$ 2. Primal feasibility: $y_i(w^T x_i + b) - 1 \geq 0$ 3. Dual feasibility: $\alpha_i \geq 0$ 4. Complementary slackness: $\alpha_i [y_i(w^T x_i + b) - 1] = 0$ Illustration: suppose after solving the dual we obtain the multipliers for four training points: 1. Point A: $\alpha = 2.5, y_i(w^T x_i + b) = 1 \rightarrow$ complementary slackness satisfied with $\alpha > 0 \Rightarrow$ A is a support vector, lying exactly on the margin. 2. Point B: $\alpha = 1.1, y_i(w^T x_i + b) = 1 \rightarrow$ also a support vector, on the opposite margin boundary. 3. Point C: $\alpha = 0, y_i(w^T x_i + b) = 2.4 \rightarrow$ strictly outside the margin; complementary slackness is satisfied trivially ($\alpha = 0$), so C is not a support vector and does not appear in $w = \sum \alpha_i y_i x_i$. 4. Point D: $\alpha = 0, y_i(w^T x_i + b) = 1.0 \rightarrow$ lies on the margin boundary but has zero multiplier; it is a boundary point but does not actively define

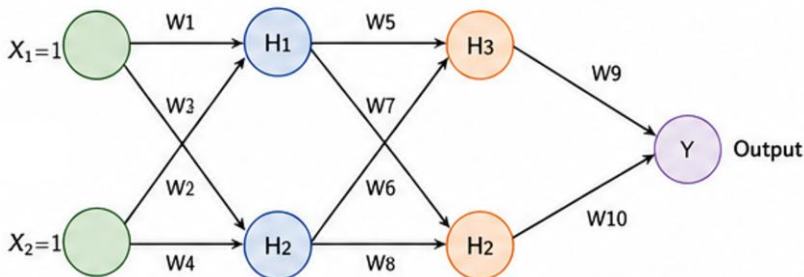
	win this solution. Only points A and B — those with strictly positive α_i — contribute to: $w = \sum_i \alpha_i y_i x_i = 2.5 \cdot y_A x_A + 1.1 \cdot y_B x_B$ which confirms that the optimal hyperplane is fully determined by the support vectors alone.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	87
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Compare Hard-Margin and Soft-Margin SVM in terms of formulation, constraints, and suitability, with reference to a dataset containing noisy or overlapping points.
Long Answer	Formulation and constraints: Hard Margin: $\min \frac{1}{2} \ w\ ^2$ subject to $y_i(w^T x_i + b) \geq 1$ Soft Margin: $\min \frac{1}{2} \ w\ ^2 + C \sum_i \xi_i$ subject to $y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$ - The slack variable ξ_i measures how far a point falls short of the required margin: $\xi_i = 0$ means the point satisfies the hard constraint, $0 < \xi_i \leq 1$ means it lies inside the margin but is still correctly classified, and $\xi_i > 1$ means it is misclassified. Key differences: - Hard Margin permits absolutely no violation, so it requires perfect linear separability and is undefined (infeasible) otherwise. - Soft Margin tolerates a controlled number of violations, governed by the regularisation parameter C, and always has a feasible solution. - Hard Margin is extremely sensitive to outliers — one noisy point can drastically shrink the margin or make the problem infeasible. - Soft Margin is far more robust: a small number of noisy points simply incur a slack penalty rather than breaking the model. Application to a dataset with noisy/overlapping points: - If such a dataset were forced through Hard-Margin SVM, either no separating hyperplane would exist, or the solver would fit the outliers exactly, producing an unnecessarily narrow, noise-sensitive margin. - Soft-Margin SVM instead allows a few points to sit inside the margin or even be misclassified (paying a cost $C \cdot \xi_i$ each), while keeping the margin as wide as possible for the majority of "clean" points. This trade-off, controlled by C, generally produces a hyperplane that generalises better to unseen data — making Soft Margin the appropriate choice whenever real-world noise or class overlap is expected.
Marks	8
-ve applicable?	N
% Applicable	N

Question No	88
Question Type	
Difficulty level	Difficult
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Derive the Lagrangian dual of the Soft-Margin SVM starting from its primal formulation, obtain the stationarity conditions, and explain the role of the resulting box constraint $0 \leq \alpha_i \leq C$ in classifying a new test point.
Long Answer	Primal: $\min \frac{1}{2} \ w\ ^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$ Lagrangian ($\alpha_i, \mu_i \geq 0$): $L = \frac{1}{2} \ w\ ^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(w^T x_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i$ Stationarity: $\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum \alpha_i y_i x_i$ $\frac{\partial L}{\partial b} = 0 \Rightarrow \sum \alpha_i y_i = 0$ $\frac{\partial L}{\partial \xi_i} = 0 \Rightarrow \alpha_i + \mu_i = C \Rightarrow \mu_i = C - \alpha_i \geq 0 \Rightarrow \alpha_i \leq C$ Substituting back eliminates w, b, ξ, and μ, giving the dual: $\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i^T x_j)$ subject to $0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0$ Decision function for a new point x: using $w = \sum \alpha_i y_i x_i$, classification needs only dot products — $f(x) = \text{sign} \left(\sum_i \alpha_i y_i (x_i^T x) + b \right)$ - Because the dot product $x_i^T x$ can be replaced with any kernel $K(x_i, x)$, this same dual/decision-function structure extends directly to non-linear SVMs. Role of the box constraint $0 \leq \alpha_i \leq C$: $\alpha_i = 0$: the point is correctly classified outside the margin and plays no role in defining w — it is not a support vector. $0 < \alpha_i < C$ (an "unbound" support vector): $\mu_i > 0$ forces $\xi_i = 0$ by complementary slackness, so the point lies exactly on a margin boundary. These points are also used to solve for b, since $y_i(w^T x_i + b) = 1$ exactly. $\alpha_i = C$ (a "bound" support vector): the point is allowed to violate the margin ($\xi_i > 0$), i.e. it lies inside the margin or is misclassified — its influence on w is capped at C so that no single noisy point can dominate the solution. Thus C directly limits how much any one training point can influence the final hyperplane, which is precisely what makes Soft-Margin SVM robust to noisy or overlapping data compared to the unconstrained (Hard-Margin) case.
Marks	8
-ve applicable?	N

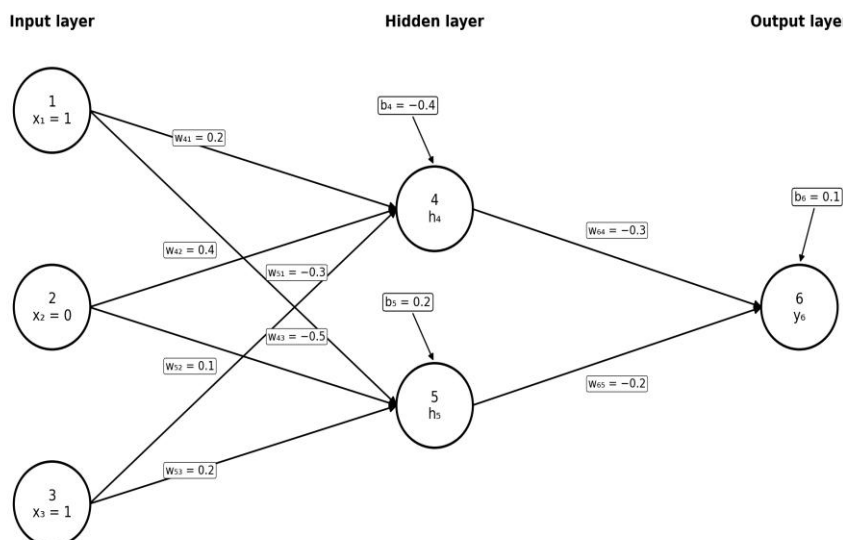
% Applicable	N
--------------	---

Unit 2 – 10 Marks Descriptive Question Bank

Question No	89
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Back-propagation
Question	Consider the neural network shown in the figure. All weights W_1 to W_{10} are initially 1, the learning rate is 0.1, and every neuron uses the ReLU activation function. For $X_1 = 1$ and $X_2 = 1$, the expected output is 5. Input Layer Hidden Layer 1 (2 neurons) Hidden Layer 2 (2 neurons) Output Layer (1 neuron)  (i) Calculate the squared error using $E = \frac{1}{2}(\text{expected output} - \text{actual output})^2$. (ii) Using one back-propagation/SGD update, calculate the new value of W_9.
Long Answer	Part A: Forward propagation Step 1: First hidden layer Upper neuron of the first hidden layer: $\text{net}_1 = X_1 \times W_1 + X_2 \times W_3 = 1 \times 1 + 1 \times 1 = 2$ $O_1 = \text{ReLU}(\text{net}_1) = \text{ReLU}(2) = 2$ Lower neuron of the first hidden layer: $\text{net}_2 = X_1 \times W_4 + X_2 \times W_2 = 1 \times 1 + 1 \times 1 = 2$ $O_2 = \text{ReLU}(\text{net}_2) = \text{ReLU}(2) = 2$ Step 2: Second hidden layer Upper neuron of the second hidden layer: $\text{net}_3 = O_1 \times W_5 + O_2 \times W_6 = 2 \times 1 + 2 \times 1 = 4$ $O_3 = \text{ReLU}(\text{net}_3) = \text{ReLU}(4) = 4$ Lower neuron of the second hidden layer: $\text{net}_4 = O_1 \times W_7 + O_2 \times W_8 = 2 \times 1 + 2 \times 1 = 4$ $O_4 = \text{ReLU}(\text{net}_4) = \text{ReLU}(4) = 4$ Step 3: Output layer

	$\text{net}_5 = O_3 \times W_9 + O_4 \times W_{10} = 4 \times 1 + 4 \times 1 = 8$
	$O_5 = \text{ReLU}(\text{net}_5) = \text{ReLU}(8) = 8$
	Part B: Squared error $E = \frac{1}{2}(T - O_5)^2$ $E = \frac{1}{2}(5 - 8)^2 = \frac{1}{2}(-3)^2 = \frac{1}{2}(9)$
	$E = 4.5$
	Part C: Detailed differentiation and update of W_9
	Step 1: SGD update rule $W_9 (\text{new}) = W_9 (\text{old}) - \eta \partial E / \partial W_9$
	Step 2: Apply the chain rule The dependency path is $W_9 \rightarrow \text{net}_5 \rightarrow O_5 \rightarrow E$. Therefore: $\partial E / \partial W_9 = (\partial E / \partial O_5)(\partial O_5 / \partial \text{net}_5)(\partial \text{net}_5 / \partial W_9)$
	Step 3: Differentiate E with respect to O_5 $E = \frac{1}{2}(T - O_5)^2$ $\partial E / \partial O_5 = \frac{1}{2} \times 2(T - O_5) \times \partial(T - O_5) / \partial O_5$ $\partial(T - O_5) / \partial O_5 = -1$ $\partial E / \partial O_5 = -(T - O_5) = O_5 - T$
	$\partial E / \partial O_5 = 8 - 5 = 3$
	Step 4: Differentiate O_5 with respect to net_5 For ReLU: $\text{ReLU}'(\text{net}) = 0 \text{ for } \text{net} < 0, \text{ and } 1 \text{ for } \text{net} > 0$ Since $\text{net}_5 = 8 > 0$:
	$\partial O_5 / \partial \text{net}_5 = \text{ReLU}'(\text{net}_5) = \text{ReLU}'(8) = 1$
	Step 5: Differentiate net_5 with respect to W_9 $\text{net}_5 = O_3 \times W_9 + O_4 \times W_{10}$ $\partial \text{net}_5 / \partial W_9 = \partial(O_3 W_9 + O_4 W_{10}) / \partial W_9$
	$\partial \text{net}_5 / \partial W_9 = O_3 + 0 = 4$
	Step 6: Compute the gradient
	$\partial E / \partial W_9 = (3)(1)(4) = 12$
	Step 7: Update W_9 $W_9 (\text{new}) = W_9 (\text{old}) - \eta(\partial E / \partial W_9)$ $W_9 (\text{new}) = 1 - 0.1(12)$

	$W_9 \text{ (new)} = 1 - 1.2 = -0.2$
Marks	10
-ve applicable?	N
% Applicable	N

Question No	90
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Back-propagation
Question	Consider the multilayer feed-forward neural network shown in the figure. The notation w_{kj} denotes the weight connecting neuron j to neuron k.  The diagram shows a neural network with three layers: Input layer (neurons 1, 2, 3), Hidden layer (neurons 4, 5), and Output layer (neuron 6). Weights are labeled on the connections: $w_{41}=0.2$, $w_{42}=0.4$, $w_{43}=-0.5$, $w_{51}=-0.3$, $w_{52}=0.1$, $w_{53}=0.2$, $w_{64}=-0.3$, and $w_{65}=-0.2$. Biases are shown as arrows pointing to each neuron: $b_4=-0.4$, $b_5=0.2$, and $b_6=0.1$. Input values are $x_1=1$, $x_2=0$, and $x_3=1$. A note states: 'All non-input neurons use sigmoid activation. Weight convention: w_{kj} is the weight from neuron j to neuron k.' The inputs are $x_1 = 1$, $x_2 = 0$ and $x_3 = 1$. The weights are $w_{41} = 0.2$, $w_{42} = 0.4$, $w_{43} = -0.5$, $w_{51} = -0.3$, $w_{52} = 0.1$, $w_{53} = 0.2$, $w_{64} = -0.3$ and $w_{65} = -0.2$. The neuron-specific biases are $b_4 = -0.4$, $b_5 = 0.2$ and $b_6 = 0.1$. Assume that neurons 4, 5 and 6 use the sigmoid activation function, the target output is $t = 1$, and the squared-error loss $E = \frac{1}{2}(t - y_6)^2$ is used. Using the negative-gradient convention $\delta_r = -\partial E / \partial a_r$, calculate the error signals δ_6, δ_4 and δ_5. Show clearly how the output-layer error is propagated to the hidden layer. Notation used: a_r denotes the net input to neuron r. h_4 and h_5 are hidden-layer outputs, and y_6 is the output-layer prediction. Each neuron has its own bias
Long Answer	Step 1: Sigmoid activation and derivative $\sigma(a) = 1 / (1 + e^{-a})$ $\sigma'(a) = \sigma(a)[1 - \sigma(a)]$ Therefore, for hidden neuron j with output h_j, the derivative is $h_j(1 - h_j)$. For output neuron 6, the derivative is $y_6(1 - y_6)$. Step 2: Forward propagation at hidden neuron 4

$$a_4 = w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + b_4$$

$$a_4 = (0.2)(1) + (0.4)(0) + (-0.5)(1) - 0.4$$

$$a_4 = -0.7$$

$$h_4 = \sigma(a_4) = \sigma(-0.7) = 0.331812$$

Step 3: Forward propagation at hidden neuron 5

$$a_5 = w_{51}x_1 + w_{52}x_2 + w_{53}x_3 + b_5$$

$$a_5 = (-0.3)(1) + (0.1)(0) + (0.2)(1) + 0.2$$

$$a_5 = 0.1$$

$$h_5 = \sigma(a_5) = \sigma(0.1) = 0.524979$$

Step 4: Forward propagation at output neuron 6

$$a_6 = w_{64}h_4 + w_{65}h_5 + b_6$$

$$a_6 = (-0.3)(0.331812) + (-0.2)(0.524979) + 0.1$$

$$a_6 = -0.104540$$

$$y_6 = \sigma(a_6) = 0.473889$$

Step 5: Squared-error loss

$$E = \frac{1}{2}(t - y_6)^2$$

$$E = \frac{1}{2}(1 - 0.473889)^2$$

$$E = 0.138396$$

Step 6: Output-layer error signal δ_6

Using the negative-gradient definition:

$$\delta_6 = -\partial E / \partial a_6$$

Apply the chain rule:

$$\delta_6 = -(\partial E / \partial y_6)(\partial y_6 / \partial a_6)$$

$$\partial E / \partial y_6 = y_6 - t$$

$$\partial y_6 / \partial a_6 = y_6(1 - y_6)$$

$$\delta_6 = (t - y_6)y_6(1 - y_6)$$

$$\delta_6 = (1 - 0.473889)(0.473889)(1 - 0.473889)$$

$$\delta_6 = 0.131169$$

Step 7: General error expression for an intermediate layer

For hidden neuron j , its error is obtained from all neurons k in the next layer:

$$\delta_j = -\partial E / \partial a_j = h_j(1 - h_j) \sum_k w_{kj} \delta_k$$

Here w_{kj} is the weight from hidden neuron j to next-layer neuron k . Because this network has only one output neuron, $k = 6$, the summation contains one term.

Step 8: Hidden-layer error signal δ_4

$$\delta_4 = h_4(1 - h_4) \sum_k w_{k4} \delta_k$$

$$\delta_4 = h_4(1 - h_4)w_{64}\delta_6$$

	$\delta_4 = (0.331812)(1 - 0.331812)(-0.3)(0.131169)$												
	$\delta_4 = -0.008725$												
	Step 9: Hidden-layer error signal δ_5												
	$\delta_5 = h_5(1 - h_5) \sum_k w_{k5} \delta_k$												
	$\delta_5 = h_5(1 - h_5)w_{65} \delta_6$												
	$\delta_5 = (0.524979)(1 - 0.524979)(-0.2)(0.131169)$												
	$\delta_5 = -0.006542$												
Final answers													
	<table><tr><th>Neuron</th><th>Error-signal equation</th><th>Value</th></tr><tr><td>6</td><td>$\delta_6 = (t - y_6)y_6(1 - y_6)$</td><td>0.131169</td></tr><tr><td>4</td><td>$\delta_4 = h_4(1 - h_4)w_{64}\delta_6$</td><td>-0.008725</td></tr><tr><td>5</td><td>$\delta_5 = h_5(1 - h_5)w_{65}\delta_6$</td><td>-0.006542</td></tr></table>	Neuron	Error-signal equation	Value	6	$\delta_6 = (t - y_6)y_6(1 - y_6)$	0.131169	4	$\delta_4 = h_4(1 - h_4)w_{64}\delta_6$	-0.008725	5	$\delta_5 = h_5(1 - h_5)w_{65}\delta_6$	-0.006542
Neuron	Error-signal equation	Value											
6	$\delta_6 = (t - y_6)y_6(1 - y_6)$	0.131169											
4	$\delta_4 = h_4(1 - h_4)w_{64}\delta_6$	-0.008725											
5	$\delta_5 = h_5(1 - h_5)w_{65}\delta_6$	-0.006542											
Marks	10												
-ve applicable?	N												
% Applicable	N												

Question No	91
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Back-propagation
Question	Gradient descent provides an efficient and principled method for learning the weights and biases of a neural network by traversing the error surface toward a minimum. (a) Using the first-order Taylor-series approximation, explain why the direction opposite to the gradient gives the steepest decrease in the loss. Derive the gradient-descent update equations for the weight vector and the bias. (5 Marks) (b) Explain the effects of a learning rate that is too small or too large, the local-minimum problem, and the role of learning-rate annealing. (5 Marks)
Long Answer	Gradient descent: The goal is to find a better way of traversing the error surface so that the minimum value can be reached quickly without using brute-force search. (a) Direction of Steepest Descent and Update Rule Step 1: Represent the parameters Let the vector of neural-network parameters be $\theta = [w, b]^T$. Assume that the parameter vector is randomly initialized. $\theta = [w, b]^T$ Step 2: Consider a small change in the parameters Let $\Delta\theta = [\Delta w, \Delta b]^T$ be a direction in which the parameters may be changed. To be conservative, move only by a small amount η in that direction. $\theta_{\text{new}} = \theta + \eta\Delta\theta$ Step 3: Apply the Taylor-series approximation to the loss For a sufficiently

small learning rate η , the higher-order terms $\eta^2, \eta^3, \dots$ are negligible. Therefore, the loss can be approximated by its first-order Taylor term.

$$L(\theta + \eta u) \approx L(\theta) + \eta u^T \nabla L(\theta)$$

$$L(\theta + \eta u) - L(\theta) \approx \eta u^T \nabla L(\theta)$$

Step 4: State the condition for a favourable move A move is favourable only when the new loss is smaller than the previous loss.

$$L(\theta + \eta u) - L(\theta) < 0$$

$$u^T \nabla L(\theta) < 0$$

Step 5: Choose the direction that decreases the loss most Let β be the angle between u and $\nabla L(\theta)$. The dot product may be written as follows:

$$u^T \nabla L(\theta) = \|u\| \|\nabla L(\theta)\| \cos(\beta)$$

This quantity is most negative when $\cos(\beta) = -1$, that is, when $\beta = 180^\circ$. Hence, the direction of movement must be opposite to the gradient.

$$\Delta \theta = u = -\nabla L(\theta)$$

Step 6: Introduce the learning rate The parameters are moved only by a small step η to avoid overshooting the minimum. The final gradient-descent rule is:

$$\theta_{(t+1)} = \theta_t - \eta \nabla L(\theta_t)$$

For the weight and bias separately:

$$w_{(t+1)} = w_t - \eta \nabla_w L(w_t, b_t)$$

$$b_{(t+1)} = b_t - \eta \nabla_b L(w_t, b_t)$$

where

$\nabla_w L = \partial L(w, b) / \partial w$ and $\nabla_b L = \partial L(w, b) / \partial b$, evaluated at $w = w_t$ and $b = b_t$

Gradient-descent algorithm

1. Set $t \leftarrow 0$.
2. Choose the maximum number of iterations.
3. While t is less than the maximum number of iterations:
 - $w_{(t+1)} \leftarrow w_t - \eta \nabla_w L(w_t, b_t)$
 - $b_{(t+1)} \leftarrow b_t - \eta \nabla_b L(w_t, b_t)$
 - $t \leftarrow t + 1$.
4. Stop when the iteration limit or another stopping condition is reached.

(b) Learning-Rate Effects and Problems of Gradient Descent

1. Learning rate is too small

- Each update changes the parameters by only a very small amount, so gradient descent takes very small steps.
- Many updates are needed to reach the minimum.
- Therefore, gradient descent can be very slow when η is too small.

2. Learning rate is too large

- A large η produces very large parameter updates.
- The update can overshoot the minimum and move to the other side of the error surface.
- Subsequent updates may repeatedly cross the minimum; the method may oscillate or may never reach the minimum.

3. Local-minimum problem

	- At a local minimum, the slope is zero; hence, the derivative or gradient term is zero. $\nabla L(\theta) = 0$ - The gradient-descent update leaves the parameter vector unchanged. $\theta_{(t+1)} = \theta_t - \eta(0) = \theta_t$ - Consequently, gradient descent may get stuck at a local minimum instead of reaching the global minimum. 4. Learning-rate annealing - There is no single ideal learning-rate value that works for every problem. - A practical approach is to try different initial learning rates, often on a logarithmic scale, and select a value that works well. - Learning-rate annealing reduces the learning rate over time. - Larger updates are made at the beginning of training, while smaller updates are made later. - This allows the network to learn good weights quickly at first and then fine-tune the weights near the minimum.
Marks	10
-ve applicable?	N
% Applicable	N

Question No	92																
Question Type																	
Difficulty level	Medium																
Chapter	Unit 2																
Topic	Back-propagation																
Question	For a multilayer artificial neural network trained using back-propagation and Stochastic Gradient Descent (SGD), derive the weight-update and bias-update expressions for a hidden-layer neuron. Assume that both the hidden-layer and output-layer neurons use the sigmoid activation function and that the squared-error loss is used. Show all derivation steps clearly and explain how the output-layer error is propagated backward to the hidden layer for each training sample.																
Long Answer	Consider a feed-forward neural network having an input layer, one hidden layer, and an output layer. The derivation is written for hidden neuron j and output neuron k. The same expressions apply to every hidden neuron in the network. Notation: <table border="1"> <thead> <tr> <th>Symbol</th><th>Meaning</th></tr> </thead> <tbody> <tr> <td>x_i</td><td>Input from input neuron i</td></tr> <tr> <td>v_{ji}</td><td>Weight connecting input neuron i to hidden neuron j</td></tr> <tr> <td>b_j</td><td>Bias of hidden neuron j</td></tr> <tr> <td>a_j</td><td>Net input to hidden neuron j</td></tr> <tr> <td>h_j</td><td>Output of hidden neuron j using sigmoid activation</td></tr> <tr> <td>w_{kj}</td><td>Weight connecting hidden neuron j to output neuron k</td></tr> <tr> <td>b_k</td><td>Bias of output neuron k</td></tr> </tbody> </table>	Symbol	Meaning	x_i	Input from input neuron i	v_{ji}	Weight connecting input neuron i to hidden neuron j	b_j	Bias of hidden neuron j	a_j	Net input to hidden neuron j	h_j	Output of hidden neuron j using sigmoid activation	w_{kj}	Weight connecting hidden neuron j to output neuron k	b_k	Bias of output neuron k
Symbol	Meaning																
x_i	Input from input neuron i																
v_{ji}	Weight connecting input neuron i to hidden neuron j																
b_j	Bias of hidden neuron j																
a_j	Net input to hidden neuron j																
h_j	Output of hidden neuron j using sigmoid activation																
w_{kj}	Weight connecting hidden neuron j to output neuron k																
b_k	Bias of output neuron k																

a_k	Net input to output neuron k
y_k	Predicted output of output neuron k
t_k	Target output for output neuron k
η	Learning rate
E	Squared-error loss

Step 1: Write the sigmoid function and its derivative

The sigmoid activation converts the net input into a value between 0 and 1.

$$\sigma(a) = 1 / (1 + e^{-a})$$

Its derivative can be expressed using the neuron output itself. This form is convenient during backpropagation because the output has already been calculated in the forward pass.

$$\sigma'(a) = \sigma(a)[1 - \sigma(a)]$$

Step 2: Perform forward propagation through the hidden layer

The net input to hidden neuron j is the weighted sum of the network inputs plus the hidden bias:

$$a_j = \sum_i v_{ji}x_i + b_j$$

The output of hidden neuron j is:

$$h_j = \sigma(a_j)$$

Therefore, the derivative of the hidden-neuron output is:

$$\partial h_j / \partial a_j = h_j(1 - h_j)$$

Step 3: Perform forward propagation through the output layer

The hidden outputs act as inputs to the output layer. The net input and output of output neuron k are:

$$a_k = \sum_j w_{kj}h_j + b_k$$

$$y_k = \sigma(a_k)$$

Step 4: Define the loss function

For multiple output neurons, the total squared-error loss for one training example is:

$$E = \frac{1}{2} \sum_k (t_k - y_k)^2$$

The hidden-layer weights and bias are adjusted in the direction that decreases E:

$$v_{ji}(\text{new}) = v_{ji}(\text{old}) - \eta(\partial E / \partial v_{ji})$$

$$b_j(\text{new}) = b_j(\text{old}) - \eta(\partial E / \partial b_j)$$

Step 5: Derive the output-layer error signal

A hidden neuron does not have a target of its own. Therefore, before finding the hidden-layer error, we first determine the error signal at each output neuron.

Apply the chain rule to output neuron k:

$$\partial E / \partial a_k = (\partial E / \partial y_k)(\partial y_k / \partial a_k)$$

The two derivatives are:

$$\partial E / \partial y_k = y_k - t_k$$

$$\partial y_k / \partial a_k = y_k(1 - y_k)$$

Hence:

$$\partial E / \partial a_k = (y_k - t_k) y_k (1 - y_k)$$

It is common to define delta using the negative-gradient sign so that the update can be written with a plus sign:

$$\delta_k = -\partial E / \partial a_k = (t_k - y_k) y_k (1 - y_k)$$

Step 6: Propagate the output error back to hidden neuron j

Changing h_j changes the net input of every output neuron connected to hidden neuron j. Therefore, the effect of h_j on the total error is the sum of its effects on all output neurons:

$$\partial E / \partial h_j = \sum_k (\partial E / \partial a_k) (\partial a_k / \partial h_j)$$

Because:

$$a_k = \sum_j w_{kj} h_j + b_k$$

we have:

$$\partial a_k / \partial h_j = w_{kj}$$

Therefore:

$$\partial E / \partial h_j = \sum_k (\partial E / \partial a_k) w_{kj}$$

Now include the derivative of the hidden sigmoid activation:

$$\partial E / \partial a_j = (\partial E / \partial h_j) (\partial h_j / \partial a_j)$$

$$\partial E / \partial a_j = h_j (1 - h_j) \sum_k w_{kj} (\partial E / \partial a_k)$$

Using $\delta_k = -\partial E / \partial a_k$, the hidden-layer delta is:

$$\delta_j = -\partial E / \partial a_j = h_j (1 - h_j) \sum_k w_{kj} \delta_k$$

Thus, the hidden error is the weighted sum of the output-layer deltas, multiplied by the derivative of the hidden sigmoid activation.

Step 7: Derive the hidden-layer weight update

The weight v_{ji} affects the loss through a_j . Applying the chain rule:

$$\partial E / \partial v_{ji} = (\partial E / \partial a_j) (\partial a_j / \partial v_{ji})$$

From $a_j = \sum_i v_{ji} x_i + b_j$:

$$\partial a_j / \partial v_{ji} = x_i$$

Since $\delta_j = -\partial E / \partial a_j$:

$$\partial E / \partial v_{ji} = -\delta_j x_i$$

Using gradient descent:

$$\Delta v_{ji} = -\eta (\partial E / \partial v_{ji}) = \eta \delta_j x_i$$

$$v_{ji}(\text{new}) = v_{ji}(\text{old}) + \eta \delta_j x_i$$

Step 8: Derive the hidden-layer bias update

The bias is treated as a weight connected to a constant input of 1. Applying the chain rule:

$$\partial E / \partial b_j = (\partial E / \partial a_j) (\partial a_j / \partial b_j)$$

Because:

$$\partial a_j / \partial b_j = 1$$

we obtain:

$$\partial E / \partial b_j = -\delta_j$$

$$\Delta b_j = -\eta (\partial E / \partial b_j) = \eta \delta_j$$

$$b_j(\text{new}) = b_j(\text{old}) + \eta \delta_j$$

	Final expressions $\delta_k = (t_k - y_k) y_k (1 - y_k)$ $\delta_j = h_j (1 - h_j) \sum_k w_{kj} \delta_k$ $v_{ji}(\text{new}) = v_{ji}(\text{old}) + \eta \delta_j x_i$ $b_j(\text{new}) = b_j(\text{old}) + \eta \delta_j$ Note: While calculating δ_j, use the output-layer weights w_{kj} from the forward pass, before those weights are updated for the current training example. Updating w_{kj} first and then using the new values changes the gradient being computed.
Marks	10
-ve applicable?	N
% Applicable	N

Question No	93																										
Question Type																											
Difficulty level	Difficult																										
Chapter	Unit 2																										
Topic	Back-propagation																										
Question	For a multilayer artificial neural network trained using back-propagation and Stochastic Gradient Descent (SGD), derive the weight-update and bias-update expressions for a hidden-layer neuron. Assume that both the hidden-layer and output-layer neurons use the tanh activation function and that the squared-error loss is used. Show all derivation steps clearly and explain how the output-layer error is propagated backward to the hidden layer for each training sample.																										
Long Answer	Consider a feed-forward neural network having an input layer, one hidden layer, and an output layer. The derivation is written for hidden neuron j and output neuron k. Notation <table border="1"> <thead> <tr> <th>Symbol</th><th>Meaning</th></tr> </thead> <tbody> <tr> <td>x_i</td><td>Input from input neuron i</td></tr> <tr> <td>v_{ji}</td><td>Weight connecting input neuron i to hidden neuron j</td></tr> <tr> <td>b_j</td><td>Bias of hidden neuron j</td></tr> <tr> <td>a_j</td><td>Net input to hidden neuron j</td></tr> <tr> <td>h_j</td><td>Output of hidden neuron j using tanh activation</td></tr> <tr> <td>w_{kj}</td><td>Weight connecting hidden neuron j to output neuron k</td></tr> <tr> <td>b_k</td><td>Bias of output neuron k</td></tr> <tr> <td>a_k</td><td>Net input to output neuron k</td></tr> <tr> <td>y_k</td><td>Predicted output of output neuron k</td></tr> <tr> <td>t_k</td><td>Target output for output neuron k</td></tr> <tr> <td>η</td><td>Learning rate</td></tr> <tr> <td>E</td><td>Squared-error loss</td></tr> </tbody> </table> Step 1: Write the tanh function and its derivative The hyperbolic tangent activation maps its input to the interval from -1 to 1.	Symbol	Meaning	x_i	Input from input neuron i	v_{ji}	Weight connecting input neuron i to hidden neuron j	b_j	Bias of hidden neuron j	a_j	Net input to hidden neuron j	h_j	Output of hidden neuron j using tanh activation	w_{kj}	Weight connecting hidden neuron j to output neuron k	b_k	Bias of output neuron k	a_k	Net input to output neuron k	y_k	Predicted output of output neuron k	t_k	Target output for output neuron k	η	Learning rate	E	Squared-error loss
Symbol	Meaning																										
x_i	Input from input neuron i																										
v_{ji}	Weight connecting input neuron i to hidden neuron j																										
b_j	Bias of hidden neuron j																										
a_j	Net input to hidden neuron j																										
h_j	Output of hidden neuron j using tanh activation																										
w_{kj}	Weight connecting hidden neuron j to output neuron k																										
b_k	Bias of output neuron k																										
a_k	Net input to output neuron k																										
y_k	Predicted output of output neuron k																										
t_k	Target output for output neuron k																										
η	Learning rate																										
E	Squared-error loss																										

$$\tanh(a) = (e^a - e^{-a}) / (e^a + e^{-a})$$

Its derivative can be expressed using the output of the neuron:

$$d[\tanh(a)]/da = 1 - \tanh^2(a)$$

Step 2: Perform forward propagation through the hidden layer

The net input to hidden neuron j is:

$$a_j = \sum_i v_{ji}x_i + b_j$$

The hidden-neuron output is:

$$h_j = \tanh(a_j)$$

Therefore:

$$\partial h_j / \partial a_j = 1 - h_j^2$$

Step 3: Perform forward propagation through the output layer

$$a_k = \sum_j w_{kj}h_j + b_k$$

$$y_k = \tanh(a_k)$$

The output activation derivative is:

$$\partial y_k / \partial a_k = 1 - y_k^2$$

Step 4: Define the loss function

$$E = \frac{1}{2} \sum_k (t_k - y_k)^2$$

Gradient descent gives:

$$v_{ji}(\text{new}) = v_{ji}(\text{old}) - \eta (\partial E / \partial v_{ji})$$

$$b_j(\text{new}) = b_j(\text{old}) - \eta (\partial E / \partial b_j)$$

Step 5: Derive the output-layer error signal

For output neuron k:

$$\partial E / \partial a_k = (\partial E / \partial y_k) (\partial y_k / \partial a_k)$$

$$\partial E / \partial y_k = y_k - t_k$$

$$\partial y_k / \partial a_k = 1 - y_k^2$$

Hence:

$$\partial E / \partial a_k = (y_k - t_k)(1 - y_k^2)$$

Using the negative-gradient delta convention:

$$\delta_k = -\partial E / \partial a_k = (t_k - y_k)(1 - y_k^2)$$

Step 6: Propagate the output error back to hidden neuron j

Hidden neuron j contributes to every connected output neuron. Therefore, its contribution to the total loss is found by summing over all output neurons:

$$\partial E / \partial h_j = \sum_k (\partial E / \partial a_k) (\partial a_k / \partial h_j)$$

$$\partial a_k / \partial h_j = w_{kj}$$

$$\partial E / \partial h_j = \sum_k (\partial E / \partial a_k) w_{kj}$$

Now apply the derivative of the hidden tanh activation:

$$\partial E / \partial a_j = (\partial E / \partial h_j) (\partial h_j / \partial a_j)$$

$$\partial E / \partial a_j = (1 - h_j^2) \sum_k w_{kj} (\partial E / \partial a_k)$$

Using $\delta_k = -\partial E / \partial a_k$:

$$\delta_j = -\partial E / \partial a_j = (1 - h_j^2) \sum_k w_{kj} \delta_k$$

	The hidden-layer error is therefore the weighted sum of the output-layer error signals, scaled by the tanh derivative at the hidden neuron. Step 7: Derive the hidden-layer weight update $\partial E / \partial v_{ji} = (\partial E / \partial a_j)(\partial a_j / \partial v_{ji})$ $\partial a_j / \partial v_{ji} = x_i$ $\partial E / \partial v_{ji} = -\delta_j x_i$ $\Delta v_{ji} = -\eta (\partial E / \partial v_{ji}) = \eta \delta_j x_i$ $v_{ji}(\text{new}) = v_{ji}(\text{old}) + \eta \delta_j x_i$ Step 8: Derive the hidden-layer bias update The bias is equivalent to a weight whose input is always 1: $\partial E / \partial b_j = (\partial E / \partial a_j)(\partial a_j / \partial b_j)$ $\partial a_j / \partial b_j = 1$ $\partial E / \partial b_j = -\delta_j$ $\Delta b_j = \eta \delta_j$ $b_j(\text{new}) = b_j(\text{old}) + \eta \delta_j$ Final expressions $\delta_k = (t_k - y_k)(1 - y_k^2)$ $\delta_j = (1 - h_j^2) \sum_k w_{kj} \delta_k$ $v_{ji}(\text{new}) = v_{ji}(\text{old}) + \eta \delta_j x_i$ $b_j(\text{new}) = b_j(\text{old}) + \eta \delta_j$ Note: Use the output-layer weights w_{kj} from the current forward pass when computing δ_j. The hidden delta should be determined before replacing those weights with their newly updated values.
Marks	10
-ve applicable?	N
% Applicable	N

Question No	94
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Optimizers
Question	Define Adagrad and explain its working procedure in detail.
Long Answer	Adagrad (Adaptive Gradient Algorithm) is an optimization algorithm used to train machine learning and deep learning models. Unlike Gradient Descent, which uses the same learning rate for all parameters, Adagrad automatically adjusts the learning rate individually for each parameter, based on the history of that parameter's gradients. Working Principle The algorithm accumulates the squared gradients for each parameter during training. Step 1: Compute the gradient

	For parameter θ, $g_t = \partial J(\theta_t) / \partial \theta$, where $J(\theta)$ is the loss function and g_t is the gradient at iteration t. Step 2: Accumulate the squared gradients $G_t = G_{t-1} + g_t^2$, where G_t is the running sum of squared gradients and $G_0 = 0$. Step 3: Update the parameters $\theta_{t+1} = \theta_t - (\eta / (\sqrt{G_t} + \epsilon)) \cdot g_t$, where η is the initial learning rate and ϵ is a small constant such as 10^{-8} used to avoid division by zero. Step 4: Repeat until convergence. Characteristics - Parameters with frequently large gradients receive smaller effective learning rates, while rarely updated parameters retain larger ones. This makes Adagrad well suited to sparse features. - Since G_t only increases, the effective learning rate decays monotonically and can become too small for learning to continue. RMSProp and Adam address this by using a decaying average instead of a running sum.
Marks	10
-ve applicable?	N
% Applicable	N

Question No	95			
Question Type				
Difficulty level	Easy			
Chapter	2			
Topic	Activation Functions			
Question	Compare the Step, Sigmoid, Tanh and ReLU activation functions in terms of expression, output range and derivative.			
Long Answer	Activation Function	Mathematical Expression	Output Range	Derivative
	Step	$f(x) = 1$ if $x \geq 0$, else 0	{0, 1}	0 for $x \neq 0$; undefined at $x = 0$
	Sigmoid	$f(x) = 1 / (1 + e^{-x})$	(0, 1)	$f(x)(1 - f(x))$; maximum 0.25 at $x = 0$
	Tanh	$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$	(-1, +1)	$1 - f(x)^2$; maximum 1 at $x = 0$
	ReLU	$f(x) = \max(0, x)$	[0, ∞)	0 for $x < 0$, 1 for $x > 0$; undefined at $x = 0$
	Observations			
	- Step is not differentiable in any useful sense, so it cannot be used with backpropagation. - Sigmoid and Tanh both saturate at their extremes, where the derivative approaches 0, which causes vanishing gradients in deep networks. Tanh is the better of the two because it is zero-centered and its maximum derivative is 1 rather than 0.25. - ReLU keeps a constant derivative of 1 over the whole positive half-line, so it neither saturates nor vanishes for active neurons, which is why it is the default in hidden layers.			
Marks	10			
-ve applicable?	N			
% Applicable	N			

Question No	96
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Optimizers
Question	Explain the complete working procedure of Stochastic Gradient Descent (SGD), from the initialization of the parameters through to the stopping criteria.
Long Answer	Stochastic Gradient Descent is an optimization algorithm that updates the model parameters using the gradient computed from a single randomly selected training sample at each iteration, rather than from the whole dataset. 1. Initialize the model parameters Initialize the parameters (weights and biases) θ randomly or using a suitable initialization technique such as Xavier or He initialization. 2. Shuffle the training dataset Randomly shuffle the training samples at the beginning of each epoch, to eliminate any bias caused by the original ordering of the data. This ensures that each sample has an equal probability of influencing the parameter updates. 3. Select a single training sample At iteration t, select one training sample (x_i, y_i) from the shuffled dataset. 4. Forward propagation Pass the selected sample through the model to compute the predicted output $\hat{y}_i$. 5. Compute the loss function Evaluate the loss for the selected sample using an appropriate loss function, such as Mean Squared Error for regression or Cross-Entropy Loss for classification: $L_i(\theta) = L(y_i, \hat{y}_i)$. 6. Compute the gradient Calculate the gradient of the loss with respect to the model parameters: $\nabla L_i(\theta) = \partial L_i(\theta) / \partial \theta$. Since the gradient is computed from only one sample, it is an unbiased but noisy estimate of the true gradient. 7. Update the model parameters Immediately update the parameters using the SGD update rule: $\theta_{t+1} = \theta_t - \eta \nabla L_i(\theta)$, where η is the learning rate. 8. Repeat for all training samples Repeat steps 3 to 7 for every training sample in the dataset. Processing all N samples once constitutes one epoch, resulting in N parameter updates per epoch. 9. Repeat until convergence Continue training over multiple epochs until one of the stopping criteria is satisfied: • The loss function converges. • The maximum number of epochs is reached. • Validation accuracy no longer improves (early stopping).
Marks	10
-ve applicable?	N
% Applicable	N

Question No	97
Question Type	
Difficulty level	Medium
Chapter	2
Topic	Support Vector Machines
Question	Explain the following properties of the SVM margin, giving a justification for each.

	(a) Why only the support vectors determine the margin. (2 Marks) (b) The effect of removing a sample that is not a support vector. (2 Marks) (c) The effect of removing a support vector. (2 Marks) (d) Why the margin can be read as a measure of the classifier's confidence. (2 Marks) (e) How the margin is related to the norm of the weight vector. (2 Marks)
Long Answer	(a) Only the support vectors determine the margin (2 Marks) - The margin is the widest empty band that can be placed around the boundary, so its width is limited by the samples nearest to the boundary on each side. - Those samples are exactly the support vectors. Every other sample lies strictly outside the band and places no restriction on its width. (b) Removing a sample that is not a support vector (2 Marks) - Such a sample satisfies $y f(x) > 1$, so it imposes no active restriction. - The widest empty band is unchanged, so the hyperplane, the margin and the support vectors are exactly as before. An SVM trained on only its support vectors gives an identical classifier. (c) Removing a support vector (2 Marks) - A support vector is an active constraint. Removing it lets the band widen until it meets the next nearest sample. - The hyperplane generally shifts, and the margin either increases or stays the same – it can never decrease, since every remaining constraint was already satisfied. (d) The margin as a measure of confidence (2 Marks) - For any sample, $y f(x)/\ w\$ is its perpendicular distance from the boundary, so a distant sample would need a large perturbation before its predicted class changed. - The margin is the smallest such distance over the training set, so a large margin means even the least confident training prediction is comfortably far from the boundary. (e) Relationship between the margin and $\ w\$ (2 Marks) - In canonical form the closest samples satisfy $w \cdot x + b = 1$, so each supporting hyperplane lies $1/\ w\$ from the boundary. - The two supporting hyperplanes are on opposite sides, so the full margin is $2/\ w\$. The margin is inversely proportional to $\ w\$: making the margin as large as possible is the same as making $\ w\$ as small as possible.
Marks	10
-ve applicable?	N
% Applicable	N

Question No	98
Question Type	
Difficulty level	Difficult
Chapter	2
Topic	Support Vector Machines
Question	An SVM trained on a two-dimensional dataset produces the hyperplane $w = (2, -1)$ and $b = -3$, in canonical form. Compute the following: - Write the equations of the decision boundary and of the two supporting hyperplanes. (2 Marks) - Calculate the margin of the classifier. (2 Marks) - Sample $E(3, 1)$ has the label $y = +1$. Determine whether it is correctly classified and whether it is a support vector. (2 Marks) - Sample $F(1, 0)$ has the label $y = -1$. Determine its position relative to the margin and whether it is a support vector. (2 Marks)

	5. Suppose the support vectors that currently limit the margin are moved farther apart in the normal (class-separation) direction, while no other sample becomes the new closest point. Explain the effect on the margin and why a wider margin can improve generalization. (2 Marks)
Long Answer	1: Equations of the hyperplanes (2 Marks) Decision boundary: $2x_1 - x_2 - 3 = 0$. In canonical form the support vectors satisfy $w \cdot x + b = \pm 1$, so the supporting hyperplanes are: • Positive class: $2x_1 - x_2 = 4$ • Negative class: $2x_1 - x_2 = 2$ 2: Margin (2 Marks) $\ w\ = \sqrt{2^2 + (-1)^2} = \sqrt{5} = 2.236$ Margin = $2/\ w\ = 2/2.236 = 0.894$ units. The distance from the boundary to each supporting hyperplane is $1/\ w\ = 0.447$. 3: Classification of E(3, 1) (2 Marks) $f(E) = 2(3) - 1 - 3 = 2$, so $y f(E) = (+1)(2) = 2$. $f(E) > 0$ gives the predicted class +1, which matches the label, so E is correctly classified. Since $y f(E) = 2 > 1$, E lies strictly outside the margin and is not a support vector. Removing it would not change the boundary. 4: Position of F(1, 0) (2 Marks) $f(F) = 2(1) - 0 - 3 = -1$, so $y f(F) = (-1)(-1) = 1$. F is correctly classified, and since $y f(F) = 1$ exactly it lies on the negative supporting hyperplane $2x_1 - x_2 = 2$. It is therefore a support vector. Moving or removing an active support vector can change the optimal boundary, although a redundant or degenerate support vector need not change it. 5: Effect of separating the closest samples (2 Marks) • Under the stated condition, moving the margin-limiting support vectors farther apart in the normal direction allows a wider empty band, so the maximum margin increases and the optimal $\ w\$ decreases. Moving points farther apart in an arbitrary direction would not, by itself, guarantee a larger SVM margin. • A wider margin increases the minimum distance of the limiting training samples from the boundary, so larger perturbations can be tolerated before those samples cross the decision boundary. • A larger margin also restricts the set of admissible separating hyperplanes, which acts as capacity control and can improve generalization.
Marks	10
-ve applicable?	N
% Applicable	N

Question No	99
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain SVM as a binary classifier. Describe the primal formulation of Hard-Margin SVM, the geometric interpretation of the margin, and derive the margin width expression $2/\ w\ $.
Long Answer	SVM as a binary classifier: 1. Support Vector Machine is a supervised learning algorithm that separates two classes by finding the hyperplane $w^T x + b = 0$ that maximises the distance (margin) to the nearest points of each class. 2. A new point x is classified as positive if $w^T x + b > 0$ and negative if $w^T x + b < 0$. Primal formulation (Hard Margin): $\min_{w,b} \frac{1}{2} \ w\ ^2 \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, m$

	1. $y_i \in \{+1, -1\}$ is the class label and x_i is the i-th training sample. Geometric interpretation of the margin: 1. The decision boundary $w^T x + b = 0$ lies midway between two parallel "gutter" hyperplanes $w^T x + b = 1$ and $w^T x + b = -1$, on which the support vectors lie. 2. The margin is the perpendicular distance between these two gutters; SVM chooses w, b so that this distance is as large as possible while still classifying every point correctly. Deriving the margin width $2/\ w\$: 1. Take any point x^+ on the gutter $w^T x + b = 1$ and the point x^- on $w^T x + b = -1$ obtained by moving from x^+ in the direction of the unit normal $w/\ w\$. 2. Write $x^- = x^+ - t \cdot (w/\ w\)$ for some distance t, and substitute into $w^T x^- + b = -1$: $w^T \left(x^+ - t \cdot \frac{w}{\ w\ } \right) + b = -1$ $w^T x^+ - t \cdot \frac{w^T w}{\ w\ } + b = -1$ 3. Since $w^T x^+ + b = 1$, this simplifies to: $1 - t \cdot \ w\ = -1 \Rightarrow t = \frac{2}{\ w\ }$ The distance t between the two gutters is exactly the margin width, giving the result: $\text{Margin Width} = \frac{2}{\ w\ }$ This shows that maximising the margin is equivalent to minimising $\ w\$, or equivalently $\frac{1}{2} \ w\ ^2$, which is the SVM primal objective.
Marks	10
-ve applicable?	N
% Applicable	N

Question No	100
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain the working of Soft-Margin SVM for classification. Discuss the role of slack variables and the regularization parameter C , and write its complete primal optimization formulation.
Long Answer	Motivation: 1. Real-world datasets are rarely perfectly linearly separable — a few points may be noisy, mislabeled, or simply overlap with the other class. Hard-Margin SVM cannot handle this, since it demands $y_i(w^T x_i + b) \geq 1$ for every point with no exceptions. Slack variables: 1. Soft-Margin SVM introduces a slack variable $\xi_i \geq 0$ for each training point, relaxing the constraint to: $y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$ 2. $\xi_i = 0 \rightarrow$ the point satisfies the original hard-margin constraint.

	3. $0 < \xi_i \leq 1 \rightarrow$ the point is correctly classified but lies inside the margin. 4. $\xi_i > 1 \rightarrow$ the point is misclassified by the decision boundary. Regularization parameter C: C controls the trade-off between a wide margin and a small total slack (i.e. few violations): - Small C: more importance is given to maximising the margin; more violations are tolerated $\rightarrow$ wider margin, better generalisation, risk of underfitting if too small. - Large C: more importance is given to minimising misclassification; fewer violations are tolerated $\rightarrow$ narrower margin, behaviour approaches Hard-Margin SVM, risk of overfitting on noisy data. Complete primal optimization problem: $\min_{w,b,\xi} \frac{1}{2} \ w\ ^2 + C \sum_i \xi_i$ subject to $y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, m$ This remains a convex quadratic optimisation problem and is solved via its Lagrangian dual, which additionally reveals which points are "bound" ($\alpha_i = C$, margin-violating) versus "unbound" ($0 < \alpha_i < C$, exactly on the margin) support vectors.
Marks	10
-ve applicable?	N
% Applicable	N

Question No	101
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	A dataset contains a few noisy or mislabeled points, making it not perfectly linearly separable. Explain why Hard-Margin SVM fails in this case, and describe how Soft-Margin SVM addresses the problem. Discuss the effect of small versus large C , with an example.
Long Answer	Why Hard-Margin SVM fails: - Hard-Margin SVM requires every training point to satisfy $y_i(w^T x_i + b) \geq 1$ with no exceptions. - If even one point is noisy or mislabeled and falls on the wrong side of the boundary, no hyperplane can satisfy this constraint for all points — the optimisation problem becomes infeasible. - Even if a separating hyperplane technically exists, an outlier close to the other class forces the margin to shrink drastically to avoid violating the constraint, producing a hyperplane that overfits that single point and generalises poorly. How Soft-Margin SVM solves this: - Soft-Margin SVM relaxes the constraint using slack variables $\xi_i \geq 0$: $y_i(w^T x_i + b) \geq 1 - \xi_i$ - and modifies the objective to penalise the total slack: $\min \frac{1}{2} \ w\ ^2 + C \sum_i \xi_i$ - This lets a small number of points violate the margin (or even be

	misclassified) at a controlled cost, rather than making the whole problem infeasible or forcing an unnaturally narrow margin. Effect of C: 1. Small $C \rightarrow$ violations are "cheap", so the optimiser accepts a few slack points in exchange for a much wider margin. This is desirable when the violations are genuinely noise, since it prevents the model from overfitting to a handful of bad points. 2. Large $C \rightarrow$ violations are "expensive", so the optimiser shrinks the margin to fit almost all points correctly, behaving close to Hard-Margin SVM and risking overfitting if the violating points really are noise rather than genuine class structure. Example: 1. Consider a medical diagnosis dataset where two patient records were mislabeled due to a data-entry error. With a large C, the SVM would twist the decision boundary sharply to correctly classify these two records, producing a narrow, unstable margin that performs poorly on new patients. 2. With a smaller, well-tuned C, the model instead treats these two records as acceptable margin violations, keeping a wide, stable margin that reflects the true underlying separation between the classes and generalises better to unseen patients.
Marks	10
-ve applicable?	N
% Applicable	N

Question No	102
Question Type	
Difficulty level	Difficult
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Starting from the primal problem of Hard-Margin SVM, derive the complete Lagrangian dual formulation, including the stationarity conditions, and show how the decision function for a new point is expressed purely in terms of the dual variables and dot products. Extend the discussion to Soft-Margin SVM, emphasizing the box constraint on the Lagrange multipliers.

Long Answer

Part A — Hard-Margin SVM

Primal:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, m$$

Lagrangian ($\alpha_i \geq 0$):

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_i \alpha_i [y_i(w^T x_i + b) - 1]$$

Stationarity conditions:

1. $\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_i \alpha_i y_i x_i$
2. $\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_i \alpha_i y_i = 0$

Substituting these back into L (eliminating w and b):

$$L(\alpha) = \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

Dual problem:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i^T x_j) \quad \text{s.t.} \quad \alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0$$

Decision function for a new point x : once the dual is solved, b is recovered from any support vector ($\alpha_i > 0$) using $y_i(w^T x_i + b) = 1$, and classification of a new point uses only dot products:

$$f(x) = \text{sign} \left(\sum_i \alpha_i y_i (x_i^T x) + b \right)$$

1. Because the data enters the dual and the decision function only through the dot product $x_i^T x_j$ (or $x_i^T x$), this term can be replaced by a kernel $K(x_i, x_j)$ to obtain non-linear decision boundaries without ever computing the mapping $\phi(x)$ explicitly — the essence of the kernel trick.

Part B — Extension to Soft-Margin SVM

Primal:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Lagrangian ($\alpha_i, \mu_i \geq 0$):

$$L = \frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(w^T x_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i$$

Stationarity:

1. $\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_i \alpha_i y_i x_i$ (identical form to Hard Margin)
2. $\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_i \alpha_i y_i = 0$
3. $\frac{\partial L}{\partial \xi_i} = 0 \Rightarrow \alpha_i + \mu_i = C$

Since $\mu_i \geq 0$, the last condition forces $\alpha_i \leq C$, which combined with $\alpha_i \geq 0$ gives the **Box Constraint**:

$$0 \leq \alpha_i \leq C$$

Resulting dual:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i^T x_j) \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0$$

	Significance of the box constraint: 1. The dual objective and the decision function $f(x) = \text{sign}(\sum \alpha_i y_i (x_i^T x) + b)$ keep exactly the same form as Hard-Margin SVM — the only change is that α_i is now capped at C. 2. $\alpha_i = 0$: ordinary point, correctly classified outside the margin. 3. $0 < \alpha_i < C$: unbound support vector, lying exactly on the margin ($\xi_i = 0$). 4. $\alpha_i = C$: bound support vector, violating the margin or misclassified ($\xi_i > 0$); its contribution to w is capped so it cannot dominate the solution. This shows that Soft-Margin SVM is a direct, minimal generalisation of Hard-Margin SVM in the dual space: the same kernel-friendly, dot-product-based structure is retained, with robustness to noisy data introduced purely by bounding each α_i above by C.
Marks	10
-ve applicable?	N
% Applicable	N

Unit 2 – 2 Marks Descriptive Question Bank

Question No	103
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Multi-layer networks
Question	A neural network has two inputs x_1 and x_2 and three ReLU hidden neurons. The hidden-layer weights (x_1, x_2) are h_1 : (-1, -2), h_2 : (-2, 2), and h_3 : (3, -1). The output neuron has no activation function and uses weights (4, -2, 1) from (h_1, h_2, h_3). There are no bias terms. Calculate the output for $x = (3, 2)$.
Long Answer	1. $z_1 = (-1)(3) + (-2)(2) = -7$, so $h_1 = \text{ReLU}(-7) = 0$. 2. $z_2 = (-2)(3) + (2)(2) = -2$, so $h_2 = \text{ReLU}(-2) = 0$. 3. $z_3 = (3)(3) + (-1)(2) = 7$, so $h_3 = \text{ReLU}(7) = 7$. 4. Output = $4h_1 - 2h_2 + h_3 = 4(0) - 2(0) + 7 = 7$.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	104
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Back-propagation
Question	Explain the most likely effect of each of the following learning-rate conditions: (i) excessively large learning rate, (ii) extremely small learning rate, (iii) appropriate learning rate, and (iv) decreasing learning-rate schedule.

Long Answer	1. Excessively large learning rate: the loss may oscillate or diverge because updates overshoot the minimum. 2. Extremely small learning rate: training progresses very slowly. 3. Appropriate learning rate: training converges steadily toward a low-loss region. 4. Decreasing learning-rate schedule: training begins with larger steps and later uses smaller steps.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	105
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Perceptrons
Question	A binary threshold neuron computes $z = -8 + 5x_1 + 5x_2$, where $x_1, x_2 \in \{0, 1\}$, and outputs 1 if $z > 0$ and 0 otherwise. Determine the logical function implemented by the neuron.
Long Answer	1. For (0,0), (0,1), and (1,0), $z \leq 0$, so the output is 0. 2. For (1,1), $z = -8 + 5 + 5 = 2 > 0$, so the output is 1. Therefore, the neuron implements the AND function.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	106
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Multi-layer networks
Question	A fully connected neural network has a 5-dimensional input, one hidden layer with 15 neurons, and an output layer with 3 neurons. Calculate the total number of trainable parameters, assuming there are no bias parameters.
Long Answer	1. Input-to-hidden weights = $5 \times 15 = 75$. 2. Hidden-to-output weights = $15 \times 3 = 45$. 3. Total trainable parameters = $75 + 45 = 120$.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	107
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Multi-layer networks
Question	A fully connected neural network has 10 input units, two hidden layers with 5 neurons in each layer, and an output layer with 10 neurons. How many weights are updated in one training iteration? Exclude bias parameters.
Long Answer	1. Input to hidden layer 1: $10 \times 5 = 50$ weights. 2. Hidden layer 1 to hidden layer 2: $5 \times 5 = 25$ weights. 3. Hidden layer 2 to output: $5 \times 10 = 50$ weights. Total weights updated = $50 + 25 + 50 = 125$.

Marks	2
-ve applicable?	N
% Applicable	N
Question No	108
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Multi-layer networks
Question	Show that two consecutive neural-network layers using only linear activations are equivalent to a single linear layer. Give the expressions for the combined weight and bias.
Long Answer	1. For two linear layers, $y = W_2(W_1x + b_1) + b_2$. 2. Rearranging, $y = (W_2W_1)x + (W_2b_1 + b_2)$. Hence, the equivalent single layer has $W = W_2W_1$ and $b = W_2b_1 + b_2$.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	109
Question Type	
Difficulty level	Difficult
Chapter	Unit 2
Topic	Back-propagation
Question	During backpropagation through a sigmoid activation $\sigma(x) = 1/(1 + e^{-x})$, state what happens to the magnitude and sign of the incoming gradient. Justify your answer.
Long Answer	1. $\sigma'(x) = \sigma(x)[1 - \sigma(x)]$ is positive for finite x , so multiplying by $\sigma'(x)$ preserves the gradient sign. 2. The maximum value of $\sigma'(x)$ is 0.25, so $ g_{\text{new}} = g \sigma'(x) \leq 0.25 g $. Thus, the gradient magnitude decreases while its sign is preserved.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	110
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Support Vector Machines
Question	Two separating hyperplanes are obtained for the same dataset. Hyperplane A has a margin of 2.5 and Hyperplane B has a margin of 4.0. Which hyperplane will an SVM select? Justify your answer.
Long Answer	1. SVM selects the hyperplane with the maximum margin. 2. Since $4.0 > 2.5$, Hyperplane B is selected. A larger margin generally gives better generalization.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	111
Question Type	
Difficulty level	Easy

Chapter	2
Topic	Optimizers
Question	SGD oscillates significantly while minimizing a loss function in a narrow valley. Name the optimization technique designed to reduce such oscillations and explain how it works.
Long Answer	1. Momentum Optimization is used to reduce such oscillations. 2. It accumulates gradients from previous iterations; consistent components are reinforced, while alternating components tend to cancel, damping oscillation across the valley.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	112
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Optimizers
Question	Compare Stochastic Gradient Descent (SGD) and Batch Gradient Descent in terms of update cost and stability.
Long Answer	1. SGD uses one training sample per update, so each update is computationally cheap but noisy. 2. Batch Gradient Descent uses the full dataset per update, so its gradient estimate is more stable but each update is more computationally expensive.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	113
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Support Vector Machines
Question	Which training data points determine the position and orientation of the optimal SVM hyperplane? Explain briefly.
Long Answer	1. The support vectors determine the optimal hyperplane. 2. They are the training samples nearest to the decision boundary and fix its position and orientation.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	114
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Activation Functions
Question	Differentiate the Sigmoid and ReLU activation functions with respect to output range, saturation, and gradient behavior.

Long Answer	1. Sigmoid outputs values in $(0, 1)$ and saturates at both extremes, which can cause vanishing gradients. 2. ReLU outputs values in $[0, \infty)$ and has gradient 1 for positive inputs; it is also cheaper to compute than Sigmoid.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	115
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Activation Functions
Question	Which commonly used activation function is zero-centered? State its output range and one training advantage.
Long Answer	1. Tanh is zero-centered and has output range $(-1, +1)$. 2. Because its activations are not all of one sign, it can generally improve convergence compared with Sigmoid.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	116
Question Type	
Difficulty level	Easy
Chapter	2
Topic	Optimizers
Question	What two optimization ideas are combined in the Adam optimizer? Briefly state the role of each.
Long Answer	1. Adam combines momentum, through a first-moment estimate of the gradient. 2. It also uses adaptive per-parameter learning rates, through a second-moment estimate of the gradient.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	117
Question Type	
Difficulty level	Medium
Chapter	2
Topic	Support Vector Machines
Question	For a support vector in a canonical-form SVM, what value does $y_i(w \cdot x_i + b)$ take? Explain briefly.
Long Answer	1. For every support vector, $y_i(w \cdot x_i + b) = 1$. 2. Canonical scaling makes the closest samples lie on the margin boundaries; other correctly classified samples have values greater than 1.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	118
Question Type	
Difficulty level	Medium
Chapter	2
Topic	Optimizers
Question	Why does the effective learning rate in Adagrad decrease as training progresses?
Long Answer	1. Adagrad accumulates the sum of past squared gradients, G_t , and this quantity grows monotonically. 2. Therefore, the effective step $\eta/(\sqrt{G_t} + \epsilon)$ becomes smaller as training progresses and may eventually make learning very slow.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	119
Question Type	
Difficulty level	Medium
Chapter	2
Topic	Activation Functions
Question	A ReLU neuron receives a negative pre-activation value at every training iteration. What happens to the neuron, and what is this problem called?
Long Answer	1. For negative inputs, the derivative of ReLU is 0, so no gradient flows through the neuron. 2. Its weights stop updating and it may remain inactive. This is called the dying ReLU problem.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	120
Question Type	
Difficulty level	Difficult
Chapter	2
Topic	Optimizers
Question	A Momentum-based optimizer starts with $v_0 = 0$, $\beta = 0.9$, and $\eta = 0.1$. The gradient is 4 at both the first and second iterations. Using $v_t = \beta v_{t-1} + \eta \nabla J(\theta)$, calculate v_2 .
Long Answer	1. $v_1 = 0.9(0) + 0.1(4) = 0.4$. 2. $v_2 = 0.9(0.4) + 0.1(4) = 0.36 + 0.4 = 0.76$.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	121
Question Type	
Difficulty level	Difficult
Chapter	2
Topic	Support Vector Machines
Question	In a linearly separable dataset, the closest positive support vector is (5, 5) and the closest negative support vector is (2, 1). The maximum-margin hyperplane is the perpendicular bisector of the segment joining them. Calculate the margin.

Long Answer	1. Distance between the two support vectors = $\sqrt{(5 - 2)^2 + (5 - 1)^2} = \sqrt{25} = 5$. 2. The hyperplane bisects this segment; each support vector is 2.5 from it. The margin, i.e. the distance between the supporting hyperplanes, is 5.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	122
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	State the prediction rule for a new data point in a Hard Margin SVM using $W^T X + b$ and the two margin boundaries.
Long Answer	1. Predict the positive class when $W^T X + b \geq 1$. 2. Predict the negative class when $W^T X + b \leq -1$. These conditions place the point outside the corresponding margin boundary.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	123
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	What are support vectors, and how do they determine the optimal SVM hyperplane?
Long Answer	1. Support vectors are the training points lying on the margin boundaries and closest to the decision boundary. 2. They uniquely determine the position of the optimal hyperplane.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	124
Question Type	
Difficulty level	Easy
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	State the main KKT conditions used in a hard-margin SVM and mention their direct implication for support vectors.
Long Answer	1. The KKT conditions include primal feasibility, dual feasibility, complementary slackness, and stationarity. 2. Complementary slackness implies that support vectors have non-zero Lagrange multipliers.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	125
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	State the role or meaning of each of the following in SVM: support vector, weight vector, slack variable, and decision boundary.
Long Answer	1. Support vector: the closest point to the decision boundary. 2. Weight vector: normal to the hyperplane. 3. Slack variable: represents margin violation. 4. Decision boundary: $W^T X + b = 0$.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	126
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain what $\xi = 0$, $0 < \xi < 1$, and $\xi > 1$ indicate in a Soft Margin SVM, and state the role of C.
Long Answer	1. $\xi = 0$: the point is correctly classified outside the margin. 2. $0 < \xi < 1$: the point is correctly classified inside the margin. 3. $\xi > 1$: the point is misclassified. 4. C is the regularization parameter controlling the trade-off between margin and misclassification.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	127
Question Type	
Difficulty level	Medium
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	State the formula for the Hard Margin SVM margin width and explain its relationship with $\ W\ $ and the objective $\frac{1}{2} \ W\ ^2$.
Long Answer	1. Margin width = $2/\ W\$, so the margin is inversely proportional to the magnitude of the weight vector. 2. Therefore, minimizing $\frac{1}{2} \ W\ ^2$ is equivalent to maximizing the margin.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	128
Question Type	
Difficulty level	Difficult

Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	Explain how increasing the regularization parameter C affects the misclassification penalty and the margin in a Soft Margin SVM.
Long Answer	1. Increasing C increases the penalty for misclassification. 2. The model therefore allows fewer violations and generally produces a narrower margin, approaching Hard Margin SVM behavior.
Marks	2
-ve applicable?	N
% Applicable	N

Question No	129
Question Type	
Difficulty level	Difficult
Chapter	Unit 2
Topic	Support Vector Machines (SVM)
Question	An SVM-based spam detector is trained on data containing a few incorrectly labeled emails. Enforcing zero training error gives a very narrow margin and poor performance on new emails. Which SVM approach should be used, and why?
Long Answer	1. Use a Soft Margin SVM with an appropriate value of C. 2. It allows limited misclassification of noisy or mislabeled samples while maintaining a wider margin, which can improve generalization.
Marks	2
-ve applicable?	N
% Applicable	N